
UNCERTAINTY IN LLM AGENTS: A COMPREHENSIVE SURVEY

**Kaiwen Shi^{1,*}, Zheyuan Zhang^{1,*}, Caiqi Zhang², Zheyuan Liu¹, Zehong Wang¹,
Shanglin Wu³, Haoran Wang³, Kai Shu³, Han Bao¹, Xiaoyu Ma⁴, Yanfang Ye^{1,†}**

¹University of Notre Dame

²University of Cambridge

³Emory University

⁴National University of Singapore

*Equal contribution

†Corresponding author

ABSTRACT

Large language models (LLMs) and LLM-based agents increasingly support consequential decisions and interact with external systems. As these systems move from generating responses to executing actions, however, a confident but unsupported commitment can become a tool call, environmental action, or state update whose effects propagate through later decisions. This risk makes uncertainty essential for characterizing when a model or agent approaches the boundary of reliable commitment. Because uncertainty must ultimately support decisions rather than remain an isolated measurement, this survey organizes the field through a general lifecycle of *Quantification, Calibration, and Utilization*. Within this lifecycle, Quantification represents uncertainty-bearing evidence, whereas Calibration relates that evidence to specified reliability outcomes; together, they provide the basis for Utilization, which determines how uncertainty changes inference, action, or learning. This final stage becomes especially consequential in LLM-based agents because each uncertainty-conditioned intervention can reshape the observations, states, and decisions that follow. Accordingly, this survey analyzes agent uncertainty utilization as closed-loop trajectory control across State Construction, Action Selection, Execution & Feedback, and State Update. Along this loop, uncertainty can repair an insufficient decision state, regulate commitment and computational effort, adapt execution to newly observed consequences, and determine which beliefs or learning signals persist. Because an uncertainty signal may be produced at one stage yet consumed at another, existing methods are further organized by signal granularity, uncertainty source stage, and application stage, thereby separating where uncertainty is computed from where it changes agent behavior. This organization, in turn, supports a broader discussion of industrial and consumer deployment, the positive and negative social impacts of uncertainty-aware agents, and open problems in uncertainty propagation, temporal calibration, causal credit assignment, dependence-aware coordination, and intervention-aware evaluation. Overall, this survey shifts attention from estimating uncertainty in isolation to using it to prevent unsupported commitments from propagating through agent trajectories.

Contents

1	Introduction	4
2	The Scope of Uncertainty	6
2.1	Uncertainty: Towards the boundary of reliable commitment	6
2.2	Conceptual Boundaries of Uncertainty	8
2.3	A Lifecycle View of Uncertainty in LLM-based Agent	9
3	Background: Uncertainty Quantification in LLMs	10
3.1	Token-Level Uncertainty Quantification	11
3.1.1	Divergence-Based Methods	11
3.1.2	Distance-Based Methods	12
3.1.3	Density-Based Methods	12
3.1.4	Mutual-Information-Based Methods	12
3.1.5	Probability-Based Methods	13
3.1.6	Entropy-Based Methods	13
3.2	Response-Level Uncertainty Quantification	13
3.2.1	Consistency-Based Methods	14
3.2.2	Sampling-Based Semantic Uncertainty	14
3.2.3	Self-Verbalized Uncertainty	15
3.2.4	Reward-Model Uncertainty	15
4	Uncertainty Quantification in LLM Agents: From Output Distributions to Interactive Trajectory	16
4.1	What Changes in Agents From Generated Outputs to Interactive Trajectories	16
4.2	Within-Rollout Quantification	17
4.2.1	Aggregation-Based Methods	17
4.2.2	Propagation-Based Methods	18
4.3	Sampling-Based Quantification	18
4.4	Discussion: Input Uncertainty	19
5	Uncertainty Calibration in LLM Agents: Aligning Confidence with Empirical Reliability	20
5.1	What Is New in Agent Uncertainty Calibration?	20
5.2	Test-Time and Runtime Calibration	22
5.3	Training-Time Calibration	23
5.4	Decision-Oriented Calibration	24
5.5	Discussion: What Counts as Calibration?	25
6	Uncertainty Utilization in LLM-Based Agents: Turning Uncertainty into Adaptive Behavior	25

6.1	What Is New in Agent Uncertainty Utilization?	26
6.2	Agent Loop Formulation	26
6.3	Application in State Construction	29
6.3.1	State Sufficiency and Clarification Triggering	29
6.3.2	Value-Aware Information Acquisition	29
6.3.3	Context Selection and Reorganization	30
6.4	Application in Action Selection	30
6.4.1	Commitment Control	31
6.4.2	Uncertainty-Guided Deliberation and Branching	31
6.4.3	Comparative Routing	31
6.5	Application in Execution	32
6.5.1	Feedback-Induced Expansion and Budget Allocation.	33
6.5.2	Feedback-Conditioned Search Adaptation.	33
6.5.3	Targeted Recovery and Sequential Termination.	34
6.6	Application in State Update	34
6.6.1	Runtime Belief and Memory Update.	35
6.6.2	Collective Belief Revision.	35
6.6.3	Credit Assignment under Delayed Outcomes.	36
6.6.4	Uncertainty-Shaped Learning Objectives.	36
6.7	Discussion: Classifying and Evaluating Uncertainty Utilization	37
7	Applications	38
7.1	AI4Science: From Experimental Acquisition to Scientific Claims	38
7.2	Healthcare: Defining the Boundary of Automation	39
7.3	VLA and Robotics: From Multimodal Evidence to Physical Commitment	40
7.4	Software and Web Agents: Clarifying and Verifying Executable Workflows	41
7.5	Conclusion	41
8	Open Problems	42
8.1	Quantification: Which Uncertainty Representations Are Actually Necessary?	42
8.2	Calibration: When Does Better Calibration Matter for Decisions?	43
8.3	Utilization: Does Uncertainty Lead to the Right Behavior?	44

1 Introduction

Artificial intelligence is expanding the range of cognitive and operational work that can be delegated to machines. Foundation models accelerate this shift by providing a common interface for language, knowledge, perception, computation, and external tools. Large language models are therefore moving beyond answer generation. They increasingly serve as agents that interpret open-ended goals, construct plans, invoke tools, interact with users and environments, and revise later decisions from feedback [1–3]. This transition creates new opportunities in scientific discovery, digital services, embodied systems, and other complex workflows. It also changes the central reliability question. The problem is no longer only whether a model can produce a plausible answer. It is whether an autonomous system can recognize when its current evidence does not support its next action [4–6].

For a standalone language model, an error may remain bounded to an incorrect textual response. For an agent, the same output may become a function call, web operation, GUI action, memory update, or embodied command [4, 6, 7]. Execution changes the external environment. It also changes the evidence available to later decisions. An unsupported commitment can therefore redirect an entire trajectory before terminal failure becomes visible [8–10]. Reliable agency must address this risk before the consequences propagate. The agent must determine whether it should proceed, clarify, retrieve, verify, revise, defer, or delegate [11–13].

This requirement places uncertainty at the foundation of reliable autonomy. In this survey, *agent uncertainty* concerns whether an agent’s evolving information state provides sufficient support for a current commitment. It also concerns what response should follow when that support is insufficient. This scope is broader than confidence in a completed answer. An agent acts through a sequence of coupled states, actions, observations, and updates. Uncertainty must therefore be interpreted over an evolving decision process [5, 8, 14]. Its value is also behavioral. A signal that identifies fragility but changes neither inference, action, nor learning remains a diagnostic observation. It does not yet function as a mechanism for reliable agency [4, 12, 15].

Research on this problem now spans probabilistic confidence, token-level entropy, sampling-based disagreement, semantic dispersion, verbalized uncertainty, process critics, trajectory-success estimation, calibration, safe planning, recovery, routing, and uncertainty-aware learning [4, 15, 16]. This breadth creates an organizational problem. These methods do not study the same object. A token-level entropy score measures local predictive dispersion. A response-level consistency score measures variation across generations. A calibrated trajectory score estimates a specified reliability outcome. An uncertainty-triggered fallback policy changes what the agent does. All may be described as uncertainty-aware, yet they address different parts of the decision process. A useful survey must therefore connect the signal being measured, the meaning assigned to that signal, and the behavior that consumes it.

Existing surveys provide strong foundations for the first parts of this problem. They primarily organize the literature around uncertainty-estimation families, uncertainty sources, model access, computational cost, and calibration quality [17–19]. This estimator-centered view is well suited to the central question of standalone LLM uncertainty: how can uncertainty in a generated output be made observable and evaluated? Recent agent-focused work shows that this question must be extended to underspecified inputs, interactive trajectories, tool use, and communication [1, 16]. However, quantification alone does not cover the full uncertainty problem. An informative signal may carry the wrong reliability meaning. A meaningful signal may still have no effect on behavior. Clarification, retrieval, routing, recovery, and learning are therefore not secondary examples of uncertainty estimation. They are distinct stages in how uncertainty becomes useful. We address this gap by shifting the organizing unit from the uncertainty estimator to the lifecycle of the uncertainty signal.

We organize this lifecycle through **Quantification**, **Calibration**, and **Utilization**. **Quantification** makes insufficient support observable. It represents uncertainty through a score, distribution, set, ranking, semantic variation, internal state, or behavioral proxy [4, 8, 20]. **Calibration** determines what that signal means with respect to a specified reliability event. The event may be response correctness, action reliability, terminal trajectory success, or future success from the current state [9, 15, 21]. **Utilization** converts the interpreted signal into an intervention. The intervention may change information acquisition, reasoning, execution, coordination, or learning [7, 11, 12]. The three stages form a dependency chain. Without Quantification, uncertainty remains hidden. Without Calibration, its decision meaning remains unclear. Without Utilization, it cannot

change the trajectory. This survey covers the complete lifecycle, with particular emphasis on Utilization because an agent’s response to uncertainty changes the evidence and decisions that follow.

Scope and lifecycle (Section 2). Section 2 establishes the conceptual basis of the survey. It first defines uncertainty through the boundary of reliable commitment. Under this view, uncertainty describes whether a model or agent can support an answer, action, reasoning path, tool invocation, or state update under the current condition. The section then distinguishes input, output, and irreducible uncertainty. It also separates uncertainty from confidence, reliability, and trustworthiness. These concepts interact, but they refer to different objects. Confidence represents support for a candidate commitment. Reliability describes dependable system behavior. Trustworthiness includes broader properties such as safety, accountability, and governance. Section 2 finally formalizes the Quantification–Calibration–Utilization lifecycle used throughout the survey.

LLM uncertainty quantification (Section 3). Section 3 reviews the uncertainty signals inherited from standalone LLMs. Token-level methods derive uncertainty from predictive probabilities, entropy, divergence, representation distance, density, or mutual information. Response-level methods instead use repeated generations, semantic variation, self-assessment, or evaluator disagreement. These signals form the computational foundation of agent uncertainty because an LLM remains the reasoning and decision backbone of many agents. However, they are defined around individual generations. Section 3 therefore establishes what can be transferred from standalone LLMs before the interaction-specific problems are introduced.

Agent uncertainty quantification (Section 4). Section 4 examines what changes when an LLM becomes part of an interactive trajectory. One rollout creates a temporal problem. Local uncertainty may need to be aggregated into a trajectory-level assessment or retained across dependent states. Alternative rollouts create a sampling problem. Uncertainty may then arise from different actions, simulated branches, complete executions, or interacting agents [4, 8, 20]. The section organizes these methods into within-rollout and sampling-based quantification. This separation distinguishes uncertainty inherited from one generation from uncertainty created by sequential interaction.

Uncertainty calibration (Section 5). A quantified signal becomes useful only after its reliability target is specified. Section 5 therefore examines how uncertainty is interpreted and corrected. It first reviews test-time and runtime methods that recalibrate existing signals. It then studies training-time methods that teach models to express or preserve calibrated uncertainty. Finally, it considers decision-oriented calibration, where calibration controls the breadth of an agent’s permitted commitment. Agent calibration extends beyond response correctness. It may target action productivity, prefix-conditioned success, terminal trajectory success, or trajectory safety [9, 11, 15, 21]. The section also examines why tool dependence, policy change, delayed feedback, and communication dependence can invalidate a fixed confidence mapping.

Uncertainty utilization (Section 6). Once uncertainty has been measured and interpreted, the remaining question is what it changes. Section 6 treats Utilization as the use of uncertainty to alter agent behavior. This literature is highly heterogeneous. Some methods repair missing information. Others control commitment, computation, execution, communication, or learning. Organizing these methods by estimator type would hide these behavioral differences. We therefore introduce a four-stage agent-loop taxonomy that classifies each method by where its uncertainty-conditioned intervention enters the interaction. This taxonomy is a classification device rather than a claim that all agents share one architecture. **State Construction** covers interventions that repair or reorganize instructions, observations, retrieved evidence, memory, and context before commitment. **Action Selection** covers interventions that determine whether the agent should commit, deliberate further, or transfer decision authority. **Execution & Feedback** covers interventions that adapt an ongoing trajectory after tools or environments return new consequences. **State Update** covers interventions that determine which beliefs, memories, communication signals, credit assignments, and policy changes persist into later decisions [5, 12, 22, 23]. This organization separates an uncertainty signal’s *source stage* from its *application stage*. It also separates signal granularity from behavioral function. A token-, response-, or trajectory-level signal may support the same intervention, while the same signal may be consumed at several stages. The taxonomy therefore classifies methods by what uncertainty actually changes rather than only by how uncertainty is computed.

Domain applications (Section 7). Methods alone do not show where uncertainty has practical value. Section 7 therefore examines domains in which uncertainty changes a consequential decision. In AI4Science,

it guides experiment, measurement, and candidate selection. In healthcare, it controls abstention, specialist consultation, and professional handoff. In robotics and VLA systems, it affects clarification, planning, grounding, action selection, and physical commitment. In software and web agents, it triggers requirement clarification, verification, additional search, and recovery. This organization allows readers to locate evidence from their own domain. It also distinguishes mature domain-level uncertainty methods from the more recent use of uncertainty inside autonomous agent loops.

Open problems (Section 8). The lifecycle and taxonomy expose several assumptions that current research has not yet established. Section 8 formulates these assumptions as direct research questions. For Quantification, it asks whether turn-level, structured, and multi-agent uncertainty provide decision value beyond simpler global signals. For Calibration, it asks when lower calibration error improves agent control and when metric gains become behaviorally irrelevant. For Utilization, it asks whether clarification, retrieval, and other uncertainty-triggered interventions resolve the underlying failure or merely change its form. The aim is not to prescribe another set of techniques. It is to identify questions whose answers could confirm, limit, or redirect existing lines of research [8, 11, 24].

Taken together, this survey follows one direction from uncertainty evidence to reliable agent behavior. It provides a common vocabulary for uncertainty objects, a comparative map of quantification and calibration methods, and an intervention-centered taxonomy of utilization. It also connects these mechanisms to concrete domains and unresolved research questions. Rather than treating uncertainty as an auxiliary score attached to a completed answer, we position it as a systems-level resource that links evidence, reliability, and intervention throughout an agent trajectory. Reliable autonomy requires agents not only to act, but also to recognize when their current information does not justify further commitment and to restrict, revise, or transfer that commitment before its consequences propagate.

2 The Scope of Uncertainty

Conceptual clarity is necessary because uncertainty is often used together with confidence, trustworthiness, reliability, calibration, and robustness, even though these concepts refer to different objects and levels of analysis. Without a clear scope, uncertainty may be reduced to a confidence score, or confused with system-level trustworthiness. We therefore first position uncertainty relative to the classical aleatoric–epistemic distinction, then formalize it as a model-centered relation to the boundary of reliable commitment. We further distinguish uncertainty from related concepts and introduce the lifecycle view of quantification, calibration, and utilization.

2.1 Uncertainty: Towards the boundary of reliable commitment

In classical machine learning, uncertainty is commonly decomposed into aleatoric uncertainty and epistemic uncertainty. The former arises from irreducible randomness or ambiguity in the data-generating process, while the latter reflects the learner’s lack of knowledge and can in principle be reduced by additional information [159]. Recent surveys on LLM uncertainty often inherit this distinction and adapt it to language generation [17–19]. This view remains useful, however, in the era of LLMs and agents, the scope of uncertainty needs to be calibrated to the behaviors these systems actually perform. Because the relevant question is often not only what the model knows, but what the model can responsibly treat as sufficiently supported. Recent work has therefore discussed uncertainty in relation to input ambiguity, reasoning divergence, decoding variability, semantic variation, tool interaction, and long-horizon trajectories [16, 18, 84]. These developments do not invalidate the classical aleatoric–epistemic distinction. Rather, they suggest that aleatoric and epistemic uncertainty should be treated as source-level categories, while the conceptual scope of uncertainty should be extended from fixed predictions to the broader commitments made by LLMs and agents.

Therefore, in this paper, we define uncertainty as **a measure of the boundary of reliable commitment**, which characterizes the degree to which a model approaches the limit of what it can reliably predict, infer, explain, decide, or act upon under a given condition.

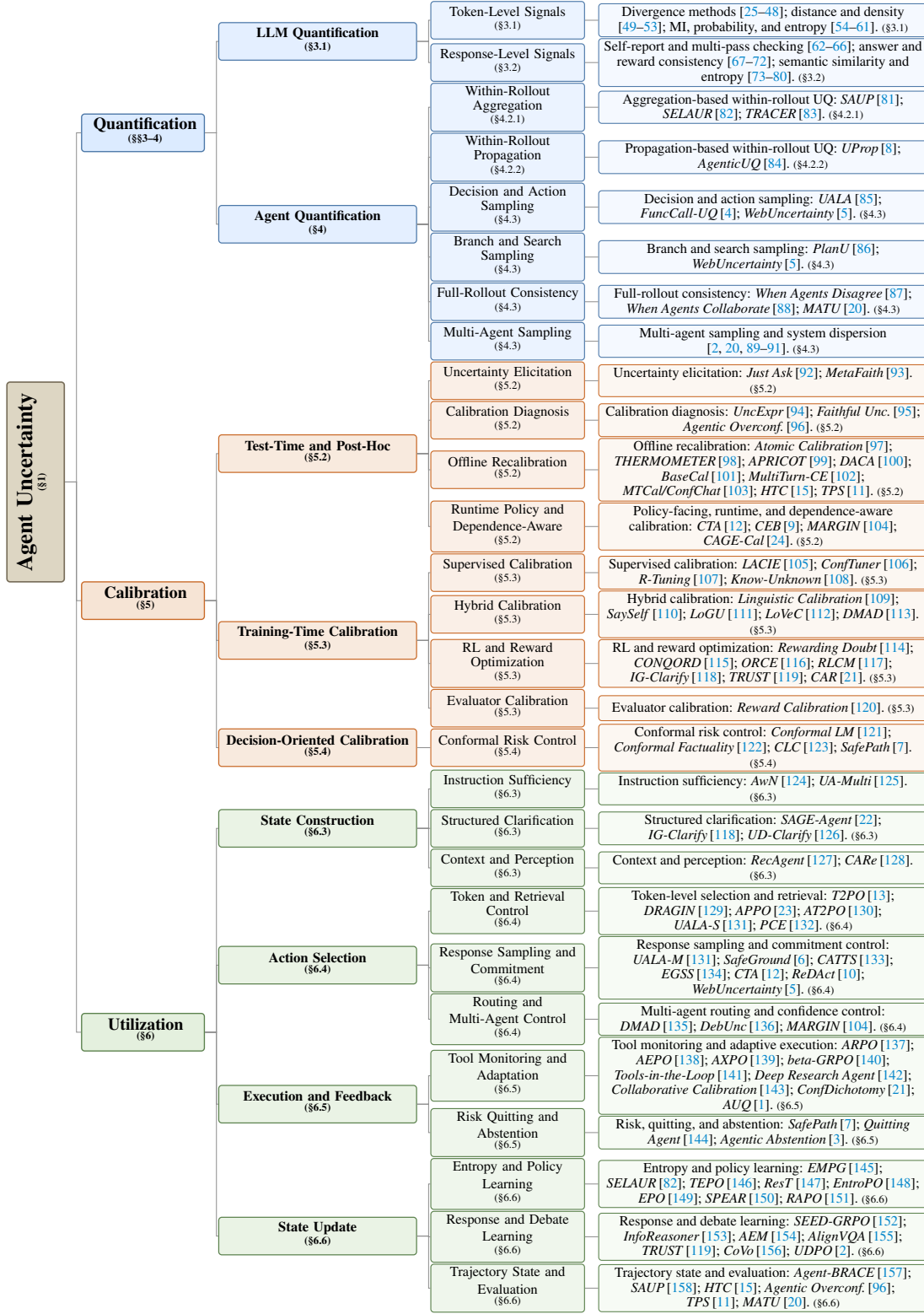


Figure 1: **Agent uncertainty: quantification, calibration, and utilization.** The overview organizes the lifecycle of agent uncertainty into quantification, calibration, and utilization. Every branch follows a common four-column hierarchy: major component, method group, method family, and literature.

Definition: Boundary of Reliable Commitment. Formally, let M denote a model and let x denote the condition faced by the model. We define $\mathcal{C}_M(x)$ as the space of possible commitments that M may make under x , where each $c \in \mathcal{C}_M(x)$ may correspond to an answer, an action, a reasoning path, a tool invocation, or a state update. Within this space, the model has a reliable-commitment region:

$$\mathcal{R}_M(x) = \{c \in \mathcal{C}_M(x) : M \text{ can reliably commit to } c \text{ under } x\}.$$

The boundary of reliable commitment is the frontier of this region:

$$\mathcal{B}_M(x) = \partial\mathcal{R}_M(x).$$

With this boundary formalized, uncertainty describes the model’s relation to this boundary. It is high when a commitment is close to the frontier between what the model can reliably commit to and what it cannot. It is low when the commitment lies well inside the reliable-commitment region.

Under this definition, uncertainty can be further organized by whether the source lies primarily in the task condition or in the model’s commitment.

- **Input uncertainty.** Input uncertainty arises when the condition x is ambiguous, incomplete, underspecified, or conflicting. In this case, the model may be uncertain not because it lacks the ability to solve the task, but because the available condition does not support a unique reliable commitment. Such uncertainty can often be reduced through interaction, such as asking clarification questions, retrieving additional evidence, or obtaining new observations.
- **Output uncertainty.** Output uncertainty arises when the model cannot reliably support a commitment even under the given condition. This uncertainty reflects limitations in the model’s knowledge, reasoning ability, tool-use competence, or behavioral stability. Unlike input uncertainty, it cannot be fully resolved by merely clarifying the input; it requires stronger model competence, better evidence integration, improved reasoning, or external verification.
- **Irreducible uncertainty** arises when the condition and the model are both insufficient to determine a single certain outcome because the underlying process is inherently variable or only partially observable. This uncertainty cannot be fully eliminated by clarification or model improvement. It can only be represented, calibrated, and managed when making downstream decisions.

For LLMs and agents, this distinction is useful because uncertainty may come either from an unclear task condition or from the model’s inability to make a reliable answer, action, tool invocation, or state update.

Moreover, a commitment is broader than a final prediction. It refers to any answer, reasoning step, tool use, state description, or decision that the model treats as sufficiently supported. This broader scope allows uncertainty to cover both traditional prediction uncertainty and the additional uncertainty sources discussed for LLMs and agents [17–19]. Under this view, a condition is low-uncertainty for a model when the model has enough knowledge, reasoning ability, or tool-use competence to support a stable commitment. When uncertainty is high, the model’s behavior suggests that multiple commitments remain plausible, or that its current commitment is close to the boundary beyond which it can no longer be treated as reliable.

2.2 Conceptual Boundaries of Uncertainty

In the literature on LLMs and agents, terms such as confidence, uncertainty, trustworthiness, and reliability are often used together, and sometimes even interchangeably. Therefore, before proceeding, we clarify several concepts that are closely related to uncertainty but should not be treated as equivalent to it, which helps delimit the scope of our survey.

- **Confidence: A Candidate-Level Representation of Uncertainty.** Confidence is closely related to uncertainty, but it is usually a positive score attached to a specific candidate, such as a predicted label, answer, action, or hypothesis. When the prediction target is fixed and the confidence score is tied to a particular class probability, well-calibrated confidence can serve as a useful uncertainty estimate, especially in

simple classification settings. However, the distinction becomes necessary when the object of uncertainty is broader than a specified candidate. A model may report confidence in a particular answer, while the relevant uncertainty concerns whether the model knows the answer at all, whether the answer is stable, or whether the model is operating within its reliable competence boundary. Intuitively, confidence asks how strongly the model supports one candidate, while uncertainty asks whether the model has enough knowledge to make a reliable commitment [160]. Accordingly, confidence is best understood as one operational representation of uncertainty, while uncertainty itself remains the broader concept that organizes what these representations are trying to measure.

- **Trustworthiness: A Multidimensional Socio-Technical Objective.** Trustworthiness is a multidimensional system-level and socio-technical property of AI systems [161–163]. It concerns whether an AI system can be relied upon, governed, audited, and deployed responsibly across its intended contexts of use. Prior frameworks characterize trustworthy AI through dimensions such as validity, reliability, safety, security, accountability, transparency, privacy, and fairness [161–163]. Uncertainty can support trustworthiness by indicating when a model’s commitment requires further verification. When uncertainty is high, the system may abstain, ask for clarification, retrieve additional evidence, or defer to human oversight. In this sense, uncertainty helps calibrate the use of an AI system, but it is not itself the full definition of trustworthiness.
- **Reliability: Robust and Consistent System Performance.** Reliability refers to the ability of a system to perform dependably under its intended conditions of use [162]. It concerns whether the system maintains robust and consistent behavior across repeated runs, perturbed inputs, changing environments, and operational stress. Uncertainty signals can support reliability by identifying low-support predictions, unstable decisions, or fragile intermediate commitments [164–166]. However, uncertainty cannot guarantee reliability alone. A model may be uncertainty-aware, behaviorally stable, or input-robust without being reliable in the broader operational sense. Intuitively, reliability is an operational property of the deployed system, whereas uncertainty is a model-centered signal that can help diagnose and control reliability failures.

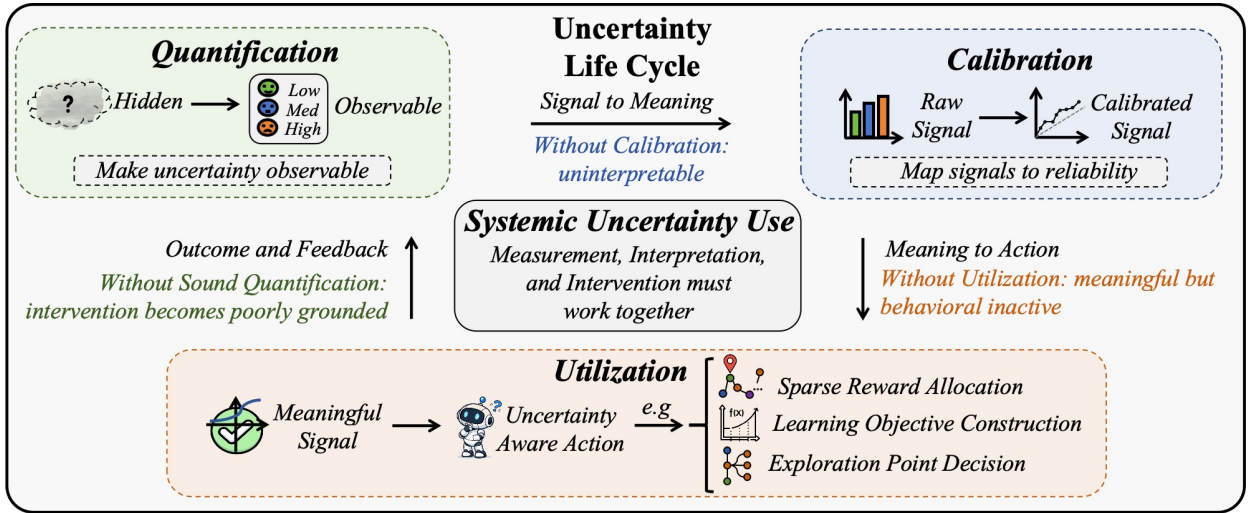


Figure 2: Uncertainty Lifecycle

2.3 A Lifecycle View of Uncertainty in LLM-based Agent

In this paper, We use the term **lifecycle** to describe how an uncertainty signal moves from estimation to interpretation and use. This lifecycle view highlights that uncertainty is not only a problem of measurement. Interpreting uncertainty signals and connecting them to downstream behavior or learning are also central and challenging problems.

- **Quantification.** Under our definition, uncertainty refers to the model’s proximity to the boundary of reliable commitment. Quantification makes this boundary observable by turning uncertainty into a usable signal, such as a score, distribution, set, ranking, categorical judgment, or behavioral proxy [17, 18, 165, 166]. It answers a measurement question: how strongly is the model’s current commitment supported relative to its reliable competence boundary? Quantification alone does not determine whether the signal is aligned with correctness, stability, or task success, nor does it specify how the system should act on the signal.
- **Calibration.** Calibration is the interpretation step of the uncertainty lifecycle. It determines whether a quantified uncertainty signal has the intended meaning. Without calibration, a signal may be measurable but not meaningful for decision-making, because its value may not reflect the reliability of the model’s commitment. Classical confidence calibration instantiates this idea by requiring predictions assigned confidence c to be correct with empirical frequency approximately c [164]. More broadly, calibration validates or adjusts an uncertainty representation so that it can be interpreted as evidence about reliable commitment [18, 160].
- **Utilization.** Utilization is the intervention step of the uncertainty lifecycle. After uncertainty has been quantified and, ideally, calibrated, utilization determines how the system should respond to it. It turns uncertainty from a descriptive signal into an operational constraint on model or agent behavior, such as deciding whether to seek more evidence, revise a commitment, defer a decision, or update a policy [18, 167]. In this sense, utilization closes the lifecycle by translating reliability awareness into behavioral control.

These three stages should not be treated as independent components: quantification without calibration may yield an uninterpretable signal, while calibration without utilization may leave uncertainty disconnected from downstream behavior. The lifecycle view therefore allows us to analyze how uncertainty moves from a model-centered reliability condition to a practical mechanism for improving LLM and agent decisions.

3 Background: Uncertainty Quantification in LLMs

A necessary foundation for studying uncertainty in LLM-based agents is uncertainty in the underlying LLM itself. Since the LLM serves as the reasoning and decision-making backbone of an agent, many uncertainty signals originally developed for standalone generation can be directly computed at individual agent decisions. Accordingly, we first review uncertainty quantification in standalone LLMs before examining what changes once the model is embedded in an interactive agent loop. The purpose of this background is not to treat LLM and agent uncertainty as independent topics. Rather, it establishes which quantification methods can be transferred directly from the backbone model and prepares the distinction between uncertainty that remains local to an LLM generation and uncertainty that emerges from sequential decisions, external observations, and agent–environment interaction.

Table 1: Taxonomy of uncertainty quantification methods in LLMs

Method Base	Method	Cluster	Train	Multi-Out	Access	Compute	Memory	Granularity	Examples
Token Level									
Divergence-Based	Kullback–Leibler Divergence	Distribution	✗	✗	White	Low	Low	Token	[25–40, 40–45]
	Rényi Divergence	Distribution	✗	✗	White	Low	Low	Token	[46]
	Jensen–Shannon Divergence	Distribution	✗	✗	White	Low	Low	Token	[47, 48]
Distance-Based	Mahalanobis Distance	Distribution	✗	✗	White	Medium	Medium	Token	[48, 49]
	Relative Mahalanobis Distance	Distribution	✗	✗	White	Medium	Medium	Token	[50]
	Fisher–Rao Distance	Distribution	✗	✗	White	Medium	Medium	Token	[51]
Density-Based	Semantic Density Estimation	Distribution	✗	✗	Black/White	High	High	Token→Seq	[51, 52]
	Robust Density Estimation	Distribution	✗	✗	Black/White	High	High	Token→Seq	[53]
Probability-Based	Pointwise Mutual Information	Mutual Info	✗	✗	White	Low	Low	Token	[54, 55]
	Conditional PMI	Mutual Info	✗	✗	White	Low	Low	Token	[54, 55]

Continued on next page

Method Base	Method	Cluster	Train	Multi-Out	Access	Compute	Memory	Granularity	Examples
	Token Probability	Probability	✗	✗	White	Low	Low	Token	[56]
	Joint Probability	Probability	✗	✗	White	Low	Low	Sequence	[57]
	Maximum Sequence Probability	Probability	✗	✗	White	Low	Low	Token→Seq	[58]
	AvgProb	Probability	✗	✗	White	Low	Low	Token→Seq	[56]
	AvgLogProb	Probability	✗	✗	White	Low	Low	Token→Seq	[58]
	Perplexity	Probability	✗	✗	White	Low	Low	Token→Seq	[58]
	Shannon Token Entropy	Entropy	✗	✓	White	Low	Low	Token	[34, 59, 60]
	Shannon Sequence Entropy	Entropy	✗	✓	White	Low	Low	Token→Seq	[61]
Response Level									
Prompt-Based	Direct Self-Report	Prompt-Based	✗	✗	Black	Low	Low	Seq	[62–64]
	Multi-Pass Self-Report	Prompt-Based	✗	✗	Black/White	Medium	Medium	Seq	[65, 66]
Consistency-Based	Identical Answer Frequency	Agreement	✗	✓	Black	Medium	Medium	Seq	[67, 68]
	Reward Model Ensemble Variance	Agreement	✗	✓	Black	High	High	Seq	[69–72]
Similarity-Based	Similarity Matrix	Semantic Similarity	✗	✓	Black/White	High	High	Seq/Claim	[67, 73]
	von Neumann Entropy	Entropy	✗	✓	Black/White	High	High	Seq/Claim	[74]
	Semantic Entropy	Entropy	✗	✓	Black/White	High	High	Seq/Claim	[75–77]
	Semantic Nearest Neighbor Entropy	Entropy	✓	✓	White	Medium	Medium	Token→Seq	[78]
	Geometric Entropy	Entropy	✗	✓	Black/White	High	High	Seq/Claim	[79, 80]

3.1 Token-Level Uncertainty Quantification

Token-level uncertainty quantification exploits the white-box signals exposed during autoregressive decoding, where each generation step produces a conditional distribution over the next token. Uncertainty can therefore be derived from local probability mass, predictive dispersion, representation familiarity, or sensitivity to alternative contexts and model states, and then aggregated across tokens when sequence-level evidence is required [28, 57, 61]. These signals are fine-grained and inexpensive to obtain, but their local nature does not necessarily reflect the semantic reliability of the completed response, motivating higher-level uncertainty estimators discussed next.

3.1.1 Divergence-Based Methods

Divergence-based methods quantify token-level uncertainty by comparing predictive distributions obtained under alternative model states or conditioning settings. These variants may arise from Bayesian parameter samples or ensembles, controlled perturbations, context ablations, or intermediate model states, so the resulting disagreement can reflect parameter uncertainty, local robustness, dependence on external evidence, or instability across internal computation [28, 34, 48]. Most methods instantiate this comparison through KL-to-mixture or closely related Rényi and Jensen–Shannon divergences, which mainly differ in how they weight or symmetrize the same predictive disagreement [46, 47].

Core Computation: Distributional Disagreement. Suppose the same decoding step is evaluated under K model states or conditioning settings, producing token distributions $\{p_t^{(k)}\}_{k=1}^K$. Their mixture distribution is [26, 28]

$$\bar{p}_t(v) = \frac{1}{K} \sum_{k=1}^K p_t^{(k)}(v).$$

A canonical divergence score is the average KL divergence from each predictive variant to this mixture:

$$u_t^{\text{div}} = \frac{1}{K} \sum_{k=1}^K \sum_{v \in \mathcal{V}} p_t^{(k)}(v) \log \frac{p_t^{(k)}(v)}{\bar{p}_t(v)}.$$

The score measures predictive instability across the selected variants, while its epistemic, contextual, or robustness interpretation is determined by the intervention used to construct those variants [28, 34, 35].

3.1.2 Distance-Based Methods

Distance-based methods quantify uncertainty through deviation from familiar reference regions rather than disagreement across predictive variants. Mahalanobis-style estimators operate in hidden-state or logit space, with relative variants removing shared background geometry, while Fisher–Rao distance extends the same reference-based principle to predictive distributions [48–51].

Core Computation: Distance from a Reference Region. Let r_t denote the representation used to evaluate the current token, and let reference component c be represented by mean μ_c and covariance Σ_c . A standard estimator uses the minimum squared Mahalanobis distance [48, 49]

$$u_t^{\text{dist}} = \min_c \left[(r_t - \mu_c)^\top \Sigma_c^{-1} (r_t - \mu_c) \right].$$

The score is small when the decoding representation falls within a familiar reference region and large when it deviates from every modeled component [48, 49].

3.1.3 Density-Based Methods

Density-based methods generalize reference distance by measuring how strongly a token representation is supported by nearby reference data, allowing familiar regions to be multimodal or non-Gaussian. Existing variants differ mainly in the representation space, density estimator, and robustness treatment used to define local support [51–53].

Core Computation: Local Density Support. Let $\{r_i\}_{i=1}^n$ denote reference representations and let $K_b(r - r_i)$ be a kernel with bandwidth b . Local support around the current representation r_t is estimated as [51, 52]

$$\hat{q}(r_t) = \frac{1}{n} \sum_{i=1}^n K_b(r_t - r_i).$$

Density-based uncertainty is then

$$u_t^{\text{dens}} = -\log \hat{q}(r_t).$$

The score is low in regions strongly supported by reference representations and high in sparse regions, making uncertainty a property of local empirical support rather than distance to a single center [51–53].

3.1.4 Mutual-Information-Based Methods

Mutual-information-based methods quantify how much a specified conditioning source contributes to the probability of a generated token, separating input-grounded support from generic continuation preference. PMI and conditional PMI mainly differ in whether the full condition or only a selected evidence source is removed when constructing the comparison [54, 55].

Core Computation: Contribution of Conditioning Information. Let x denote the information whose contribution is being measured and y_t the generated token. Pointwise mutual information compares token support with and without x [54, 55]

$$\text{PMI}_t(y_t; x) = \log \frac{p_\theta(y_t \mid x, y_{<t})}{p_\theta(y_t \mid y_{<t})}.$$

A large value indicates that x substantially increases support for the token, whereas a value near zero indicates that the same continuation is already supported by the generated prefix alone [54, 55].

3.1.5 Probability-Based Methods

Probability-based methods derive uncertainty directly from the support assigned to the realized decoding path. Token probability and surprisal provide local scores, while joint probability, average log-probability, perplexity, and related variants extend the same signal to complete generations through different aggregation or normalization rules [56–58].

Core Computation: Support for the Realized Generation. For a generated token y_t , token surprisal is [57]

$$u_t^{\text{prob}} = -\log p_\theta(y_t \mid x, y_{<t}).$$

A length-normalized sequence score is obtained by averaging token surprisal [57, 58]

$$U_{\text{AvgNLL}}(y \mid x) = -\frac{1}{T} \sum_{t=1}^T \log p_\theta(y_t \mid x, y_{<t}).$$

Perplexity is the exponential transformation of the same average [58]

$$\text{PPL}(y \mid x) = \exp(U_{\text{AvgNLL}}(y \mid x)).$$

These scores differ in scale and normalization but all evaluate the support assigned to the generation selected by the model [56–58].

3.1.6 Entropy-Based Methods

Entropy-based methods evaluate uncertainty over the full predictive distribution rather than only the token ultimately selected. Token entropy measures local competition among next-token alternatives, while sequence entropy extends this quantity over the alternative prefixes induced by the autoregressive generation process [61].

Core Computation: Token and Sequence Entropy. Given an input x and an autoregressive prefix $y_{<t}$, predictive token entropy is [61]

$$H_t = -\sum_{v \in \mathcal{V}} p_\theta(v \mid x, y_{<t}) \log p_\theta(v \mid x, y_{<t}),$$

where $\mathcal{V}$ is the vocabulary. The entropy of the complete autoregressive sequence distribution is [61]

$$H(Y_{1:T} \mid x) = \sum_{t=1}^T \mathbb{E}_{Y_{<t} \sim p_\theta} \left[-\sum_{v \in \mathcal{V}} p_\theta(v \mid x, Y_{<t}) \log p_\theta(v \mid x, Y_{<t}) \right].$$

The first quantity evaluates local predictive dispersion under a fixed prefix, whereas the second integrates that dispersion over prefixes that may arise under the model’s sequence distribution [61].

3.2 Response-Level Uncertainty Quantification

Response-level uncertainty quantification treats the complete generated answer, rather than individual decoding steps, as the unit of analysis. It estimates uncertainty from properties of one or multiple full responses, such as agreement across repeated generations, semantic variation among sampled answers, explicit self-reported confidence, or the stability of an external evaluator [168–170]. This level is especially useful when different token sequences may express the same semantic commitment or when token-level confidence does not directly reflect the reliability of the final answer [171, 172]. Because these signals primarily measure behavioral consistency, semantic dispersion, self-assessment, or evaluator stability, they should not be interpreted as correctness probabilities without additional calibration [169, 173].

3.2.1 Consistency-Based Methods

Consistency-based methods estimate uncertainty from the reproducibility of repeated generations. Agreement can be defined through exact answer frequency, claim-level support, semantic similarity, or learned evaluators, but all variants interpret stronger cross-sample agreement as lower behavioral uncertainty [62, 174, 175]. Because repeated agreement does not imply correctness, systematic misconceptions may still appear confidently consistent [62, 176].

Core Computation: Agreement Across Repeated Responses. Given an input x , the model generates N responses [62, 176–178]

$$y^{(1)}, \dots, y^{(N)} \sim p_\theta(y \mid x).$$

A comparison function g maps each response to an answer class or another comparison unit. If n_c responses are assigned to class c , its empirical frequency is

$$\hat{p}(c \mid x) = \frac{n_c}{N}.$$

The dominant-response confidence and corresponding uncertainty are

$$C_{\text{cons}}(x) = \max_c \hat{p}(c \mid x), \quad U_{\text{cons}}(x) = 1 - C_{\text{cons}}(x).$$

When response agreement is graded rather than discrete, let $s(y^{(i)}, y^{(j)}) \in [0, 1]$ denote pairwise compatibility and compute [179–182]

$$U_{\text{pair}}(x) = 1 - \frac{2}{N(N-1)} \sum_{i < j} s(y^{(i)}, y^{(j)}).$$

Both forms interpret reproducibility as low uncertainty, with the comparison function determining whether agreement is defined over answers, claims, or graded semantic relations [174, 175].

3.2.2 Sampling-Based Semantic Uncertainty

Sampling-based semantic methods move beyond surface agreement by estimating uncertainty over the meanings represented by multiple generated responses. Existing approaches range from hard semantic classes to graded kernels, neighbourhoods, and hierarchical representations, while efficient variants approximate the same multi-sample uncertainty with cheaper similarity models, adaptive sampling, or learned probes [168, 172, 183, 184]. Their common target is dispersion over semantic commitments rather than variation among strings.

Core Computation: Entropy over Response Meanings. Given N sampled responses [168, 171]

$$y^{(1)}, \dots, y^{(N)} \sim p_\theta(y \mid x),$$

a semantic equivalence relation partitions them into meaning classes $\mathcal{C}_1, \dots, \mathcal{C}_M$. Their empirical probabilities are

$$\hat{p}_m = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[y^{(i)} \in \mathcal{C}_m],$$

and hard-cluster semantic uncertainty is

$$H_{\text{sem}}(x) = - \sum_{m=1}^M \hat{p}_m \log \hat{p}_m.$$

For graded semantic relations, pairwise similarities form a positive-semidefinite kernel matrix K whose normalized form is [172]

$$\rho = \frac{K}{\text{tr}(K)}.$$

A soft uncertainty score is then given by von Neumann entropy:

$$H_{\text{kernel}}(x) = -\text{tr}(\rho \log \rho) = -\sum_j \lambda_j \log \lambda_j,$$

where $\{\lambda_j\}$ are the eigenvalues of ρ . The two forms differ in whether semantic variation is represented through discrete classes or graded relations [168, 172].

3.2.3 Self-Verbalized Uncertainty

Self-verbalized methods directly ask the model to assess the reliability of its own response through numerical confidence, linguistic expressions, knowledge judgments, or reflection-enhanced reports [63, 169, 185]. They provide a convenient black-box uncertainty signal, but the resulting value depends on the elicitation protocol and should not be assumed to represent an already calibrated correctness probability [169, 173].

Core Computation: Elicited Self-Assessment. Given an input x and response y , the model produces a confidence report z with or after the answer [63, 169, 185, 186]

$$(y, z) \sim p_{\theta}(y, z \mid x).$$

A deterministic interpretation function ϕ maps the numerical or linguistic report to a confidence score:

$$c_{\text{verb}}(x, y) = \phi(z) \in [0, 1].$$

The corresponding uncertainty is

$$u_{\text{verb}}(x, y) = 1 - c_{\text{verb}}(x, y).$$

The function ϕ normalizes the elicited format rather than recovering a unique latent confidence variable, so the resulting score is specific to the prompting and parsing protocol [173].

3.2.4 Reward-Model Uncertainty

Reward-model uncertainty shifts the quantified object from the generator to the evaluator that assigns response quality. It is commonly estimated through disagreement among reward models or heads, predicted reward variance, Bayesian approximations, or representation-based support, with high uncertainty indicating that the supervision signal itself may be unreliable [69, 170, 187, 188]. This uncertainty can therefore be used to identify rewards that require conservative adjustment, relabeling, or stronger evaluation.

Core Computation: Disagreement Among Response Evaluators. Let K reward models or reward heads assign scalar scores [69, 170]

$$r_1(x, y), \dots, r_K(x, y)$$

to a response y . Their mean reward is

$$\bar{r}(x, y) = \frac{1}{K} \sum_{k=1}^K r_k(x, y),$$

and ensemble uncertainty is

$$U_{\text{RM}}(x, y) = \frac{1}{K-1} \sum_{k=1}^K (r_k(x, y) - \bar{r}(x, y))^2.$$

The mean estimates response quality, whereas variance measures whether that evaluation remains stable across evaluator realizations [69, 170, 189].

4 Uncertainty Quantification in LLM Agents: From Output Distributions to Interactive Trajectory

The preceding section introduced uncertainty quantification for standalone LLM generation. Since an LLM serves as the reasoning and decision-making backbone of an agent, many single-turn signals, such as token probability, entropy, response consistency, and semantic disagreement, can be directly applied to an individual agent decision. These methods remain important primitives, but they do not capture the full uncertainty of an agent process.

An agent extends generation into a sequence of decisions and interactions. Its later decisions depend not only on previous model outputs, but also on tool results, environmental feedback, user responses, and communication with other agents. This introduces two new quantification problems. Within one rollout, local uncertainty must be aggregated or propagated across dependent steps. Across repeated rollouts, uncertainty must be estimated from variation in outcomes, action paths, structured tool calls, or system-level interactions. Agent uncertainty quantification therefore extends single-turn LLM signals from isolated outputs to multi-step interactive processes.

Table 2: Taxonomy of uncertainty quantification in LLM-based agents. **Within-rollout methods** aggregate or propagate uncertainty along one interaction trajectory, whereas **sampling-based methods** estimate uncertainty from variation across actions, branches, executions, or agents. N_a , N_b , and N_τ denote sampled actions, branches, and complete trajectories, respectively.

Method Category	Samples	Process Object	Core Computation	Signal	Train	Granularity	Setting	Refs.
A. Within-Rollout Quantification: $U_{\text{within}}(\tau) = \mathcal{F}(\{u_t, \phi_t\}_{t=1}^T)$								
Aggregation-Based	1	Local signals along trajectory	Pooling / weighting / critical-step aggregation	Entropy, confidence, surprisal, mismatch	Mixed	Token / step $\rightarrow$ trajectory	Single-agent	[81–83]
Propagation-Based	1 / N_{TDP}	Dependent transitions and memory states	Dependency estimation / belief propagation	MI/PMI, confidence, belief probabilities	No	Step $\rightarrow$ state / trajectory	Sequential / memory	[8, 84]
B. Sampling-Based Quantification: $U_{\text{sample}}(x) = \text{Disp}(\{\psi(z^{(i)})\}_{i=1}^N)$								
Decision / Action Sampling	N_a	Candidate actions or answers	Frequency / entropy / semantic clustering	Answers, tool calls, action semantics	No	Decision / action	Tool / web	[4, 5, 85]
Branch / Search Sampling	N_b	Search nodes and branches	Return dispersion / tree-based uncertainty	Node values, outcomes, branch confidence	No	Node / path	Planning / web	[5, 86]
Full-Rollout Consistency	N_τ	Complete executions	Outcome / path consistency	Outcomes, action paths, trajectory embeddings	No / Unsup.	Trajectory	Single- / multi-agent	[20, 87, 88]
Multi-Agent Sampling	$K \times N$	Agents across rounds / runs	Cross-agent dispersion / decomposition	Agent outputs, round entropy, trajectory tensors	No / Unsup.	Agent / system	Multi-agent	[2, 20, 89–91]

4.1 What Changes in Agents From Generated Outputs to Interactive Trajectories

Many uncertainty signals developed for standalone LLMs remain available in agents, but their interpretation changes once generation becomes part of an interactive trajectory. Let an agent rollout be

$$(h_0, a_1, o_1, h_1, \dots, a_T, o_T, h_T), \quad a_t \sim \pi_\theta(\cdot \mid h_{t-1}), \quad o_t \sim \mathcal{E}(\cdot \mid h_{t-1}, a_t), \quad h_t = \mathcal{G}(h_{t-1}, a_t, o_t), \quad (1)$$

where uncertainty is embedded in a process jointly shaped by the policy, external feedback, and state updates [11].

- **Local uncertainty becomes process-dependent.** An uncertain action changes which observation is obtained next, while uncertain or noisy feedback changes the state conditioning later decisions. A locally confident step can therefore still inherit unreliability from earlier interactions [5, 137].

- **One trajectory creates aggregation and propagation problems.** Local signals can be aggregated into a process-level assessment through contextual or cross-granularity aggregation [81, 82]. Their cross-step dependence can instead be retained explicitly so that earlier uncertainty remains relevant after the state evolves [8, 84].
- **Sampling expands from responses to process objects.** Agent uncertainty can be estimated by resampling immediate actions or simulated branches [4, 190], as well as by comparing repeated complete executions or variation across interacting agents [2, 87]. These objects require different comparison spaces because action disagreement, search uncertainty, trajectory instability, and multi-agent disagreement describe different process-level uncertainties.

Table 2 summarizes the two computational paradigms motivated by these changes. Within-rollout methods quantify uncertainty along one interaction process, whereas sampling-based methods estimate variation across alternative realizations of that process.

4.2 Within-Rollout Quantification

Within-rollout quantification derives process-level uncertainty from evidence observed along a single interaction trajectory. Its central question is how local uncertainty should remain useful beyond the step at which it is measured. Aggregation compresses local signals into a trajectory-level assessment. Propagation instead preserves how current uncertainty depends on earlier states and decisions. We review these two method families in turn.

Takeaways

Within one rollout, process uncertainty is quantified from local token- or step-level signals in two main ways:

- **Trajectory aggregation.** Local uncertainty estimates are combined into a trajectory-level score through fixed pooling, context-dependent weighting, cross-granularity transformation, or tail-sensitive emphasis.
- **Cross-step propagation.** Current uncertainty is decomposed into locally generated and inherited components, or carried forward through an uncertainty-aware state or memory so that unresolved risk remains available to later decisions.

4.2.1 Aggregation-Based Methods

Aggregation-based methods assume that local token- or step-level uncertainty has already been estimated and ask how these signals should contribute to a higher-level trajectory assessment. The aggregation may change the weight of different steps, transform uncertainty across temporal granularities, or emphasize rare but consequential high-risk regions rather than average behavior.

Core Computation. Within-Trajectory Aggregation. Let u_t denote a local uncertainty or risk signal at step t , and let ϕ_t encode its process-level relevance, such as position, interaction context, or consistency with the returned observation. Aggregation-based uncertainty can be abstracted as

$$U_{\text{agg}}(\tau) = \mathcal{A}\left(\{(u_t, \phi_t)\}_{t=1}^T\right).$$

The functional $\mathcal{A}$ may implement fixed pooling, contextual weighting, cross-granularity transformation, or selective emphasis on high-risk steps. Its role is therefore to determine which local uncertainty signals dominate the resulting process-level assessment.

Representative aggregation strategies. Representative methods differ in which local signals receive greater influence when constructing a process-level assessment. *SAUP* learns situation-dependent weights from the relation among the task, intermediate reasoning, action, and returned observation, allowing local uncertainty to contribute differently across interaction contexts [81]. *SELAUR* instead converts token-level entropy, least

confidence, and margin into step- and trajectory-level quantities, addressing the mismatch between where uncertainty is measured and where the agent objective is defined [82]. *TRACER* focuses on sparse but consequential failures by combining local risk indicators with tail-sensitive aggregation so that extreme trajectory regions are not diluted by many benign steps [83]. These methods emphasize contextual relevance, cross-granularity transformation, and tail risk within the same aggregation paradigm.

4.2.2 Propagation-Based Methods

Propagation-based methods retain the temporal dependence of uncertainty rather than collapsing local signals into one summary. They ask how uncertainty at the current step depends on earlier decisions, or how uncertainty-bearing information should remain available in the evolving state so that later decisions can condition on unresolved reliability concerns.

Core Computation. Cross-Step Uncertainty Dependence. Let u_t^{int} denote uncertainty attributed to the current decision, and let u_t^{ext} denote uncertainty inherited through dependence on the preceding process. A dependency-based formulation can be written as

$$u_t = u_t^{\text{int}} + u_t^{\text{ext}}, \quad u_t^{\text{ext}} \propto I(A_t; A_{<t} \mid x),$$

where $I(A_t; A_{<t} \mid x)$ characterizes dependence between the current and preceding decisions. A complementary state-retention formulation preserves an uncertainty-bearing representation through

$$b_t = \mathcal{P}(b_{t-1}, u_t, a_t, o_t).$$

The former estimates inherited uncertainty through statistical dependence, whereas the latter explicitly carries uncertainty information forward in the agent state.

Statistical and memory-based propagation. The two representative approaches correspond to these two forms of temporal dependence. *UProp* decomposes current uncertainty into intrinsic and inherited components and estimates cross-step dependence through mutual-information-based quantities, providing a statistical account of how earlier decisions contribute to current uncertainty [8]. *AgenticUQ* instead preserves verbalized confidence and semantic uncertainty explanations in an uncertainty-aware memory, allowing unresolved uncertainty from earlier steps to remain visible to subsequent reasoning [84]. Aggregation compresses uncertainty into a process-level assessment, whereas propagation retains information about its origin or continued relevance as the trajectory evolves.

4.3 Sampling-Based Quantification

Sampling-based quantification estimates uncertainty by generating alternative realizations of an agent object and measuring their dispersion in an appropriate comparison space. The interpretation depends primarily on what is resampled. Immediate actions reveal decision ambiguity, simulated branches represent uncertainty over future consequences, repeated executions measure process reproducibility, and multi-agent samples expose variation across participants or interaction structures [91].

Takeaways

Sampling-based methods generate alternative agent objects, map them into a decision-relevant comparison space, and quantify their dispersion. The sampled object determines the level of uncertainty:

- **Immediate-action sampling.** Alternative responses or tool calls are generated from the same state and compared through frequency, entropy, semantic difference, or executable behavior.
- **Future-branch sampling.** Alternative search nodes or continuations are simulated, and uncertainty is measured from the dispersion of their returns, outcomes, or branch confidence.
- **Full-rollout sampling.** Complete executions are repeated and compared through final outcomes, action paths, divergence points, or trajectory representations to estimate process reproducibility.

- **Multi-agent sampling.** Variation across agents, interaction rounds, and repeated system runs is used to separate intra-agent instability, inter-agent disagreement, and interaction-level uncertainty.

Core Computation. Dispersion over Sampled Agent Objects. Let ξ denote the condition held fixed and let

$$z^{(i)} \sim Q(\cdot \mid \xi), \quad i = 1, \dots, N,$$

denote sampled agent objects. Each sample is mapped into a comparison representation $\psi(z^{(i)})$, after which uncertainty is computed as

$$U_{\text{sample}}(\xi) = \text{Disp} \left(\{\psi(z^{(i)})\}_{i=1}^N \right).$$

The sampled object may be an immediate action, a simulated branch, a complete execution, or an individual agent within a collaborative system. The conditioning variable ξ and representation ψ determine whether the resulting dispersion reflects decision ambiguity, search uncertainty, rollout reproducibility, or system-level disagreement [89, 90].

Decision-level sampling. Sampling-based agent uncertainty differs first in how finely the alternative processes are resampled. At the decision level, methods hold the current state fixed and compare alternative immediate commitments. *UALA* uses disagreement among sampled responses to determine whether external assistance is needed [85], while Ye et al. cluster sampled function calls in abstract-syntax-tree space so that uncertainty reflects differences in executable behavior rather than surface form [4].

Sampling future consequences. Sampling can extend from immediate actions to their possible consequences. *PlanU* represents the return of each search node through a quantile distribution and uses its dispersion during tree search [190]. *WebUncertainty* instead couples action uncertainty with adaptive branch expansion, evaluation, and pruning as simulated evidence accumulates [5]. The comparison object therefore changes from alternative current actions to alternative future branches generated from the same decision state.

Full-rollout reproducibility. At the coarsest single-agent level, repeated complete executions expose whether the overall process is reproducible. Mehta et al. compare action paths and divergence positions across rollouts [87], and further examine whether repeated executions converge toward similar internal representations [88]. This shifts the uncertainty object from local alternatives to the stability of the complete interaction process.

Multi-agent variation. Multi-agent systems introduce another source of variation because uncertainty may arise within one agent, between agents, or from the interaction process itself. *The Value of Variance* separates intra-agent revision, inter-agent disagreement, and system-level instability through hierarchical uncertainty estimates [2]. *MATU* preserves variation across agents, steps, and repeated runs in a tensorized trajectory representation [20]. Multi-agent sampling therefore asks not only whether the collective output is unstable, but also which level of the collaborative process contributes that instability.

4.4 Discussion: Input Uncertainty

Input uncertainty can remain hidden behind confident outputs. Within-rollout and sampling-based methods quantify uncertainty after the agent has constructed a state and begun to choose actions. They therefore assume that the instructions, observations, retrieved evidence, memory, and inter-agent messages already provide a reliable basis for decision making. This assumption may fail before any output is generated. The input state may be incomplete, ambiguous, conflicting, outdated, or false. Input uncertainty must therefore be distinguished from output uncertainty. An agent can silently adopt one interpretation of an underspecified request and then reason consistently within it. Token entropy, sampling agreement, and verbalized confidence may all indicate high certainty even though the underlying interpretation remains unsupported [126]. Existing output-side signals are conditioned on the constructed state; they do not independently establish that the state itself is reliable. This separation is also supported by evidence that common confidence signals can fail to detect subtle instruction-following errors [191].

Agent interaction makes input uncertainty dynamic and potentially reducible. Classical uncertainty frameworks often associate input-side uncertainty with noise that lies outside the model [159, 192]. This view is incomplete for interactive agents. Natural-language inputs can contain lexical, referential, and pragmatic ambiguity [193, 194]. Tool-using agents face additional sources: an instruction may omit required arguments, presuppose an incorrect world state, or conflict with the observations returned during execution [124, 195, 196]. Tool outputs, memory entries, and messages from other agents also become new inputs at later turns, allowing unreliable information to accumulate across the trajectory [197, 198]. Yet these failures are not always irreducible. Clarification can resolve an underspecified request, retrieval can supply missing evidence, and re-observation can repair an incomplete state. Input uncertainty should therefore be characterized by both its source and its reducibility. An uncertainty source may lie outside the model while remaining removable through interaction [16].

Current methods detect state insufficiency, but its calibration remains unresolved. Input-side methods ask whether the condition admits several plausible interpretations or lacks information required for action. Existing approaches sample alternative readings, track uncertainty over required tool arguments, test whether contextual evidence supports the instruction’s assumptions, or measure sensitivity to input perturbations [22]. These methods have begun to connect input uncertainty with clarification and state repair. However, current benchmarks mainly evaluate the resulting behavior: whether the agent asks an appropriate question and whether the task eventually succeeds. They rarely test whether the estimated input uncertainty is itself accurate. This leaves two basic questions. What reliability event should input uncertainty be calibrated against when the user’s intended meaning is latent or genuinely ambiguous? How can a method show that it measures uncertainty in the constructed state rather than repackaging downstream output confidence? Until these questions are resolved, input uncertainty remains less formally evaluated than output uncertainty, despite arising earlier in the decision process. Its downstream uses in clarification, information acquisition, and context repair are discussed in Section 6.3.

Input uncertainty therefore extends agent quantification upstream. Before asking whether the agent is uncertain about which action to take, one must first ask whether its current state supports any reliable action at all.

5 Uncertainty Calibration in LLM Agents: Aligning Confidence with Empirical Reliability

Uncertainty calibration asks whether a reported confidence can be interpreted as the empirical probability of a specified outcome. For a binary event Y , perfect calibration requires

$$\Pr(Y = 1 \mid \hat{p} = p) = p, \quad (2)$$

so that predictions assigned confidence p are correct approximately a fraction p of the time [199, 200]. Calibration is needed because LLM and agent confidence can remain systematically misaligned with observed correctness or trajectory success even when the underlying uncertainty signal is informative [15, 200]. Calibrated probabilities can then support verification, exploration, deferral, or delegation under explicit risk and cost considerations [12, 201].

5.1 What Is New in Agent Uncertainty Calibration?

Agent calibration retains the same statistical meaning as conventional LLM calibration, but the calibrated event and the data-generating process become part of an evolving interaction [15, 201].

- **From response correctness to process-conditioned outcomes.** Agent calibration can target action productivity, prefix-level continuation success, or terminal trajectory success rather than only the correctness of one completed response [9, 15, 201]. A representative target is

$$q_t = \mathbb{P}^\pi(Y = 1 \mid \mathcal{H}_t), \quad (3)$$

which conditions reliability on the current interaction history and deployment policy [201].

- **From passive reporting to a calibration–action loop.** Calibrated uncertainty can directly change subsequent behavior by triggering retrieval, verification, continued exploration, or delegation; once the policy changes, the distribution on which later calibration is evaluated changes as well [12, 201].

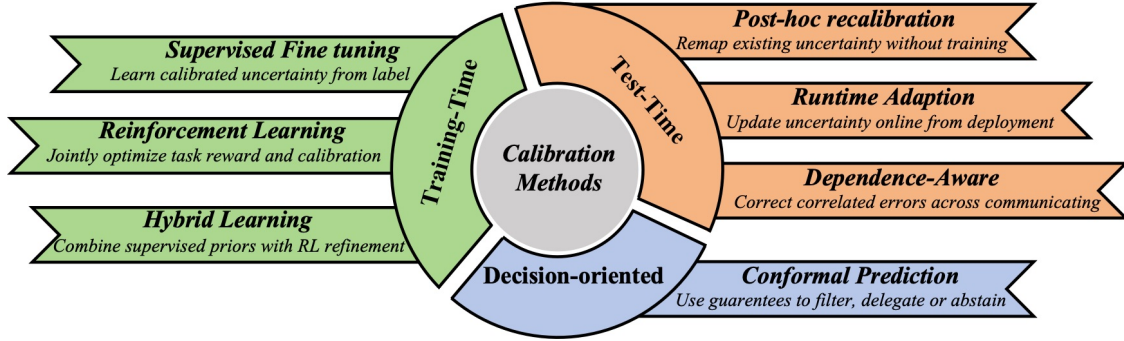


Figure 3: Uncertainty Calibration

- **From a fixed predictor to an evolving interaction system.** Tool-dependent reliability, delayed feedback, runtime distribution shift, and correlated errors in multi-agent communication can make a fixed calibrator unreliable during deployment [21, 24, 104]. Conventional single-response calibration can therefore be viewed as a fixed-policy, one-step special case of the broader agent setting [15, 201].

Accordingly, we organize agent uncertainty calibration into three lines: *test-time and runtime calibration*, *training-time calibration*, and *decision-oriented calibration*.

Table 3: Taxonomy of uncertainty calibration and calibration-related methods for LLMs and LLM-based agents. The section headers denote the high-level calibration stage, while the first column groups methods by a standardized method family. The *Calibrated Object* column records the reliability event or behavior targeted by each method. *Calibration Type*, *Interaction Regime*, and *Primary Setting* use controlled vocabularies so that conceptually equivalent methods are described consistently. The *Agent-Applicable* column is checked for methods originally developed in agent settings and for LLM methods whose core mechanism can be transferred to action-, turn-, prefix-, trajectory-, tool-, or coordination-level reliability targets without changing the method family.

Method Category	Method / Framework	Calibrated Object	Calibration Type	Interaction Regime	Primary Setting	Agent-Applicable	Ref.
A. Test-time elicitation and post-hoc recalibration							
Uncertainty Elicitation	Just Ask	Answer correctness	Elicitation	Single-turn	LLM	–	[92]
	MetaFaith	Expression faithfulness	Elicitation	Long-form	LLM	–	[93]
Calibration Diagnosis	UncExpr	Answer correctness	Diagnosis	Single-turn	LLM	–	[94]
	Faithful Unc.	Expression faithfulness	Diagnosis	Long-form	LLM	–	[95]
Offline Recalibration	Agentic Overconf.	Trajectory success	Diagnosis	Tool-use	AGENT	✓	[96]
	Atomic Calibration	Claim correctness	Post-hoc	Long-form	LLM	–	[97]
	THERMOMETER	Answer correctness	Post-hoc	Single-turn	LLM	✓	[98]
	APRICOT	Answer correctness	Post-hoc	Single-turn	LLM	✓	[99]
	DACA	Answer correctness	Post-hoc	Single-turn	LLM	✓	[100]
	BaseCal	Answer correctness	Post-hoc	Single-turn	LLM	✓	[101]
	MultiTurn-CE	Turn correctness	Post-hoc	Multi-turn	LLM	✓	[102]
	MTCal / ConfChat	Turn correctness	Hybrid	Multi-turn	LLM	✓	[103]
	Agentic CC (HTC)	Trajectory success	Post-hoc	Tool-use	AGENT	✓	[15]
	TPS	Prefix success	Post-hoc	Tool-use	AGENT	✓	[11]
Policy-Facing Recalibration	CTA	Answer correctness	Post-hoc	Tool-use	AGENT	✓	[12]
Runtime Recalibration	CEB	Action productivity	Online	Tool-use	AGENT	✓	[9]
	MARGIN	Agent correctness	Online	Multi-agent	AGENT	✓	[104]
Dependence-Aware Recalibration	CAGE-Cal	Panel correctness	Post-hoc	Multi-agent	AGENT	✓	[24]
B. Training-time calibration							
Supervised Calibration	LACIE	Listener reliance	Preference FT	Single-turn	LLM	✓	[105]
	ConfTuner	Answer correctness	SFT	Reasoning	LLM	✓	[106]
	R-Tuning	Knowledge boundary	SFT	Single-turn	LLM	✓	[107]
	Know-Unknown	Answer correctness	SFT	Single-turn	LLM	✓	[108]
Hybrid Calibration	Linguistic Calibration	User forecast	Hybrid	Long-form	LLM	✓	[109]
	SaySelf	Answer correctness	Hybrid	Reasoning	LLM	✓	[110]
	LoGU	Claim correctness	Hybrid	Long-form	LLM	–	[111]
	DMAD	Agent correctness	Hybrid	Multi-agent	AGENT	✓	[113]
RL / Reward Optimization	Rewarding Doubt	Answer correctness	RL	Single-turn	LLM	✓	[114]
	CONQORD	Answer correctness	RL	RAG	LLM	✓	[115]
	ORCE	Answer correctness	RL	Reasoning	LLM	✓	[116]

Continued on next page

Method Category	Method / Framework	Calibrated Object	Calibration Type	Interaction Regime	Primary Setting	Agent-Applicable	Ref.
	RLCM	Reasoning reliability	RL	Reasoning	LLM	✓	[117]
	LoVeC	Claim correctness	RL	Long-form	LLM	–	[112]
	Reward Calibration	Confidence-quality alignment	Reward-model / RL	Single-turn	LLM	✓	[120]
	IG-Clarify	Clarification utility	RL	Multi-turn	AGENT	✓	[118]
	TRUST	Tool-use reliability	RL	Tool-use	AGENT	✓	[119]
	ConfDichotomy (CAR)	Tool-use reliability	RL	Tool-use	AGENT	✓	[21]
C. Decision-oriented calibration							
<i>Conformal Risk Control</i>	Conformal LM	Coverage risk	Conformal	Single-turn	LLM	✓	[121]
	Conformal Factuality	Factuality risk	Conformal	Long-form	LLM	–	[122]
	CLC	Commitment specificity	Conformal	Long-form	LLM	–	[123]
	SafePath	Trajectory safety	Conformal	Navigation	AGENT	✓	[7]

5.2 Test-Time and Runtime Calibration

Post-training calibration follows a natural sequence. Diagnosis first tests whether an uncertainty signal carries the intended reliability meaning. Offline recalibration then learns a fixed correction under a reference distribution. Runtime recalibration updates that correction as deployment conditions change. We use these three lines as the main organization and treat elicitation, policy-facing use, and dependence-aware correction as complementary cases.

Takeaways

Post-training methods diagnose whether an uncertainty signal has the intended meaning, then calibrate it through either a fixed offline mapping or an adaptive runtime update. The main approaches are:

- **Diagnosis and target identification.** Raw uncertainty is compared with answer correctness, expression faithfulness, or trajectory success. This analysis reveals the mismatch and identifies the reliability event that calibration should represent.
- **Offline recalibration.** A calibration set is used to learn a fixed mapping from raw uncertainty to empirical reliability. The calibrated target can be a claim, answer, turn, complete trajectory, or prefix-conditioned success event.
- **Runtime recalibration.** The mapping is updated from action or agent outcomes observed during deployment. This allows confidence semantics to adapt to new states, distribution shifts, delayed feedback, and heterogeneous agents.

Diagnosis identifies the calibration failure. Before correcting an uncertainty signal, one must establish whether it carries the intended meaning. *UncExpr* evaluates whether elicited confidence tracks answer correctness, while *Faithful Uncertainty* examines whether verbal uncertainty reflects the model’s underlying uncertainty [94, 95]. *Agentic Overconfidence* moves the same diagnosis to tool-use trajectories by comparing agent confidence with eventual trajectory success [96]. These methods do not alter the reported signal. They identify which reliability event is misaligned and establish why recalibration is needed.

Offline recalibration maps existing signals to reliability. Once the target is fixed, offline methods learn a correction from a calibration set. Early methods focus on bounded generations. *Atomic Calibration* operates at the claim level, while *THERMOMETER*, *APRICOT*, *DACA*, and *BaseCal* calibrate answer correctness [97–101]. Multi-turn methods then shift the target from one answer to a sequence of turns. *MultiTurn-CE* and *MTCal/ConfChat* estimate turn-level correctness across an interaction [102, 103]. Agent methods extend the same principle to complete processes. *HTC* maps trajectory dynamics and local stability to terminal success, whereas *TPS* targets the probability of eventual success from each trajectory prefix [11, 15]. The development therefore broadens the calibrated object from answers and claims to turns, trajectories, and evolving prefixes.

Runtime recalibration updates confidence during deployment. An offline mapping assumes that the relation between confidence and reliability remains stable. Deployment breaks this assumption when the agent reaches new states, receives new outcomes, or operates with different collaborators. *CEB* uses hindsight-labeled productive and unproductive actions as retrieved experience for step-level confidence updates [9]. *MARGIN* extends this correction to heterogeneous agent pools. It maintains agent- and confidence-band-specific estimates of empirical correctness from delayed deployment feedback [104]. Runtime calibration therefore treats confidence semantics as an evolving property of the deployed system rather than a fixed mapping learned once.

Complementary methods extend the calibration interface. Several methods act before, after, or around these three main lines. *Just Ask* and *MetaFaith* improve how uncertainty is elicited or expressed, but do not themselves learn a reliability correction [92, 93]. *CTA* supplies an isotonic-calibrated success prior to the policy when choosing between direct commitment and further exploration [12]. *CAGE-Cal* instead corrects collective confidence when communication creates dependence among agent errors [24]. These methods change how uncertainty is produced, consumed, or combined. They complement diagnosis, offline recalibration, and runtime recalibration rather than define the main developmental sequence.

5.3 Training-Time Calibration

Training-time calibration changes how uncertainty is learned rather than correcting it after generation. The central distinction is how reliability supervision enters the training process. Supervised methods learn uncertainty behavior directly from labeled or preference-ordered examples. Hybrid methods use separate stages to establish an uncertainty representation and then align its meaning. Reinforcement-learning methods place calibration inside the reward that shapes generation or agent behavior. We organize this subsection according to these three families in Table 3.

Takeaways

Training-time calibration builds reliability into the learned model rather than correcting its uncertainty afterward. It follows three main training strategies:

- **Supervised calibration.** Labeled or preference-ordered examples directly teach the model to recognize unsupported cases and align its confidence with correctness, knowledge boundaries, or downstream reliance.
- **Hybrid calibration.** An initial stage establishes how uncertainty is expressed. A later preference- or reward-based stage aligns that representation with answer correctness, claim reliability, user interpretation, or multi-agent use.
- **RL optimization.** Calibration is embedded in the learning objective so that uncertainty and behavior are optimized jointly. Proper-scoring, ranking, information-gain, and uncertainty-aware rewards can target responses, reasoning, clarification, tool use, or coordination, while reward correction prevents unsupported confidence from being reinforced.

Supervised calibration learns a direct reliability signal. Post-hoc calibration can reinterpret an uncertainty signal, but it cannot change how the model expresses that signal. Supervised calibration addresses this limit by placing reliability supervision directly into fine-tuning. *R-Tuning* constructs refusal-aware examples that teach the model to answer within its knowledge boundary and abstain beyond it [107]. *Know-Unknown* instead learns uncertainty estimation from a small set of correct and incorrect examples, showing that reliable uncertainty behavior must often be trained rather than elicited by prompting alone [108]. These methods first establish whether the model can recognize unsupported answers. Later methods make the reported signal more explicit and decision-relevant. *ConfTuner* trains verbalized numerical confidence through a tokenized Brier objective. The proper loss aligns the reported value with answer correctness without requiring externally supplied confidence labels [106]. *LACIE* shifts the target from model correctness alone to listener reliance. It constructs listener-aware preference pairs and uses preference optimization to teach both explicit and implicit confidence cues [105]. Supervised calibration therefore progresses from recognizing the knowledge boundary to communicating uncertainty in a form that a downstream user can interpret.

Hybrid calibration separates learning the format from aligning its meaning. Direct supervision can establish how uncertainty should be expressed. It does not always ensure that the expression remains aligned with correctness or downstream decisions. Hybrid methods divide these goals across training stages. *Linguistic Calibration* first uses supervised fine-tuning to produce long-form confidence statements. It then applies reinforcement learning so that these statements support calibrated forecasts by downstream users [109]. *SaySelf* follows the same broad progression. It first trains the model to generate confidence values and self-reflective uncertainty rationales, then uses reinforcement learning to align confidence with answer correctness [110]. Long-form generation further requires uncertainty at the level of individual claims. *LoGU* constructs claim-level uncertainty data through iterative refinement, then combines supervised fine-tuning with direct preference optimization [111]. The common role of hybrid calibration is therefore not simply to add another optimization stage. The first stage makes uncertainty expressible. The second stage determines whether that expression carries the intended reliability meaning.

RL and reward optimization align uncertainty with learned behavior. Calibration becomes a reinforcement-learning problem when uncertainty must be optimized jointly with generation, reasoning, or interaction. At the response level, *Rewarding Doubt* uses a logarithmic proper-scoring reward to penalize both overconfidence and underconfidence [114]. *CONQORD* combines response quality with an order-preserving reward so that better answers receive higher verbalized confidence [115]. *ORCE* separates answer generation from confidence estimation and uses rank-based reinforcement learning to align the ordering of confidence with estimated correctness [116]. *RLCM* moves calibration inside the reasoning process by rewarding a wider confidence margin between correct and incorrect intermediate completions [117]. *LoVeC* extends reward-based calibration to long-form generation by training the model to attach numerical confidence to individual statements [112]. Agent settings change both the calibrated object and the behavior being optimized. *IG-Clarify* rewards questions according to the information they provide about the user’s underlying goal [118]. *TRUST* introduces uncertainty-aware rewards to preserve the separation between reliable and unreliable tool decisions during policy optimization [119]. *CAR* jointly optimizes task accuracy and confidence calibration while accounting for the different effects of evidence and verification tools [21]. *DMAD* extends this principle to multi-agent debate. It trains agents both to express calibrated confidence and to use the confidence reported by other agents during belief revision [113]. Reward-based calibration also depends on the reliability of the reward itself. *Reward Calibration* shows that a reward model may favor confident expressions even when confidence is not supported by response quality. It therefore corrects both reward-model training and reward calculation within the PPO pipeline [120]. This result exposes a final dependency: a policy cannot learn calibrated uncertainty from a reward that systematically favors overconfidence.

Across these three families, calibration enters learning at progressively deeper levels. Supervised methods teach a direct uncertainty behavior. Hybrid methods separate uncertainty expression from reliability alignment. Reinforcement-learning methods make uncertainty part of the objective that shapes responses, actions, coordination, and the supervision used to learn them.

5.4 Decision-Oriented Calibration

Decision-oriented calibration uses conformal prediction to convert uncertainty into a statistically controlled decision rule. Rather than only reporting confidence, it restricts which responses or trajectories the agent may return or execute under a target coverage or risk level.

Takeaways

- **Conformal prediction provides decision-level risk control.** It retains only responses or trajectories that satisfy the calibrated requirement and triggers abstention, filtering, or delegation when no candidate can be supported.

Controlling commitment with conformal guarantees. Even a well-calibrated success probability does not determine how broadly an agent should commit. Conformal methods instead use calibration data to construct an output set, filtering rule, or delegation boundary with an explicit coverage or risk guarantee [7, 121]. *Conformal Language Modeling* provides a transferable precursor by calibrating how many generations

to sample and which candidates to retain so that the returned set contains an acceptable response with a user-specified probability [121]. *SafePath* transfers this principle from candidate responses to autonomous trajectories by conformally filtering high-risk paths and delegating control when no sufficiently safe trajectory can be certified [7]. Decision-oriented calibration therefore controls the breadth of the agent’s permitted commitment rather than merely attaching another confidence value to a single action or trajectory [7, 121].

5.5 Discussion: What Counts as Calibration?

Calibration is not any method that computes uncertainty or improves agent reliability. A method belongs to calibration only when it establishes or adjusts the relation between an uncertainty signal and a specified reliability event, such as answer correctness, action reliability, or trajectory success. The target and granularity of this relation must therefore be stated explicitly. Elicitation exposes an uncertainty signal, while diagnosis tests whether that signal already tracks the intended event; neither operation corrects the relation by itself. Post-hoc calibration learns a fixed correction, runtime calibration updates that correction from deployment feedback, and training-time calibration builds the alignment into the learned model. By contrast, using uncertainty only to select samples, allocate credit, regulate exploration, or weight experience does not constitute calibration unless the objective also aligns the signal with a defined reliability target. A single system may contain several of these operations, so classification should occur at the mechanism level rather than the paper level. Decision-oriented conformal methods remain within calibration because they use calibration data to convert a score into a statistically controlled coverage or risk boundary. Under this definition, calibration answers one precise question: how reliably does an uncertainty signal represent the event it is claimed to predict?

6 Uncertainty Utilization in LLM-Based Agents: Turning Uncertainty into Adaptive Behavior

Uncertainty utilization begins when an uncertainty signal changes system behavior. Quantification makes uncertainty observable, while calibration aligns the signal with a defined reliability event. Neither step alone determines what the system should do next [11, 12]. In standalone LLMs, utilization usually remains local to one generation. It determines whether the model answers or abstains, which candidates it retains, how it revises the current response, or how response-level behavior is optimized [107, 114, 121]. Agents extend this role across the full interaction loop. During State Construction, uncertainty identifies when the decision context is incomplete or unreliable. Clarification supplies missing intent or task constraints. Value-aware information acquisition selects the question or source expected to reduce decision-relevant uncertainty most. Context reorganization filters, re-ranks, or restructures evidence that is already available but poorly represented [22, 124, 127]. During Action Selection, uncertainty controls whether and how the agent commits. Retrieval supplies evidence that is still missing. Abstention or termination blocks a commitment that remains unsupported. Deliberation assigns more reasoning, sampling, or verification to contested decisions. Comparative routing transfers authority to a model or agent judged more reliable for the current choice [13, 104, 140]. During Execution & Feedback, uncertainty responds to consequences that become visible only after an action or simulation. Local expansion reopens an unstable transition and generates alternative continuations from that point. Rollout budget allocation divides a fixed sampling budget between complete trajectories and local branches. Search adaptation uses returned outcomes to expand promising branches, prune weak ones, back-track, or regenerate an action. Recovery repairs a localized failure, while termination stops execution when further attempts are unlikely to justify their cost [3, 5, 137]. During State Update, uncertainty determines which information and experience influence later decisions. Belief and memory revision weakens, corrects, or replaces unsupported claims. Confidence-weighted communication controls how strongly messages from other agents enter the shared state. Delayed credit assignment maps a final outcome back to the earlier decisions that should receive learning pressure. Uncertainty-shaped rewards and exploration then determine which behaviors the updated policy reinforces, suppresses, or preserves [23, 82, 157].

These methods are classified by the intervention they execute rather than by the estimator they use. A signal may be produced at one stage and consumed at another, so its source stage and application stage must remain separate. This organization treats agent uncertainty utilization as closed-loop trajectory control rather than output-local uncertainty handling.

6.1 What Is New in Agent Uncertainty Utilization?

From output-local handling to lifecycle-wide control. The central extension from LLMs to agents is that uncertainty no longer resolves only how to handle one uncertain output, but governs a sequence of interdependent commitments distributed across the task lifecycle [1, 12, 124]. Before an action is selected, uncertainty about instruction sufficiency, user intent, retrieved context, or multimodal consistency can trigger clarification or repair of the constructed state [118, 124, 125, 128]. During Action Selection, uncertainty can determine whether the agent commits to a candidate action, continues exploring, or switches to an alternative plan [12, 133, 134]. During Execution & Feedback, it can trigger retrieval, tool invocation, verification, retry, early stopping, or delegation rather than merely changing the wording of the final response [7, 129, 131, 141]. After feedback is obtained, uncertainty can further support belief revision, memory repair, policy adaptation, or other State Update operations that affect subsequent decisions [22, 127, 157].

From co-located use to cross-stage control. In conventional LLM utilization, the uncertainty signal is often extracted from and applied to the same bounded response, whereas an agent may produce a signal at one stage and consume it at a later stage of the interaction [22, 114, 121]. For example, uncertainty constructed from the initial instruction or retrieved context can influence not only Action Selection but also later execution and state revision [22, 127, 128]. Conversely, token- or response-level uncertainty originating during Action Selection can be propagated forward to control tool execution, reflection, or policy updates [82, 137, 138]. The same uncertainty estimator may therefore serve different utilization roles depending on where its signal enters the loop, making the *uncertainty source stage* and the *application stage* separate dimensions rather than interchangeable labels [104, 141, 157].

From local response utility to trajectory and system utility. Because an uncertainty-conditioned action changes the subsequent interaction history, the usefulness of an agent uncertainty signal cannot be judged only by whether it ranks the current response correctly [11, 12]. Its value also depends on whether it improves eventual task success, avoids unnecessary tool or computation cost, supports recovery from early errors, and maintains acceptable risk over the complete trajectory [1, 7, 15]. In multi-agent systems, utilization additionally includes trust weighting, agent selection, delegation, consensus formation, and communication control, because one agent’s uncertainty can directly alter the beliefs and actions of other agents [104, 136, 202]. Training rewards, process supervision, and credit assignment are therefore important forms of State Update utilization, but they represent only one branch alongside clarification, exploration, tool control, risk intervention, and multi-agent coordination [13, 82, 146]. Agent uncertainty utilization is thus best understood as the use of uncertainty for *closed-loop trajectory control*, whereas LLM uncertainty utilization more commonly remains localized around the production and handling of an individual output [7, 12, 104].

6.2 Agent Loop Formulation

Agent uncertainty should be analyzed within the decision loop of an agent rather than as a property of a single output. Recent LLM-agent work commonly treats agents as systems that combine state or context representation, planning, action or tool use, feedback, and memory-based update. For example, surveys of LLM-based autonomous agents organize agents around modules such as memory, planning, and action [203], while planning-oriented surveys emphasize that LLM agents make sequential decisions through planning, action execution, and feedback-driven adjustment [204]. Reflection- and memory-based agents further show that feedback from previous trials or interactions can be written into memory and reused to improve later behavior [205, 206]. These works suggest a minimal loop for uncertainty analysis: an agent constructs a state, selects an action, executes the action and receives feedback, and then updates its internal state, memory, plan, or policy. Uncertainty may enter at each stage, but its object and consequence differ across the loop. We therefore formulate the agent loop into four stages to clarify where uncertainty arises, what kind of commitment is being made, and how uncertainty may propagate to later decisions.

- **State Construction: uncertain inputs and beliefs.** State construction is the stage where an agent forms its current representation of the task or environment. Before selecting an action, the agent must interpret user instructions, observations, retrieved evidence, tool outputs, memory, and contextual information. These sources may be incomplete, ambiguous, conflicting, noisy, or outdated. This stage is supported by LLM-agent architectures that rely on memory, context, and environmental observations to support later

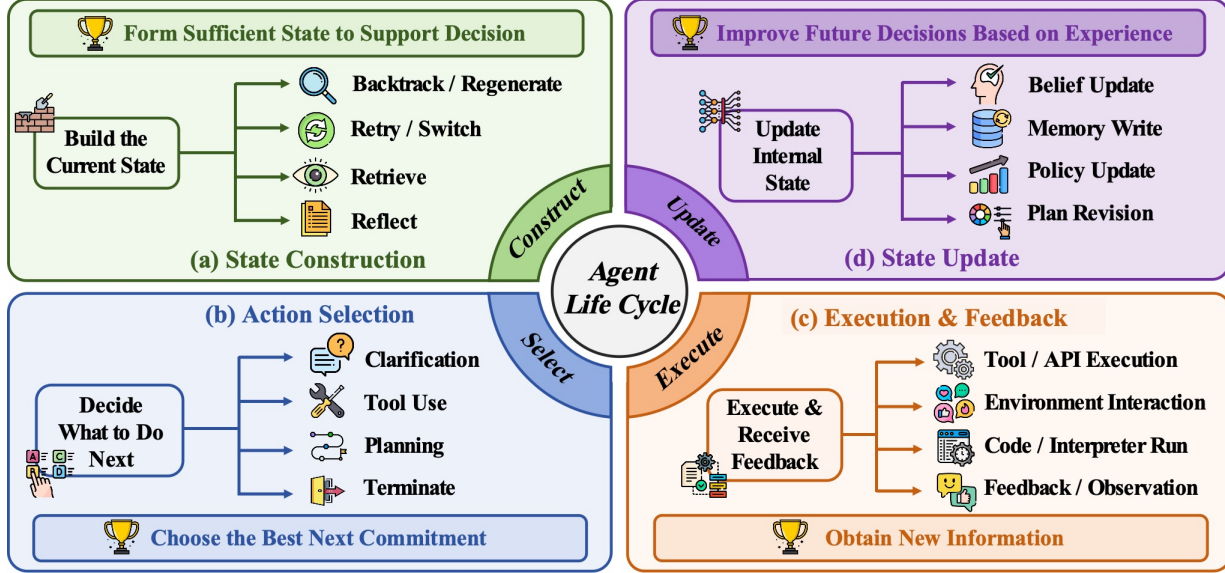


Figure 4: Agent Loop

planning and action [203, 206]. Uncertainty at this stage concerns whether the agent has constructed a reliable state before acting.

- **Action Selection: uncertain choices and plans.** Action selection is the stage where the agent decides what to do next under the constructed state. The selected commitment may be an answer, a clarification request, a retrieval decision, a tool invocation, a plan, or a termination decision. This stage corresponds to the planning and action components emphasized in recent LLM-agent frameworks, where models generate plans or actions based on the current context [203, 204]. Uncertainty at this stage concerns whether the chosen action or plan is sufficiently supported by the current state.
- **Execution and Feedback: uncertain outcomes.** Execution and feedback describe the stage where the selected action is carried out and the agent receives an outcome. Execution may involve tools, APIs, retrieval systems, code interpreters, embodied environments, or other agents. This stage is central to LLM-agent settings where actions interact with external systems and produce observations or feedback for later reasoning [203, 204]. Uncertainty at this stage does not mainly come from the model’s next-token distribution, but from the outcome of interacting with external systems or environments.
- **State Update: revising beliefs, memory, and policy.** State update is the stage where the agent decides how to incorporate new feedback into its internal state, memory, belief, plan, or policy. Reflection- and memory-based agents show that feedback can be transformed into stored experience and reused to improve later decisions [205, 206]. Uncertainty at this stage concerns whether the new information is reliable enough to influence future behavior. State update is not necessarily a new source of uncertainty by itself; rather, it consumes uncertainty signals from earlier stages and determines how they affect future commitments.

Table 4: Taxonomy of uncertainty-related methods for LLM agents, organized by signal granularity, uncertainty source stage, and application stage. **Source** denotes the stage from which the uncertainty signal is computed. **SC**, **AS**, **EX**, and **SU** denote State Construction, Action Selection, Execution, and State Update, respectively. A checkmark under **AS**, **EX**, or **SU** indicates that the uncertainty signal explicitly intervenes at that stage; blank cells denote quantification, calibration, or diagnosis without a stated intervention there.

Method Category	Method	Model	Source	Application Stage			Ref.
				Action Selection	Execution	State Update	
Input-level signal							

Continued on next page

Method Category	Method	Model	Source	Application Stage			Ref.
				Action Selection	Execution	State Update	
Instruction Sufficiency	Instruction Sufficiency Judgment	AwN	SC	✓			[124]
	Underspecification Detection	UA-Multi	SC	✓	✓		[125]
Structured Clarification	Structured Parameter Belief	SAGE-Agent	SC	✓	✓	✓	[22]
	Information Gain	IG-Clarify	SC	✓	✓	✓	[118]
	Request / Action Uncertainty Decomp.	UD-Clarify	SC	✓			[126]
Context / Perception	Component Relevance Scoring	RecAgent	SC	✓	✓		[127]
	Map Confidence + Multi-View Consistency	CARe	SC	✓	✓		[128]
Token-level signal							
Entropy Dynamics / Control	Token-/Turn-Level Uncertainty Dynamics	T ² PO	AS	✓			[13]
	Post-Tool Token Entropy	ARPO	AS		✓	✓	[137]
	Entropy Pre-Monitoring	AEPO	AS		✓	✓	[138]
	Step-Wise Token Entropy	EMPG	AS			✓	[145]
	Combined Token Uncertainty	SELAUR	AS			✓	[82]
	Entropy Reduction	TEPO	AS			✓	[146]
	Entropy-Informed Token Weights	ResT	AS			✓	[147]
	Policy Entropy Preservation	EntroPO	AS			✓	[148]
	Phase-Dependent Policy Entropy	EPO	AS			✓	[149]
	Progressive Policy-Entropy Curriculum	SPEAR	AS			✓	[150]
Probability-Guided Retrieval	Token Uncertainty + Attention	DRAGIN	AS	✓			[129]
Entropy-Shaped Retrieval Branching / Credit	Pre-/Post-Retrieval Step Entropy	RAPO	AS			✓	[151]
	Uncertainty-Likelihood Branching Score	APPO	AS	✓		✓	[23]
Probability-Based	Entropy-Guided Tree Expansion	AT ² PO	AS	✓		✓	[130]
	Single-Output Token Uncertainty	UALA-S	AS	✓			[131]
	Mean Tool-Call Token Probability	AXPO	AS	✓		✓	[139]
	Min. Search-Span Token Probability	β -GRPO	AS			✓	[140]
Likelihood-Based	Scenario Likelihood Scoring	PCE	AS	✓			[132]
Response-level signal							
Sampling / Dispersion	Semantic Entropy	SEED-GRPO	AS			✓	[152]
	Multi-Sample Answer Disagreement	UALA-M	AS	✓			[131]
Tool-Conditioned UQ	Spatial Sample Dispersion	SafeGround	AS	✓			[6]
	Vote Entropy + Top-2 Margin	CATTS	AS	✓			[133]
	Candidate-Solution Entropy	EGSS	AS	✓	✓		[134]
	Joint LLM-Tool Predictive UQ	Tools-in-the-Loop	AS/EX				[141]
Response-Level Learning Signal	Semantic Information Gain	InfoReasoner	AS/EX			✓	[153]
	Response-Level Relative Surprisal	AEM	AS			✓	[154]
Confidence Calibration	Progressive Claim Confidence	Deep Research Agent	AS				[142]
	Deliberative Confidence Calibration	Collaborative Calibration	AS				[143]
	Debate-Based Confidence Calibration	AlignVQA	AS			✓	[155]
	Calibrated Success Prior	CTA	AS	✓			[12]
Decision / Tool Control	Tool-Type-Conditioned Calibration	ConfDichotomy	AS/EX			✓	[21]
	Predictive-Uncertainty Deferral	ReDAct	AS	✓			[10]
	Uncertainty-Separation Reward	TRUST	AS			✓	[119]
	Task-/Action-Level Uncertainty	WebUncertainty	SC/AS	✓	✓		[5]
Multi-Agent Confidence	Confidence-Modulated Debate	DMAD	AS			✓	[135]
	Confidence-Weighted Message Attention	DebUnc	AS			✓	[136]
	Online Confidence-Band Calibration	MARGIN	AS/EX	✓		✓	[104]
Trajectory-level signal							
Verbalized State / Trajectory	Confidence + Semantic Explanation	AUQ	AS	✓	✓	✓	[1]
	Verbalized State Uncertainty	Agent-BRACE	SC	✓		✓	[157]
Trajectory / Debate Learning	Consistency-Volatility over Reasoning Paths	CoVo	AS			✓	[156]
	Hierarchical Debate Uncertainty	UDPO	AS			✓	[2]
Risk / Selective Control	Conformal Path Risk	SafePath	AS	✓			[7]
	Prompted Risk-Aware Quitting	Quitting Agent	AS	✓			[144]

Continued on next page

Method Category	Method	Model	Source	Application Stage			Ref.
				Action Selection	Execution	State Update	
Trajectory Estimation / Evaluation	Sequential Abstention Signal	Agentic Abstention	AS/EX	✓	✓		[3]
	Situation-Weighted UQ Propagation	SAUP	AS/EX				[158]
	Holistic Trajectory Calibration	HTC	AS/EX				[15]
	Success-Probability Self-Assessment	Agentic Overconf.	AS/EX				[96]
	Trajectory Proper Score	TPS	AS/EX				[11]
Multi-Agent Trajectory	Tensor Decomposition	MATU	AS/EX				[20]

6.3 Application in State Construction

State Construction assembles instructions, observations, retrieved evidence, memory, and prior interaction history into the context for the agent’s next commitment. Uncertainty matters at this stage when the constructed state cannot yet support a reliable decision. The failure may arise because information is missing, because possible clarifications differ in value, or because available evidence is noisy, redundant, or poorly organized. Existing methods therefore follow three connected lines. They diagnose whether the state is sufficient, select the information that can repair it most effectively, and reorganize evidence that is already available but not yet represented reliably [22, 124, 127].

Takeaways

At State Construction, uncertainty repairs the decision context before commitment. This occurs in three ways:

- **State-sufficiency diagnosis.** Missing intent, context, observations, or constraints are detected before they become an unsupported action.
- **Value-aware information acquisition.** Clarification questions and information sources are prioritized by their expected decision value and interaction cost.
- **Context selection and reorganization.** Available evidence is filtered, re-ranked, or restructured when the problem lies in its representation rather than its absence.

6.3.1 State Sufficiency and Clarification Triggering

Detecting missing information. The first line prevents an agent from silently converting an underspecified request into an executable commitment. *AwN* introduces a prompted ask-when-needed mechanism. At each tool-invocation round, it encourages the model to request clarification when the instruction lacks information required for reliable execution [124]. This design still places underspecification diagnosis and task execution within the same agent. *UA-Multi* separates these functions architecturally. An Intent Agent checks the state history for missing user intent or repository context, while a Main Agent navigates the repository, edits code, and executes tools [125]. *UD-Clarify* further separates request uncertainty from action confidence and triggers clarification according to the former. This distinction prevents an agent from appearing confident after silently selecting one of several plausible interpretations [126]. The development therefore moves from prompting the agent to notice missing information, to separating diagnosis from execution, and finally to representing the source of uncertainty explicitly.

6.3.2 Value-Aware Information Acquisition

Selecting useful clarification. Detecting that a state is insufficient does not determine which question should be asked. Candidate clarifications may differ in both decision value and interaction cost. *SAGE-Agent* addresses this choice through explicit value-of-information optimization. It maintains a structured belief over candidate tool calls and parameter domains, then selects questions through cost-penalized Expected Value of Perfect Information with an additional penalty for redundant questioning [22]. *IG-Clarify* instead learns a

reusable clarification policy. It measures how an interaction changes the likelihood of the underlying user goal and converts this change into an Information Gain Reward for reinforcement learning [118]. Both methods move beyond ambiguity detection. *SAGE-Agent* computes question value over an explicit candidate space, whereas *IG-Clarify* learns question selection from goal-level belief updates.

6.3.3 Context Selection and Reorganization

Repairing an already populated state. Clarification is appropriate when information is missing. It is unnecessary when the relevant evidence is already present but selected or represented poorly. *RecAgent* addresses observation selection by filtering and ranking GUI components through keyword matching, semantic matching, and LLM-based intent recommendation. It reserves a separate interaction module for preferences that remain unresolved [127]. After relevant observations are selected, *WebUncertainty* changes how the planning context is represented. An Analysis Agent estimates the remaining objectives and task-level uncertainty. The Planning Agent then uses explicit decomposition under low uncertainty and reactive planning when uncertainty remains high [5]. *CARe* extends this repair process to embodied navigation. After an initial map-based plan fails, it preserves the pre-explored semantic map but selects a new candidate region through uncertainty and multi-view consistency [128]. These methods show that state repair need not add new information. It can instead change which existing evidence enters the next decision and how that evidence is organized.

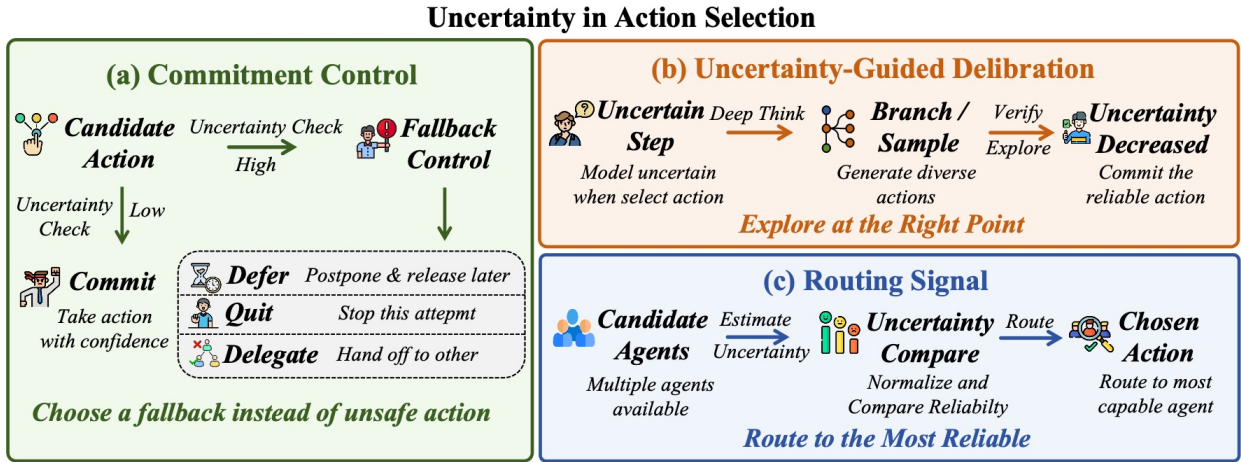


Figure 5: Action Selection

6.4 Application in Action Selection

Action Selection is where uncertainty is most directly converted from a reliability signal into behavioral control. Existing methods can be organized by the intervention induced by uncertainty rather than by the form of the underlying estimator. For commitment control, *Search Wisely* uses confidence to determine whether the agent should rely on internal knowledge or invoke external retrieval, while *SafeGround* prevents uncertain GUI predictions from being immediately converted into difficult-to-reverse actions [6, 140]. For deliberation control, *T²PO* uses uncertainty dynamics to determine when additional reasoning is needed, whereas *CATTS* allocates additional test-time samples only to contested decision points [13, 133]. For comparative routing, *MARGIN* calibrates confidence across heterogeneous agents before allocating decision authority [104]. These three functions provide the roadmap of this subsection: uncertainty can determine whether the agent should commit, where additional computation should be allocated, and which alternative should control the next decision.

Takeaways

At Action Selection, uncertainty controls the agent’s next commitment. It supports three main forms of intervention:

- **Commitment control.** Insufficient support can trigger retrieval, deferral, abstention, termination, or delegation instead of immediate execution.
- **Deliberation and branching.** Contested decisions receive additional reasoning, sampling, search, or verification before the agent commits.
- **Comparative routing.** Comparable reliability estimates are used to assign decision authority across candidate models or agents.

6.4.1 Commitment Control

Support-sensitive commitment. Commitment control asks whether the current evidence supports executing a candidate action. *PCE* extracts assumptions about hidden objects and collaborators from the reasoning process. It then evaluates candidate actions through their plausibility, goal-directed value, and execution cost [132]. *Search Wisely* moves this decision to the boundary between internal reasoning and external information acquisition. It uses confidence-sensitive optimization to penalize both unnecessary search and unsupported non-search decisions [140]. When an action directly changes the environment, *SafeGround* estimates disagreement across stochastic grounding predictions and applies a calibrated threshold before executing uncertain GUI coordinates [6]. This line therefore progresses from evaluating whether an action is supported, to deciding whether more evidence is required, and finally to withholding a consequential action when uncertainty remains too high.

Fallbacks after rejected commitment. Rejecting the current action does not determine what the agent should do instead. Different methods introduce different forms of non-commitment. *ReDAct* transfers a decision from a low-cost model to a stronger executor when uncertainty exceeds a threshold [10]. *Quitting Agent* treats termination as a valid policy outcome when the current trajectory is unlikely to succeed [144]. *Agentic Abstention* makes this choice sequential. It allows the agent to gather more evidence before deciding whether to continue or stop [3]. *SafePath* constructs a conformal set of admissible navigation trajectories and delegates control when no autonomous path satisfies the required safety condition [7]. These methods differ in the fallback they select: deferral changes the executor, abstention preserves the option to continue, quitting ends the trajectory, and delegation transfers authority to an external controller.

6.4.2 Uncertainty-Guided Deliberation and Branching

Deciding whether to deliberate. Uncertainty need not force rejection when it remains reducible through additional computation. *T²PO* monitors uncertainty dynamics across tokens and turns. It triggers additional thinking when uncertainty reduction stalls and resamples turns that contribute little exploration progress [13]. Its intervention therefore addresses the first deliberation question: has the reasoning process made enough progress to justify commitment, or should it receive another opportunity to explore?

Allocating additional computation. Once further deliberation is justified, the agent must decide where and how to spend it. *CATTS* estimates contention from the empirical distribution of candidate web actions. It uses entropy and the margin between the most supported actions to allocate additional samples only when the decision remains unresolved [133]. *EGSS* extends this principle to software engineering. It combines uncertainty-guided search expansion with test-suite augmentation, so that new candidates are accompanied by stronger environment-grounded evidence [134]. The development therefore moves from deciding whether more computation is needed, to locating the contested decision, and finally to choosing whether additional resources should expand the candidate set, strengthen verification, or do both.

6.4.3 Comparative Routing

Routing among available executors. Deliberation generates additional candidates from the current decision-maker. Comparative routing instead chooses among decision-makers that already exist. The central question is which model or agent should receive authority over the next action. This choice becomes difficult when candidate executors report confidence on incompatible scales or fail differently across task distributions. *MARGIN* addresses this problem by continuously recalibrating each agent’s confidence from runtime outcomes.

It then uses the corrected estimates for agent selection and confidence-weighted coordination [104]. Uncertainty becomes a valid routing signal only when the reliability estimates attached to candidate executors are sufficiently comparable.

Separating executor routing from message weighting. Uncertainty can also change how strongly information from another agent affects later reasoning. This intervention does not select the immediate executor. It revises the shared information state. We therefore discuss message-level influence, including *DebUnc*, under Collective Belief Revision in State Update [136]. Comparative Routing remains focused on whether another available executor is sufficiently more reliable to justify transferring decision authority.

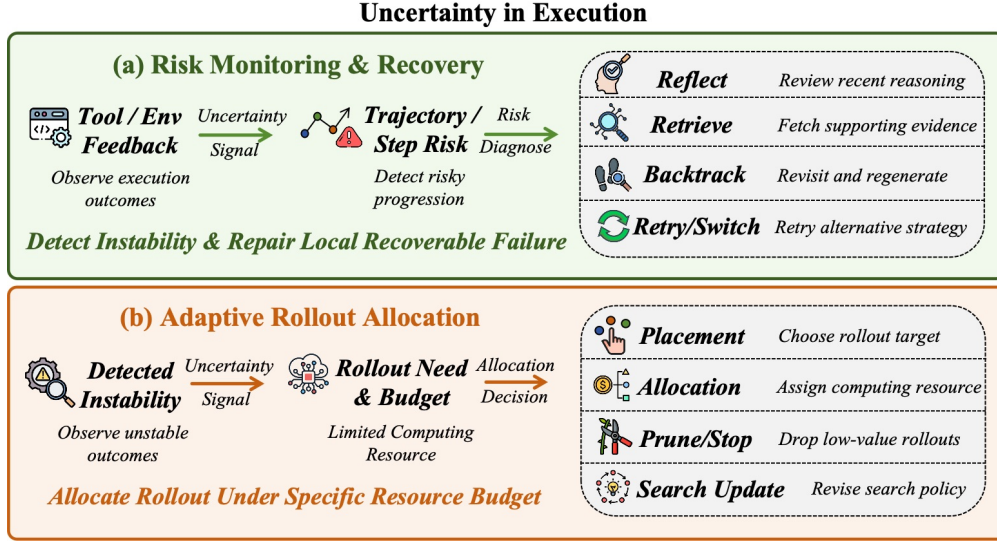


Figure 6: Execution

6.5 Application in Execution

Execution begins when a selected action interacts with an external process and produces evidence that was unavailable at the time of selection. Uncertainty therefore no longer concerns only whether the proposed action was sufficiently supported; it must determine how newly observed or simulated consequences should change the ongoing trajectory. Existing work forms three connected lines. First, *ARPO* uses entropy increases after tool feedback to identify transitions that require local expansion, while *AEPO* further controls how a finite rollout budget should be distributed between complete and partial executions [137, 138]. Second, *WebUncertainty* uses task- and action-level uncertainty to revise planning, branch evaluation, and search behavior as simulated outcomes accumulate [5]. Third, *AUQ* preserves the uncertainty associated with earlier actions as diagnostic information for targeted recovery, whereas *Agentic Abstention* uses evidence accumulated during interaction to determine when continued execution is no longer worthwhile [1, 3]. These three lines provide the roadmap of this subsection: uncertainty can determine where an execution should be expanded, how feedback should revise the active search process, and whether the resulting failure should be repaired or allowed to terminate the trajectory.

Stage boundary. This organization also clarifies the boundary between Execution and its neighboring stages. Generating alternatives before any tool or environmental consequence is available belongs primarily to Action Selection, whereas reopening a transition after feedback has been observed belongs to Execution. For training frameworks, uncertainty-conditioned rollout construction is counted here when it changes which continuations are actually executed, while the resulting parameter update is treated as a downstream consequence of the same framework rather than as a second methodological leaf. By contrast, a mapping that only evaluates completed trajectories without specifying an online intervention remains a calibration mechanism [15, 137, 138].

Takeaways

During Execution, uncertainty turns newly observed consequences into online trajectory control. Its main roles are:

- **Feedback-induced expansion and budget allocation.** Local instability identifies which transitions require additional branches and where a limited rollout budget should be spent.
- **Search and plan adaptation.** Task- and action-level uncertainty determines whether the agent should expand, prune, backtrack, regenerate, or revise the active plan as new outcomes accumulate.
- **Targeted recovery and termination.** Uncertainty provenance supports focused repair, while accumulated evidence indicates when continued execution is unlikely to be worthwhile.

6.5.1 Feedback-Induced Expansion and Budget Allocation.

Localizing feedback-induced instability. External feedback can expose uncertainty that was not visible when the tool action was initially selected, making the first execution problem one of localization: the agent must identify which interaction boundary has become unstable before deciding where to generate additional experience. *ARPO* observes that token entropy rises sharply immediately after tool feedback, with open-ended search outputs producing larger fluctuations than more deterministic interpreter results. It uses this signal in an adaptive rollout mechanism that balances complete-trajectory sampling with local step-level expansion at high-entropy tool interactions, allowing alternative continuations to reuse the preceding reasoning history rather than restarting the entire trajectory. Advantage attribution over these locally expanded transitions then connects the generated experience to subsequent policy optimization [137].

Allocating a finite rollout budget. Local expansion becomes inefficient, however, when every high-entropy tool transition independently requests more branches. *AEPO* addresses this pressure by treating uncertainty as a budget-allocation signal rather than an unrestricted expansion trigger. Its entropy pre-monitoring mechanism divides a fixed sampling budget between global trajectories and local branches, while a penalty on consecutive high-entropy tool calls prevents repeated expansion along the same unstable path. The framework further preserves the learning value of the resulting experience through entropy-aware advantage estimation and gradient treatment, but its execution-level contribution is the regulation of which uncertain transitions receive additional rollout resources [138]. The progression is therefore from detecting where tool feedback destabilizes a continuation to deciding how much of a limited execution budget that region should receive.

6.5.2 Feedback-Conditioned Search Adaptation.

Task-level planning adaptation. Allocating additional branches does not yet determine how the search process should respond to the evidence returned by those branches. The controller must distinguish a state in which no candidate is sufficiently supported from one in which several candidates remain plausible, because these uncertainty regimes call for different revisions. *WebUncertainty* addresses this problem through a dual-level controller. At the task level, an Analysis Agent estimates uncertainty from the instruction, current webpage observation, interaction history, and remaining objectives; the Planning Agent then selects explicit task decomposition in more familiar states and reactive stepwise planning when the task remains uncertain [5].

Action-level search revision. Within the same framework, action confidence is normalized into a distribution over candidate web actions. Average confidence is used as a proxy for total evidence strength, whereas normalized entropy represents competition among available candidates. Low evidence is consequently treated differently from high competition under stronger evidence. As MCTS simulations return new outcomes, this distinction determines whether the agent should continue exploration, suppress an unproductive branch, backtrack to an earlier state, or regenerate the proposed action [5]. Planning adaptation, branch evaluation, and recovery are therefore not separate uses of the same paper; they are connected components of one search controller in which the diagnosed uncertainty regime determines how simulated feedback changes the ongoing execution.

6.5.3 Targeted Recovery and Sequential Termination.

Recoverable failures. Search adaptation remains appropriate only while the failure appears locally recoverable. When the execution has already accumulated an unreliable commitment, recovery additionally requires information about what was uncertain and why, rather than only a scalar indication that the current trajectory is risky. *AUQ* jointly generates an action, a verbalized confidence value, and a semantic uncertainty explanation, and stores these elements together with the corresponding observation in an Uncertainty-Aware Memory. When confidence falls below a threshold, Uncertainty-Aware Reflection reuses the stored explanation as a diagnostic cue for targeted reflection, retrieval expansion, or tool switching [1]. The method therefore links uncertainty provenance to a specific recovery operation, avoiding both passive risk reporting and indiscriminate reflection over the entire trajectory.

Irrecoverable trajectories. Not every failure remains recoverable, because interaction may reveal that the requested goal cannot be achieved with the available environment, tools, or evidence. *Agentic Abstention* formulates this case as a sequential decision problem in which the agent may continue acting, gather more information, answer, or abstain at each turn. The need to stop may therefore become visible only after apparently reasonable actions fail to produce a feasible continuation. Its CONVOLVE mechanism distills completed interaction trajectories into reusable stopping guidance, allowing later agents to recognize similar infeasible situations before incurring the same sequence of unproductive tool calls [3]. Execution control consequently progresses from repairing a localized failure to recognizing when further repair has negative expected value and should give way to termination.

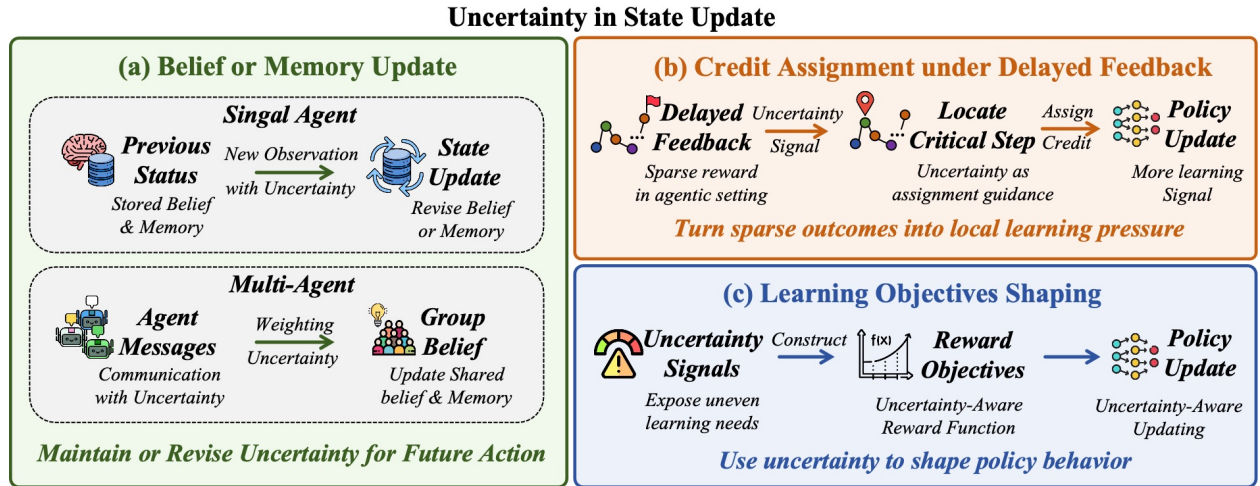


Figure 7: State Update

6.6 Application in State Update

State Update determines which consequences of an interaction remain available to future decisions. This persistence occurs at two different timescales. At runtime, *Agent-BRACE* revises a compact belief state whose claims carry explicit certainty, while *Confidence-MAD* makes collaborative belief revision depend on the confidence communicated by other agents [135, 157]. Across episodes, *AT²PO* and *APPO* localize delayed learning pressure within branched trajectories [23, 130], whereas *SELAUR* and *TRUST* illustrate how uncertainty can reshape the rewards that determine which behaviors become persistent in the policy [82, 119]. These functions provide the roadmap of this subsection: uncertainty can revise an internal belief state, change how communicated evidence enters a collective state, locate where a delayed outcome should affect learning, or shape what the updated policy should prefer.

Takeaways

At State Update, uncertainty determines what information and experience should shape future decisions. Its main roles are:

- **Belief and memory revision.** New evidence can retain, weaken, correct, or replace claims in the state used by later decisions.
- **Collective belief weighting.** Confidence attached to inter-agent messages controls how strongly each message influences the shared reasoning state.
- **Delayed credit assignment.** Uncertainty helps localize where terminal outcomes should exert learning pressure and how strongly the corresponding experience should update the policy.
- **Learning-objective shaping.** Uncertainty-aware rewards can prioritize epistemic progress, suppress unreliable experience, preserve useful exploration, or maintain the distinction between reliable and unreliable actions.

6.6.1 Runtime Belief and Memory Update.

Runtime State Update must do more than retain the complete interaction history; it must represent which beliefs remain supported after new evidence arrives. *Agent-BRACE* makes this state transition explicit by representing the environment as atomic natural-language claims annotated with ordinal certainty labels. New observations revise the affected claims, and the policy conditions on the resulting compact belief state rather than repeatedly reasoning over the entire trajectory [157]. Uncertainty is therefore not merely stored as an auxiliary trace: it is attached to the representation that determines the agent’s next decision.

This criterion distinguishes persistent state revision from immediate reflection or recovery. A recovery mechanism may use an uncertainty trace to repair the current action, but it becomes State Update only when the revised representation remains available as part of the state consumed by subsequent decisions [1, 157].

6.6.2 Collective Belief Revision.

Uncertainty-weighted collective revision. In multi-agent systems, State Update also determines how information received from other agents changes the beliefs that condition later decisions. *DebUnc* attaches uncertainty estimates to inter-agent messages and uses confidence-sensitive attention or explicit confidence cues to modulate their downstream influence. A communicated answer is therefore not incorporated as an equally reliable premise by default; its uncertainty determines how strongly it enters subsequent reasoning [136]. *Confidence-MAD* makes this update mechanism more explicit by having agents communicate calibrated numerical confidence and condition later belief revisions on the confidence reported by their peers. Its diversity-aware initialization determines which candidate hypotheses enter the debate, while confidence determines how strongly those hypotheses influence later rounds [135]. The progression is therefore from attaching reliability information to individual messages to using that information as an explicit control on the evolution of the shared reasoning state.

Related multi-agent methods should be separated according to whether uncertainty is actually the update signal. Deliberative calibration and aggregation methods that revise collective answers or confidence estimates without maintaining a persistent uncertainty-bearing state are treated under Calibration [143, 155]. Methods that retain messages according to disagreement or organize communication through answer communities are useful comparisons, but they do not perform confidence-weighted belief revision [202, 207].

A remaining challenge is that confidence-weighted revision is reliable only when confidence provides meaningful comparative evidence across participants. Agents that share model families, prompts, retrieved evidence, or earlier messages may become confidently wrong for the same reason, causing repeated updates to reinforce rather than correct a common error. Collective State Update therefore requires uncertainty-aware influence to be considered together with dependence among information sources, rather than treating multiple confident agents as independent evidence [135, 136].

6.6.3 Credit Assignment under Delayed Outcomes.

Constructing counterfactual experience. Before a delayed outcome can be assigned to individual decisions, the learning process may need to construct experience that isolates an uncertain transition. *ARPO* uses entropy changes around tool interactions to create partial branches at unstable decisions while allowing the alternatives to share the preceding reasoning history [137]. *AXPO* narrows this corrective construction further: when all rollouts following one reasoning prefix fail, it preserves the prefix and resamples only the tool action and its continuation before optimizing over the resulting alternatives [139]. Uncertainty therefore first determines which local counterfactual experience should enter the update rather than requiring the entire trajectory to be regenerated.

Localizing delayed credit. Once a branched experience structure is available, the delayed outcome must be mapped back to the decisions that produced it. *AT²PO* uses a turn-level interaction tree to propagate sparse trajectory rewards back to the turns responsible for expanded branches [130]. *APPO* instead combines local uncertainty with policy-induced likelihood changes over future continuations to identify finer-grained consequential positions and rescale procedure-level advantages around them [23]. The two approaches localize learning pressure at different granularities: one follows the explicit turn-level branch structure, whereas the other estimates which individual positions exert the greatest influence on later behavior.

After the responsible experience has been located, uncertainty can further determine how strongly that experience updates the policy. *AEPO* uses entropy-aware advantage estimation and gradient treatment to stabilize learning around uncertain regions [138], whereas *EMPG* jointly conditions gradient modulation on uncertainty and terminal outcome, strengthening confident correct actions, penalizing confident errors, and attenuating highly uncertain steps [145]. The former regulates optimization around unstable experience, while the latter assigns different update semantics to uncertainty depending on whether the final trajectory succeeds or fails.

This modulation also raises the question of which granularity should carry the learning signal. *ResT* constructs entropy-informed token weights and progressively reallocates training pressure from structured tool-format tokens toward semantic reasoning tokens [147]. *AEM* instead argues that the environment reacts to complete agent responses and therefore derives a response-level uncertainty proxy to rescale the policy update [154]. These approaches expose a trade-off between precise token-level localization and response-level signals that better match the unit of environmental interaction.

A persistent difficulty is that uncertainty does not necessarily identify causal responsibility for the final outcome. A difficult but recoverable step may be highly uncertain without causing failure, whereas a confident early commitment may determine the remainder of an unsuccessful trajectory. Current methods therefore rely on branch structure, continuation likelihood change, entropy, or response-level surprisal as attribution proxies, leaving causal credit assignment under sparse feedback unresolved [23, 145, 154].

6.6.4 Uncertainty-Shaped Learning Objectives.

Rewarding epistemic progress. Credit assignment determines where an observed outcome should exert learning pressure, whereas uncertainty-shaped objectives determine what behavior the updated policy should prefer. A first line rewards external interactions that produce epistemic progress. *InfoReasoner* estimates semantic information gain over belief states and rewards retrieval steps that reduce epistemic uncertainty [153]. *TEPO* similarly rewards tool interactions according to the uncertainty reduction they produce in subsequent predictions [146]. *SELAUR* extends this principle across the trajectory by injecting token-level uncertainty into failure-aware step- and trajectory-level rewards, providing denser supervision when terminal feedback alone is sparse [82]. The progression is therefore from rewarding a specific information-gathering action, to rewarding the uncertainty reduction caused by that interaction, and finally to using uncertainty throughout the trajectory as an additional supervision signal.

Regulating learning intensity and exploration. A second line uses uncertainty to regulate how strongly and how broadly the policy learns from collected experience. *SEED-GRPO* uses semantic entropy across sampled answers to modulate policy updates according to prompt-level uncertainty [152], while *CoVo* combines consistency with the eventual answer and volatility toward competing answers to favor stable but non-collapsed

reasoning trajectories [156]. This control of update strength connects naturally to the broader exploration schedule. *SPEAR* gradually shifts from intrinsic tool-exploration rewards toward self-imitation of successful experience as training progresses [150]; *EPO* uses phase-dependent entropy-aware optimization to avoid both premature convergence and excessive late-stage exploration [149]; and *EntroPO* preserves policy entropy so that multi-turn coding agents retain enough trajectory diversity for later search [148]. Together, these methods show that uncertainty is not always a quantity to minimize: depending on the learning stage, it can concentrate updates, preserve useful diversity, or determine when exploration should give way to consolidation.

Shaping multi-agent and tool-use objectives. The same principle becomes more structured when uncertainty concerns interacting agents or the reliability semantics of the uncertainty signal itself. *UDPO* constructs agent-specific rewards from intra-agent revision, inter-agent conflict, and system-level uncertainty, so that less stable participants receive stronger stabilization while the group continues to optimize a shared task objective [2]. *TRUST* uses an uncertainty-based repulsive reward and key-turn supervision to prevent reinforcement learning from collapsing the distinction between reliable and unreliable tool actions [119]. *ConfDichotomy* further incorporates calibration into tool-agent optimization after observing that evidence and verification tools induce different confidence behaviors [21]. These objectives therefore do more than reduce uncertainty: they determine which uncertainty-aware behaviors and reliability semantics become persistent properties of the learned policy.

6.7 Discussion: Classifying and Evaluating Uncertainty Utilization

Utilization begins when uncertainty changes the agent loop. An uncertainty estimate may identify an unreliable state, action, or trajectory, but diagnosis alone does not constitute utilization. The signal becomes a utilization mechanism only when it changes what the agent observes, selects, executes, retains, or learns. A structured decomposition therefore remains quantification until an explicit rule uses it to clarify, retrieve, reject, expand, route, recover, revise, or update [20]. Classification should follow this direct intervention rather than the estimator that produces the signal. The same estimator may support several behaviors, and one system may contain mechanisms from several stages. These mechanisms should be classified separately rather than assigning the entire paper to a single category.

The application stage is determined by when the intervention acts and what it changes. State Construction uses uncertainty to repair the decision context before a commitment is formed. Action Selection uses it to control whether the agent commits, spends more computation, or transfers decision authority. Execution begins once a tool, environment, simulation, or other agent returns new consequences. Uncertainty then changes the active trajectory through expansion, pruning, backtracking, regeneration, recovery, or termination. State Update begins when the effect persists beyond the current intervention. It changes a belief, memory item, message weight, credit signal, learning objective, or policy that will influence later decisions [22, 137, 140, 157]. This boundary separates the stage where uncertainty is computed from the stage where it changes behavior and prevents one signal from being counted repeatedly simply because it appears throughout the loop.

Evaluation must match the behavior induced by uncertainty. A strong uncertainty score does not establish that the resulting intervention is useful. State repair should be evaluated by whether clarification or information acquisition improves later decisions relative to its interaction cost. Commitment control should report both reliability and the coverage lost through withholding, abstention, or delegation [6, 7]. Deliberation and execution should be compared under fixed reasoning, sampling, and tool budgets, because their value depends on whether additional computation changes the outcome rather than merely creates more branches [133, 134, 138]. Routing should show that transferring authority improves downstream performance, not only that it selects the most confident executor [104]. State updates should instead be judged by their persistent effects, including belief correction, credit localization, optimization stability, retained exploration, and generalization to later tasks [21, 23].

Useful utilization depends on uncertainty source, recoverability, and persistence. Similar uncertainty values can arise from different failures. An unstable transition may reflect an unsupported earlier action, noisy tool feedback, environmental stochasticity, incorrect evidence integration, or correlated errors among communicating agents. These causes do not justify the same response. Additional search may repair miss-

ing evidence, while a shared grounding error may require re-observation and an infeasible trajectory may require termination. Uncertainty should therefore be evaluated by whether it selects an intervention suited to the failure and whether that intervention improves the trajectory at an acceptable cost [1, 3, 5]. Utilization also does not always require uncertainty to decrease. Information acquisition may reward uncertainty reduction, while exploration-oriented learning may preserve uncertainty to avoid premature policy collapse [146, 148, 150, 153]. Other methods use uncertainty to reweight evidence or regulate update strength rather than minimize it directly [136, 145]. The common criterion is decision value: uncertainty utilization is successful when the induced change produces a more reliable trajectory, state, or learned policy.

7 Applications

The practical value of uncertainty becomes visible when it changes a consequential decision. Accordingly, this section compares applications along four dimensions: the commitment at stake, the uncertainty that determines whether the commitment is supported, the intervention triggered when support is insufficient, and the evidence that the intervention improves the relevant outcome. This decision-centered view separates the operational use of uncertainty from the mere reporting of confidence and provides a common basis for examining AI4Science, healthcare, VLA and robotics, and software and web agents. The maturity of the evidence, however, differs across these domains. AI4Science, healthcare, and robotics inherit established traditions in predictive uncertainty quantification, active learning, and classical decision making, but much of this work stops before uncertainty enters a sequential autonomous loop. Software and web agents connect uncertainty more directly to clarification, verification, planning, and recovery, although they are often evaluated in controlled digital environments. The discussion therefore distinguishes mature domain-level uncertainty methods from agent-level mechanisms that use uncertainty to govern sequential decisions.

7.1 AI4Science: From Experimental Acquisition to Scientific Claims

Uncertainty has long selected what gets tested; scientific agents now extend that role across the discovery loop. Scientific discovery faces a basic allocation problem. Experiments, simulations, and measurements are expensive, while the candidate space is often too large to search exhaustively. Bayesian optimization and active learning use predictive uncertainty to determine where the next unit of experimental or computational budget should be spent. This principle now supports molecular simulation, physical measurement, and low-data drug discovery [208–212]. Classical acquisition assumes an enumerable design space, a fixed objective, and a specified acquisition rule. Uncertainty is therefore attached to a predictive model and can often be validated through held-out prediction. Self-driving laboratories close this loop physically, but they often retain the same assumptions. Human scientists still define the hypothesis space and the experimental objective [213, 214]. Agentic systems change this division of labor. An agent may generate a hypothesis, design a method to test it, execute that method through an instrument or simulator, and incorporate the result into the next iteration. This shift is now visible across biology, chemistry, and materials science [215–220]. Uncertainty must therefore support more than candidate acquisition. It must remain useful as the system constructs and revises the scientific problem itself.

Agentic science relocates uncertainty to hypotheses, experimental designs, and scientific inferences. Once the design space is generated rather than enumerated, candidate-level predictive uncertainty is no longer sufficient. Hypothesis uncertainty asks whether a proposed explanation is worth testing. Design uncertainty asks whether the selected protocol can distinguish that explanation from its competitors. Inference uncertainty asks whether the resulting measurement supports the claim that the agent is about to make. The A-Lab case shows why the final transition matters. The system reported the autonomous synthesis of a large set of new inorganic compounds in 2023 [217], and the paper was corrected in 2026 [221]. Solid-state chemists had argued that many of the reported materials were not new compounds. They were already-known compositionally disordered forms of the ordered targets [222]. The failure arose during automated interpretation of X-ray diffraction data. The assigned structures were not adequately supported by the measurements. The weak link was therefore neither target selection nor robotic synthesis. It was the transition from measurement to scientific claim. The system reported no anomaly, and the problem surfaced only after domain experts re-examined the published results. This is precisely the scrutiny that autonomous discovery aims to econo-

mize. When inference uncertainty remains hidden, unsupported conclusions can leave the laboratory, enter the literature, and shape later research.

Different uncertainty sources trigger refinement, measurement, verification, and abstention. Current systems begin to attach uncertainty to each stage of the scientific loop. At the hypothesis level, *HypoAgents* maintains a posterior over candidate hypotheses. It evaluates novelty, relevance, and feasibility, updates the posterior from retrieved evidence, and uses entropy to decide which hypotheses require further refinement [223]. At the design level, *AIMS* uses uncertainty inside a closed physical loop. It identifies weak observations, selects new measurement locations, and distinguishes between competing explanations of a physical system [224]. *MDForge* instead shapes simulation-pipeline construction through a verbal reward because even one candidate execution may be too costly for unrestricted trial and error [225]. At the inference level, abstention-aware verification decomposes a scientific claim into minimal conditions, checks each condition against the evidence, and abstains when that evidence does not license an answer [226]. Related biomedical systems estimate uncertainty over retrieval and generation, then filter unsupported summaries before they enter downstream analysis [227]. Language-model-based optimizers serve a narrower role. They improve the surrogate behind classical acquisition rather than governing hypothesis generation, experimental design, or claim verification [228].

Experimental acquisition is mature; autonomous scientific inference is not. Inference-level uncertainty has received the least attention. A scientific agent may reach the correct answer through unsupported reasoning, while final-answer evaluation fails to expose the weak inference [229]. A recent audit examines thirty-five autonomous research systems, including nine that operate in a closed loop. In each closed-loop system, the internal checker reports a result without quantifying how far that result can be trusted. No external party has independently validated the checker itself [230]. This gap is especially consequential in science. Coding and mathematical agents can often evaluate candidates repeatedly through tests or formal checkers. A scientific agent cannot. Its final authority is an experiment that may be slow, costly, and difficult to repeat. Surrogate-model uncertainty already supports experimental acquisition well. The less mature problem begins after acquisition, when an autonomous system must decide which expensive action to take and whether the resulting evidence licenses a scientific claim.

7.2 Healthcare: Defining the Boundary of Automation

Uncertainty defines whether a clinical case should remain automated. Healthcare decisions have asymmetric consequences. An incorrect automated decision may cause harm, while unnecessary referral may increase cost and delay. A clinical system must therefore do more than produce a diagnosis or risk score. It must determine whether the current patient evidence supports the proposed decision and whether the model remains reliable for that case. Research on language-model uncertainty for electronic health records studies this problem in both white-box and black-box settings [231]. These methods expose predictions that should not be treated as equally reliable. Distributional fit adds another requirement. A prediction may be expressed with high confidence even when the patient lies outside the conditions under which the model was validated. Agentic triage makes the behavioral role explicit. *AT-CXR* estimates confidence and distributional fit for each chest X-ray case. It either produces an automated decision or abstains and requests human review [232]. Uncertainty therefore sets the boundary between autonomous prediction and professional oversight.

Confidence also routes expertise, consultation, and human handoff. Clinical reasoning may require several forms of expertise. A system must decide whether its current evidence is sufficient, whether another specialist should be consulted, and whether a clinician should retain the final decision. Recent multi-agent medical systems combine task-specific competence with self-assessed confidence. They use these signals to recruit expert agents, combine their judgments, or request an additional review [233]. This development moves uncertainty beyond a score attached to one diagnosis. It becomes a routing signal for triage, consultation, and handoff. Direct agent-specific evidence nevertheless remains limited. Many studies use retrospective datasets, simulated consultations, or proof-of-concept multi-agent systems. Healthcare UQ is mature at the predictive level, but uncertainty-aware clinical control remains early. Its clearest current role is to limit unsupported autonomy rather than replace professional judgment.

7.3 VLA and Robotics: From Multimodal Evidence to Physical Commitment

Visual evidence creates uncertainty that decoder-side signals cannot diagnose. A vision–language model must read an image rather than rely only on parametric knowledge. Uncertainty therefore concerns both whether the image supports the answer and whether the model actually used that evidence. Support can fail in three distinct ways. The image may lack the information required by the task. The visual encoder may fail to represent information that is present. Correctly encoded evidence may also lose to the language prior during generation [234]. Common LLM uncertainty signals cannot separate these failures. Token probabilities, sampled consistency, and verbalized confidence are produced by the decoder after the visual evidence has or has not influenced the representation. They inherit the decoder’s final stance rather than test its visual basis. Reliance on the image can decline as the response becomes longer [235]. Repeated samples also share the same visual encoding and language prior, so a shared failure may appear as agreement. Visual evidence provides a direct diagnostic handle because it can be removed, degraded, localized, or replaced. Quantification can perturb the evidence and measure how strongly the answer depends on it [236]. It can also attribute uncertainty to failures of perception, recognition, or external knowledge [237]. Calibration can test whether confidence responds when the relevant visual region is degraded. Utilization can then trigger an intervention unavailable to a language-only agent: looking again rather than reasoning longer [238]. The useful property is often the source of uncertainty, not only its magnitude. Different sources support different interventions. Current methods, however, still tend to bind one signal to one predefined response.

VLA uncertainty spans perception, language grounding, planning, and action. A vision–language–action policy must interpret a language goal, ground that goal in visual observations, and convert the grounded state into an executable action. Failure at one stage can distort every stage that follows. An unsupported goal interpretation may direct perception toward the wrong evidence. Incorrect grounding may then produce a fluent and stable action sequence. Final action confidence can therefore conceal weak premises earlier in the chain. Current work begins to connect uncertainty with concrete VLA decisions. Sequential calibration methods estimate task-success confidence from action-probability sequences and update that estimate throughout an episode [239]. Test-time methods use self-uncertainty to adapt both visual inspection and action execution, rather than resampling actions while leaving perception fixed [240]. Other methods estimate epistemic uncertainty from disagreement in generative action models. The signal supports failure detection during deployment and selects costly expert demonstrations for active fine-tuning [241]. These methods show how uncertainty can trigger a new observation, revised grounding, replanning, conservative execution, failure detection, or model adaptation. They also show the current boundary: individual uncertainty sources increasingly affect behavior, but end-to-end uncertainty use across the full perception–language–action chain remains fragmented.

Uncertainty decomposition matters only when it changes the intervention. Action dispersion should first be interpreted against task-relevant outcomes. Several distinct motions may complete the same task. A controller that penalizes variation may therefore reject valid alternatives. The reverse failure is also possible. Highly consistent actions may all inherit the same incorrect visual grounding or goal interpretation. Agreement can then conceal a shared error. This does not make end-to-end action-level signals inherently insufficient. After temporal-difference calibration, *TDQC* obtains competitive success estimates from sequences of action probabilities [239]. Such a signal may be enough when the only decision is whether to act or defer. Predicting failure, however, does not identify how the failure should be resolved. An occluded target may require a new viewpoint. An ambiguous object reference may instead require clarification rather than additional action sampling. Related multimodal planning work separates perception uncertainty from decision uncertainty to support active sensing and model refinement [238]. Source-specific uncertainty is therefore useful when it distinguishes interventions, not simply when it exposes more intermediate scores. The key comparison is whether decomposition improves recovery over an end-to-end controller under the same observations, intervention options, and computation budget.

Cross-stage dependence limits local confidence and safety guarantees. A local confidence estimate is meaningful only relative to the assumptions it takes as given. A planner may be confident that a motion reaches its selected object without resolving whether that object matches the user’s intent. The score evaluates execution under a grounding assumption; it does not validate that assumption. *KnowNo*, for example, provides task-completion guarantees with human assistance at the planning level [242]. Its formulation assumes that

objects are already grounded in the textual input and that proposed actions execute successfully. The guarantee therefore does not cover uncertainty introduced by visual perception or low-level control. Combining stage-specific scores requires tracking which upstream assumptions each score treats as fixed. The system must also preserve unresolved perceptual or semantic ambiguity without treating every uncertain observation as a reason to stop. Intervention thresholds must reflect physical consequences. A recoverable missed grasp and a damaging collision may share a failure label, yet differ sharply in severity and reversibility. Evaluation should therefore measure whether the controller reduces unsafe actions without rejecting valid alternatives. It should also account for recovery success, unnecessary interruptions, and sensing or computation delay.

Robotics uses uncertainty to regulate embodied commitment. Robotic systems may separate instruction interpretation, perception, planning, and execution into distinct modules rather than use one end-to-end VLA policy. They nevertheless face the same operational question: whether the current information supports physical action. *Ask-to-Act* uses uncertainty to choose between acting and requesting clarification when the intended goal remains unclear [243]. *PCE* exposes hidden assumptions and evaluates candidate actions through scenario likelihood, task value, and execution cost [132]. *ReDAct* instead uses predictive uncertainty to decide whether the current model should act or defer to a stronger executor [10]. These methods connect uncertainty to three robotic interventions: repairing an incomplete task state, revising a fragile plan, and transferring control before execution. Evidence for such mechanisms is growing, but it remains concentrated in simulation and bounded embodied tasks. Their reliability under open-world physical interaction remains less established.

7.4 Software and Web Agents: Clarifying and Verifying Executable Workflows

Specification uncertainty prevents premature executable commitments. Coding and web agents act on artifacts that can be compiled, tested, deployed, clicked, or submitted. An unsupported decision therefore does more than produce an incorrect sentence. It modifies a repository or changes the state of an external interface. Coding failures may arise from weak implementation ability, but they may also arise from incomplete requirements or missing repository context. These causes require different responses. Additional reasoning may help with a difficult implementation. It cannot determine which of several underspecified changes the user intended. *Ask or Assume?* separates underspecification diagnosis from code execution. The agent requests additional information when the issue does not uniquely determine the intended change, but avoids unnecessary questions when the specification is already sufficient [125]. Structured uncertainty therefore determines whether the agent should modify the repository or first repair the task specification.

Uncertainty localizes verification, planning, and recovery. Software development contains several dependent stages. Planning, implementation, testing, and review consume one another’s outputs. An unreliable intermediate artifact can therefore propagate through the remaining workflow. *UA-ChatDev* estimates confidence across development phases and triggers retrieval-based verification when an intermediate result remains uncertain [244]. Line-level hallucination detection localizes the suspected error and supports targeted correction rather than complete regeneration [245]. Web agents face a related distinction. Their uncertainty may concern the overall task plan or one browser action. Task-level uncertainty can select the planning strategy, while action-level uncertainty can allocate additional search among candidate interactions [5]. The agent can therefore reconsider a plan or action before an incorrect click, navigation decision, or submission changes the environment. Software and web environments provide some of the clearest evidence for uncertainty-guided agent behavior because tests and environment feedback expose whether clarification, verification, or retry improved the outcome. This advantage is also a limitation. Deterministic tests and benchmark environments provide unusually clear feedback. Success under these conditions does not establish that the same mechanisms remain reliable when outcomes are delayed, ambiguous, or only partially observable.

7.5 Conclusion

The relevant uncertainty is defined by the commitment it must support. The domains above make different commitments. AI4Science selects experiments and determines whether evidence licenses a scientific claim. Healthcare decides whether a case remains automated or moves to professional oversight. VLA and robotics decide whether multimodal evidence supports physical action. Software and web agents decide

whether an executable modification or interaction is sufficiently supported. Their relevant uncertainty therefore differs. Scientific systems face uncertainty over hypotheses, designs, measurements, and inferences. Clinical systems face uncertainty over case applicability and evidence sufficiency. VLA and robotic systems face uncertainty across perception, grounding, planning, and execution. Software and web agents face uncertainty over task specifications, context, candidate actions, and verification evidence.

Application value is demonstrated by the intervention, not by the uncertainty score alone. Across domains, uncertainty can select an experiment, acquire another measurement, abstain from a diagnosis, request a specialist, look again, re-ground an object, revise a plan, clarify a software requirement, strengthen a test, or transfer control. The strongest evidence for an uncertainty application is therefore a demonstrated change in a consequential decision. Evaluation must then test whether that change improves the relevant trade-off among task success, information gain, risk, resource cost, delay, unnecessary intervention, and harm. It must also distinguish mature domain-level uncertainty estimation from agent-level uncertainty use. A calibrated predictor does not by itself establish that an autonomous system can recognize insufficient support and respond before the resulting commitment becomes consequential.

8 Open Problems

The open problems below do not ask which additional technique should be introduced. Instead, they question assumptions that have emerged as agent uncertainty research has become increasingly fine-grained, calibrated, and behavior-oriented. In each case, the literature has established that a more sophisticated uncertainty mechanism can be constructed, but has not yet established whether that added sophistication materially improves agent decisions or outcomes. We therefore formulate each problem as a direct question whose answer could confirm, limit, or redirect an existing line of research.

8.1 Quantification: Which Uncertainty Representations Are Actually Necessary?

Is Turn-Level Uncertainty Necessary for Multi-Turn Agents? Research on trajectory uncertainty has moved from assigning confidence only to the final answer, to estimating uncertainty at individual turns, and then to aggregating or propagating these local estimates across the full trajectory. This progression implicitly assumes that uncertainty attached to each thought, action, or observation provides information that cannot be recovered from the trajectory as a whole. However, a holistic estimator conditioned on the complete interaction history may already predict eventual success without assigning a separate uncertainty value to every intermediate turn. Turn-level estimation also introduces additional computation, noisy local signals, and difficult choices about how those signals should be combined or propagated. At the same time, a single trajectory-level score may reveal that the overall process is risky without showing when the risk emerged or whether the agent still had an opportunity to respond. The fundamental question therefore comes before the design of better propagation methods: *does turn-level uncertainty provide a practical capability that trajectory-level uncertainty cannot provide, or does it mainly add complexity to a problem that can be solved holistically?*

Does Finer-Grained Uncertainty Actually Improve Agent Decisions? Uncertainty quantification has progressively decomposed a generic confidence score into uncertainty over user specifications, tool choices, function arguments, assumptions, actions, and plans. This decomposition is intuitively attractive because an agent does not merely generate text; it commits to structured decisions whose individual components may fail for different reasons. Nevertheless, identifying which field is uncertain does not automatically tell the agent whether the complete commitment is reliable. The risk of a tool invocation may depend jointly on the recipient, account, amount, permission, and operation type, while the consequence of the action may matter more than the uncertainty of any one field. Increasing granularity may therefore produce a more detailed description of fragility without changing the eventual decision to execute, verify, or abstain. It may also introduce additional estimation and calibration errors at every decomposed level. *How much structure must uncertainty represent before it becomes genuinely actionable, and at what point does further decomposition become descriptive complexity without measurable reliability gains?*

When Does Multi-Agent Agreement Reduce Uncertainty Rather Than Amplify Shared Error? Multi-agent uncertainty research has developed from measuring disagreement among final answers to examining

reasoning alignment, communication dependence, interaction dynamics, and system-level confidence. This development exposes a basic assumption behind multi-agent systems: adding more agents is useful only when they contribute sufficiently independent evidence or reasoning. Agents may agree because they reached the same conclusion independently, but they may also agree because they share the same foundation model, retriever, prompt, memory, training data, or upstream tool error. Communication can further synchronize initially different agents, creating consensus by spreading one mistake through the group. Under these conditions, adding agents may reduce measured disagreement while leaving actual reliability unchanged or even making the shared error harder to challenge. Calibrating each agent separately does not resolve this problem if their failures remain correlated after composition. *Under what conditions does multi-agent agreement constitute genuinely new evidence, and when is apparent consensus only a repeated or communication-amplified version of the same uncertainty?*

8.2 Calibration: When Does Better Calibration Matter for Decisions?

Must Uncertainty Be Calibrated at Every Turn? Calibration research has moved from evaluating one terminal confidence score to assigning probability-like meaning to the entire confidence process produced throughout a trajectory. A complete calibrated trace appears stronger because it describes how confidence evolves as the agent reasons, acts, and observes new information. Yet it remains unclear whether every intermediate estimate needs to be calibrated with equal precision. Many turns are internal, reversible, or behaviorally inconsequential, and calibration errors at those turns may never affect an external decision. A process-level metric can therefore reward accurate probabilities at numerous unimportant steps while obscuring whether confidence was reliable at the few moments when the agent committed to an action, selected a tool, or decided not to seek help. Conversely, calibrating only the final score may detect miscalibration after the opportunity to change behavior has already passed. *Does calibrating the full uncertainty trace materially improve agent control, or is reliable calibration needed only at the consequential decision points where a different confidence value could still change the trajectory?*

Can Runtime Calibration Be Evaluated When Its Own Interventions Hide Errors? Calibration has increasingly shifted from one-time offline correction toward runtime adaptation because confidence semantics change across tasks, users, tools, models, and deployment environments. However, an uncertainty-aware agent does not passively observe deployment outcomes; its own confidence determines which outcomes are allowed to occur. Low-confidence cases may be interrupted, deferred to a stronger model, handed to a human, or abandoned before completion, while high-confidence cases remain under autonomous execution. The resulting feedback is therefore selective: the system observes outcomes for the cases it chose to handle, but often never observes what would have happened on the cases it removed. This creates the possibility of self-confirming calibration. A system may appear increasingly well calibrated simply because its most difficult cases are no longer included in the observed evaluation set, rather than because its probability estimates have become more reliable. *Can runtime calibration be meaningfully validated when uncertainty-based intervention changes which outcomes are observed and conceals precisely the failures needed to test whether the calibration remains correct?*

Do Better Calibration Metrics Actually Improve Agent Success? Agent uncertainty research has devoted substantial effort to reducing metrics such as ECE, Brier score, and related response- or trajectory-level calibration errors. These metrics are useful for evaluating whether reported probabilities correspond to empirical outcomes, but their improvement is often treated as evidence that the agent itself has become more reliable. That implication is not guaranteed. Agent control usually depends on whether confidence changes a concrete decision near a threshold: whether to continue, retrieve, verify, defer, or stop. A substantial improvement in average calibration error may leave all of these decisions unchanged, producing no improvement in task success, safety, or resource allocation. Conversely, a model with slightly worse global calibration may be more accurate around the particular boundary that determines intervention and may therefore achieve better outcomes. It is possible that calibration only needs to be sufficiently accurate for the relevant control decision, after which further optimization of numerical calibration metrics offers little practical value. *How much calibration is enough, and when do further gains in calibration metrics stop translating into meaningful gains in agent success or control?*

8.3 Utilization: Does Uncertainty Lead to the Right Behavior?

Does Clarification Resolve Uncertainty or Transfer It to the User? Clarification research has progressed from detecting underspecified requests to deciding when an agent should ask and which missing information a question should target. This development treats clarification as a deliberate uncertainty-reduction mechanism rather than a generic conversational behavior. However, asking a question does not guarantee that uncertainty is removed. The user may also be uncertain, may misunderstand the question, may provide an incomplete or inconsistent answer, or may revise a preference after the agent has already acted on earlier information. Additional questions can also increase interaction burden, enlarge the context, interrupt an otherwise recoverable workflow, and create new opportunities for misunderstanding. An agent may therefore become more active in asking questions without becoming more successful at completing the underlying task. Clarification can even make uncertainty less visible by replacing an explicit ambiguity with an answer that appears definite but is unreliable. *When does clarification genuinely reduce uncertainty relevant to the next decision, and when does it merely move that uncertainty from the agent to the user or hide it behind additional interaction?*

Does Retrieval Add Reliable Evidence or Merely Increase Confidence? Uncertainty-guided retrieval has developed from a binary decision about whether to search toward controlling search depth, retries, stopping, and evidence sufficiency. This progression assumes that additional retrieval can reduce uncertainty by providing information unavailable to the model. Yet retrieved information may be noisy, contradictory, redundant, outdated, or derived from the same underlying source. Multiple documents can therefore appear to provide independent support while merely repeating one unsupported claim. Language models may also become more confident because retrieved text is detailed or persuasive, even when it does not establish the conclusion required for the task. This creates a particularly dangerous interaction when confidence is also used as the stopping signal: unreliable evidence may increase confidence and cause the agent to terminate the search precisely when further verification is most needed. *How can we determine whether retrieval has actually reduced epistemic uncertainty by adding reliable evidence, rather than simply increasing the agent’s subjective confidence in a conclusion that remains unsupported?*

Can Uncertainty Alone Identify the Right Intervention? Uncertainty is now used to trigger additional reasoning, resampling, retrieval, clarification, verification, retry, deferral, and abstention. Existing studies show that uncertainty can improve each of these mechanisms when the available intervention is specified in advance. A clarification method assumes that uncertainty should lead to a question; a retrieval method assumes that it should lead to search; a deferral method assumes that it should lead to a stronger model. These results establish that uncertainty can determine *when* to activate a particular mechanism, but not that it can determine *which* mechanism is appropriate. The same uncertainty value may arise because information is missing, reasoning is unstable, a tool has failed, available evidence is contradictory, or the task lies outside the agent’s competence. These cases may require fundamentally different responses, and applying the wrong intervention can waste computation, introduce further noise, or reinforce the original error. *Does an uncertainty signal contain enough information to identify the appropriate response, or does current uncertainty-guided control confuse detecting fragility with diagnosing what should be done about it?*

Across these problems, the common unresolved assumption is that more detailed uncertainty representations, lower calibration error, and more uncertainty-driven behavior will naturally produce more reliable agents. The development of the field has shown that each of these improvements is technically possible, but it has not yet established where they provide indispensable decision value and where they merely produce richer descriptions, better-looking metrics, or more elaborate control mechanisms. The central research question is therefore not simply whether uncertainty can be estimated more precisely, but whether that precision changes agent behavior in a way that consistently improves the final outcome.

References

- [1] Jiaxin Zhang, Prafulla Kumar Choubey, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. Agentic uncertainty quantification. *arXiv preprint arXiv:2601.15703*, 2026.
- [2] Luoxi Tang, Yuqiao Meng, J. Costa, Yingxue Zhang, Muchao Ye, and Zhaohan Xi. The value of variance: Mitigating debate collapse in multi-agent systems via uncertainty-driven policy optimization. *ArXiv*, abs/2602.07186, 2026. URL <https://api.semanticscholar.org/CorpusID:285453817>.
- [3] Hang Luo, Bingbing Wen, and Lucy Lu Wang. Agentic abstention: Do agents know when to stop instead of act? 2026. URL <https://api.semanticscholar.org/CorpusID:289683726>.
- [4] Zihuiwen Ye, Lukas Aichberger, Michael Kirchhof, Sinead Williamson, Luca Zappella, Yarin Gal, Arno Blaas, and Adam Goliński. Uncertainty quantification for llm function-calling. *ArXiv*, abs/2604.22985, 2026. URL <https://api.semanticscholar.org/CorpusID:287776889>.
- [5] Lingfeng Zhang, Yongan Sun, Jinpeng Hu, Hui Ma, Ying Yang, Kuien Liu, Zenglin Shi, and Meng Wang. Webuncertainty: Dual-level uncertainty driven planning and reasoning for autonomous web agent. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 13072–13082, 2026.
- [6] Qingni Wang, Yue Fan, and Xin Eric Wang. Safeground: Know when to trust gui grounding models via uncertainty calibration. *ArXiv*, abs/2602.02419, 2026. URL <https://api.semanticscholar.org/CorpusID:285270772>.
- [7] Achref Doula, Max Mühlhäuser, and Alejandro Sánchez Guinea. Safepath: Conformal prediction for safe llm-based autonomous navigation. *ArXiv*, abs/2505.09427, 2025. URL <https://api.semanticscholar.org/CorpusID:278602858>.
- [8] Jinhao Duan, James Diffenderfer, Sandeep Madireddy, Tianlong Chen, Bhavya Kailkhura, and Kaidi Xu. Uprop: Investigating the uncertainty propagation of llms in multi-step agentic decision-making. *ArXiv*, abs/2506.17419, 2025. URL <https://api.semanticscholar.org/CorpusID:280000726>.
- [9] Yaopei Zeng, Congchao Wang, JianHang Chen, Nan Wang, Yurui Chang, and Lu Lin. Critic experience bank: Self-evolving step-level confidence estimation for llm agents. *arXiv preprint arXiv:2607.12397*, 2026.
- [10] Dzianis Piatrashyn, Nikita Kotelevskii, Kirill Grishchenkov, Nikita V. Glazkov, Ivan Nasonov, Ilya Makarov, Timothy Baldwin, Preslav Nakov, Roman Vashurin, and Maxim Panov. Redact: Uncertainty-aware deferral for llm agents. *ArXiv*, abs/2604.07036, 2026. URL <https://api.semanticscholar.org/CorpusID:287249104>.
- [11] Suresh Raghu, Satwik Pandey, and Shashwat Pandey. Proper scoring rules for agentic uncertainty quantification. 2026. URL <https://api.semanticscholar.org/CorpusID:288670982>.
- [12] Wenxuan Ding, Nicholas Tomlin, and Greg Durrett. Calibrate-then-act: Cost-aware exploration in llm agents. *ArXiv*, abs/2602.16699, 2026. URL <https://arxiv.org/abs/2602.16699>.
- [13] Haixin Wang, Hejie Cui, Chenwei Zhang, Xin Liu, Shuwei Jin, Shijie Geng, Xin-Yu Zhang, Nasser Zalmout, Zhenyu Shi, and Yizhou Sun. T²po: Uncertainty-guided exploration control for stable multi-turn agentic reinforcement learning. 2026. URL <https://api.semanticscholar.org/CorpusID:287950900>.
- [14] Joykirat Singh, Zaid Khan, Archiki Prasad, Justin Chih-Yao Chen, Akshay Nambi, Hyunji Lee, Elias Stengel-Eskin, and Mohit Bansal. Agent-brace: Decoupling beliefs from actions in long-horizon tasks via verbalized state uncertainty. 2026. URL <https://api.semanticscholar.org/CorpusID:288258161>.
- [15] Jiaxin Zhang, Caiming Xiong, and Chien-Sheng Wu. Agentic confidence calibration. *ArXiv*, abs/2601.15778, 2026. URL <https://api.semanticscholar.org/CorpusID:284934915>.
- [16] Michael Kirchhof, Gjergji Kasneci, and Enkelejd Kasneci. Position: Uncertainty quantification needs reassessment for large-language model agents. *ArXiv*, abs/2505.22655, 2025. URL <https://api.semanticscholar.org/CorpusID:278960260>.

- [17] Olaoluwa Shorinwa, Zhiting Mei, Justin Lidard, Allen Z. Ren, and Anirudha Majumdar. A survey on uncertainty quantification of large language models: Taxonomy, open research challenges, and future directions. *ACM Computing Surveys*, 58:1 – 38, 2024. URL <https://api.semanticscholar.org/CorpusID:274597654>.
- [18] Xiaou Liu, Tiejun Chen, Longchao Da, Chacha Chen, Zhen-Yu Lin, and Hua Wei. Uncertainty quantification and confidence calibration in large language models: A survey. *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, 2025. URL <https://api.semanticscholar.org/CorpusID:277150701>.
- [19] Zhiqiu Xia, Jinxuan Xu, Yuqian Zhang, and Hang Liu. A survey of uncertainty estimation methods on large language models. In *Annual Meeting of the Association for Computational Linguistics*, 2025. URL <https://api.semanticscholar.org/CorpusID:276742323>.
- [20] Tiejun Chen, Huaiyuan Yao, Jia Chen, Evangelos E. Papalexakis, and Hua Wei. Every response counts: Quantifying uncertainty of llm-based multi-agent systems through tensor decomposition. *ArXiv*, abs/2604.08708, 2026. URL <https://api.semanticscholar.org/CorpusID:287351911>.
- [21] Weihao Xuan, Qingcheng Zeng, Heli Qi, Yunze Xiao, Junjue Wang, and Naoto Yokoya. The confidence dichotomy: Analyzing and mitigating miscalibration in tool-use agents. *ArXiv*, abs/2601.07264, 2026. URL <https://api.semanticscholar.org/CorpusID:284648884>.
- [22] Manan Suri, Puneet Mathur, Nedim Lipka, Franck Dernoncourt, Ryan A. Rossi, and Dinesh Manocha. Structured uncertainty guided clarification for llm agents. *ArXiv*, abs/2511.08798, 2025. URL <https://api.semanticscholar.org/CorpusID:282939794>.
- [23] Xucong Wang, Ziyu Ma, Yong Wang, Yuxiang Ji, Shidong Yang, Guanhua Chen, Pengkun Wang, and Xiangxiang Chu. Appo: Agentic procedural policy optimization. 2026. URL <https://api.semanticscholar.org/CorpusID:289167216>.
- [24] Jiatan Huang, Mingchen Li, Ziming Li, Sunjae Kwon, Hong Yu, and Chuxu Zhang. Counterfactual graph for multi-agent llm calibration. *arXiv preprint arXiv:2605.30653*, 2026.
- [25] Neil Houlsby, Ferenc Huszár, Zoubin Ghahramani, and Máté Lengyel. Bayesian active learning for classification and preference learning. *arXiv preprint arXiv:1112.5745*, 2011.
- [26] Andrey Malinin and Mark Gales. Predictive uncertainty estimation via prior networks. *Advances in neural information processing systems*, 31, 2018.
- [27] Andrey Malinin and Mark Gales. Reverse kl-divergence training of prior networks: Improved uncertainty and adversarial robustness. *Advances in neural information processing systems*, 32, 2019.
- [28] Andrey Malinin and Mark Gales. Uncertainty estimation in autoregressive structured prediction. *arXiv preprint arXiv:2002.07650*, 2020.
- [29] Yijun Xiao and William Yang Wang. On hallucination and predictive uncertainty in conditional language generation. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 2734–2744, 2021.
- [30] Chen Ling, Xujiang Zhao, Xuchao Zhang, Wei Cheng, Yanchi Liu, Yiyun Sun, Mika Oishi, Takao Osaki, Katsushi Matsuda, Jie Ji, et al. Uncertainty quantification for in-context learning of large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3357–3370, 2024.
- [31] Yasin Abbasi Yadkori, Ilja Kuzborskij, András György, and Csaba Szepesvári. To believe or not to believe your llm. *arXiv preprint arXiv:2406.02543*, 2024.
- [32] Lukas Aichberger, Kajetan Schweighofer, Mykyta Ielanskyi, and Sepp Hochreiter. Semantically diverse language generation for uncertainty estimation in language models. *arXiv e-prints*, pages arXiv–2406, 2024.
- [33] Bairu Hou, Yujian Liu, Kaizhi Qian, Jacob Andreas, Shiyu Chang, and Yang Zhang. Decomposing uncertainty for large language models through input clarification ensembling. *arXiv preprint arXiv:2311.08718*, 2023.

- [34] Tunyu Zhang, Haizhou Shi, Yibin Wang, Hengyi Wang, Xiaoxiao He, Zhuowei Li, Haoxian Chen, Ligong Han, Kai Xu, Huan Zhang, et al. Tokur: Token-level uncertainty estimation for large language model reasoning. *arXiv preprint arXiv:2505.11737*, 2025.
- [35] Mingda Li, Xinyu Li, Weinan Zhang, and Longxuan Ma. Esi: Epistemic uncertainty quantification via semantic-preserving intervention for large language models. *arXiv preprint arXiv:2510.13103*, 2025.
- [36] Seongmin Lee, Hsiang Hsu, Chun-Fu Chen, and Duen Horng Chau. Probing llm hallucination from within: Perturbation-driven approach via internal knowledge. *arXiv preprint arXiv:2411.09689*, 2024.
- [37] Jianyi Zhang, Da-Cheng Juan, Cyrus Rashtchian, Chun-Sung Ferng, Heinrich Jiang, and Yiran Chen. Sled: Self logits evolution decoding for improving factuality in large language models. *Advances in Neural Information Processing Systems*, 37:5188–5209, 2024.
- [38] Ann-Kathrin Dombrowski and Guillaume Corlouer. An information-theoretic study of lying in llms. In *ICML 2024 Workshop on LLMs and Cognition*, 2024.
- [39] Zhewei Kang, Xuandong Zhao, and Dawn Song. Scalable best-of-n selection for large language models via self-certainty. *arXiv preprint arXiv:2502.18581*, 2025.
- [40] Xuandong Zhao, Zhewei Kang, Aosong Feng, Sergey Levine, and Dawn Song. Learning to reason without external rewards. *arXiv preprint arXiv:2505.19590*, 2025.
- [41] Qingcheng Zeng, Mingyu Jin, Qinkai Yu, Zhenting Wang, Wenyue Hua, Zihao Zhou, Guangyan Sun, Yanda Meng, Shiqing Ma, Qifan Wang, et al. Uncertainty is fragile: Manipulating uncertainty in large language models. *arXiv preprint arXiv:2407.11282*, 2024.
- [42] Chung-Ching Chang, David Reitter, Renat Aksitov, and Yun-Hsuan Sung. K1-divergence guided temperature sampling. *arXiv preprint arXiv:2306.01286*, 2023.
- [43] Xiaomeng Hu, Yiming Zhang, Ru Peng, Haozhe Zhang, Chenwei Wu, Gang Chen, and Junbo Zhao. Embedding and gradient say wrong: A white-box method for hallucination detection. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1950–1959, 2024.
- [44] Manish Bhatt. Predictive coding and information bottleneck for hallucination detection in large language models. *arXiv preprint arXiv:2601.15652*, 2026.
- [45] Can Xie, Ruotong Pan, Xiangyu Wu, Yunfei Zhang, Jiayi Fu, Tingting Gao, and Guorui Zhou. Unlocking exploration in rlvr: Uncertainty-aware advantage shaping for deeper reasoning. *arXiv preprint arXiv:2510.10649*, 2025.
- [46] Maxime Darrin, Pablo Piantanida, and Pierre Colombo. Rainproof: An umbrella to shield text generator from out-of-distribution data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5831–5857, 2023.
- [47] Maya Kruse, Majid Afshar, Saksham Khatwani, Anoop Mayampurath, Guanhua Chen, and Yanjun Gao. Simple yet effective: An information-theoretic approach to multi-llm uncertainty quantification. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 30481–30492, 2025.
- [48] Seonghyeon Park, Jewon Yeom, Jaewon Sok, Jeongjae Park, Heejun Kim, and Taesup Kim. Efficient epistemic uncertainty estimation for large language models via knowledge distillation. *arXiv preprint arXiv:2602.01956*, 2026.
- [49] Artem Vazhentsev, Lyudmila Rvanova, Ivan Lazichny, Alexander Panchenko, Maxim Panov, Timothy Baldwin, and Artem Shelmanov. Token-level density-based uncertainty quantification methods for eliciting truthfulness of large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 2246–2262, 2025.
- [50] Jie Ren, Jiaming Luo, Yao Zhao, Kundan Krishna, Mohammad Saleh, Balaji Lakshminarayanan, and Peter J Liu. Out-of-distribution detection and selective generation for conditional language models. *arXiv preprint arXiv:2209.15558*, 2022.

- [51] Xiaomin Li, Zhou Yu, Ziji Zhang, Yingying Zhuang, Swair Shah, Narayanan Sadagopan, and Anurag Beniwal. Semantic volume: Quantifying and detecting both external and internal uncertainty in llms. *arXiv preprint arXiv:2502.21239*, 2025.
- [52] Xin Qiu and Risto Miikkulainen. Semantic density: Uncertainty quantification for large language models through confidence measurement in semantic space. *Advances in neural information processing systems*, 37:134507–134533, 2024.
- [53] KiYoon Yoo, Jangho Kim, Jiho Jang, and Nojun Kwak. Detection of adversarial examples in text classification: Benchmark and baseline via robust density estimation. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 3656–3672, 2022.
- [54] Junya Takayama and Yuki Arase. Relevant and informative response generation using pointwise mutual information. In *Proceedings of the First Workshop on NLP for Conversational AI*, pages 133–138, 2019.
- [55] Liam Van der Poel, Ryan Cotterell, and Clara Meister. Mutual information alleviates hallucinations in abstractive summarization. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 5956–5965, 2022.
- [56] Zehui Liao, Shishuai Hu, Ke Zou, Huazhu Fu, Liangli Zhen, and Yong Xia. Vision-amplified semantic entropy for hallucination detection in medical visual question answering. In *Medical Image Computing and Computer Assisted Intervention (MICCAI)*, 2025. arXiv:2503.20504.
- [57] Andrey Malinin and Mark Gales. Uncertainty estimation in autoregressive structured prediction. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=jN5y-zb5Q7m>.
- [58] Ekaterina Fadeeva, Roman Vashurin, Akim Tsvigun, Artem Vazhentsev, Sergey Petrakov, Kirill Fedyanin, Daniil Vasilev, Elizaveta Goncharova, Alexander Panchenko, Maxim Panov, et al. Lm-polygraph: Uncertainty estimation for language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 446–461, 2023.
- [59] Jinhao Duan, Hao Cheng, Shiqi Wang, Alex Zavalny, Chenan Wang, Renjing Xu, Bhavya Kailkhura, and Kaidi Xu. Shifting attention to relevance: Towards the predictive uncertainty quantification of free-form large language models, 2024. URL <https://arxiv.org/abs/2307.01379>.
- [60] Beining Xu and Yongming Lu. Tecp: Token-entropy conformal prediction for llms, 2025. URL <https://arxiv.org/abs/2509.00461>.
- [61] Andrey Malinin and Mark Gales. Uncertainty estimation in autoregressive structured prediction, 2021. URL <https://arxiv.org/abs/2002.07650>.
- [62] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can llms express their uncertainty? an empirical evaluation of confidence elicitation in llms. *arXiv preprint arXiv:2306.13063*, 2023.
- [63] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.
- [64] Roi Cohen, Konstantin Dobler, Eden Biran, and Gerard de Melo. I don’t know: Explicit modeling of uncertainty with an [idk] token. *Advances in Neural Information Processing Systems*, 37:10935–10958, 2024.
- [65] Xuezhong Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- [66] Potsawee Manakul, Adian Liusie, and Mark Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 9004–9017, 2023.
- [67] Christian Hobelsberger, Theresa Winner, Andreas Nawroth, Oliver Mitevski, and Anna-Carolina Haensch. Systematic evaluation of uncertainty estimation methods in large language models. *arXiv preprint arXiv:2510.20460*, 2025.

- [68] Henning Bartsch, Ole Jorgensen, Domenic Rosati, Jason Hoelscher-Obermaier, and Jacob Pfau. Self-consistency of large language models under ambiguity. In *Proceedings of the 6th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 89–105, 2023.
- [69] Yuanzhao Zhai, Han Zhang, Yu Lei, Yue Yu, Kele Xu, Dawei Feng, Bo Ding, and Huaimin Wang. Uncertainty-penalized reinforcement learning from human feedback with diverse reward lora ensembles. *arXiv preprint arXiv:2401.00243*, 2023.
- [70] Debangshu Banerjee and Aditya Gopalan. Towards reliable alignment: Uncertainty-aware rlhf. *arXiv preprint arXiv:2410.23726*, 2024.
- [71] Xingzhou Lou, Dong Yan, Wei Shen, Yuzi Yan, Jian Xie, and Junge Zhang. Uncertainty-aware reward model: Teaching reward models to know what is unknown. *arXiv preprint arXiv:2410.00847*, 2024.
- [72] Jacob Eisenstein, Chirag Nagpal, Alekh Agarwal, Ahmad Beirami, Alex D’Amour, DJ Dvijotham, Adam Fisch, Katherine Heller, Stephen Pfohl, Deepak Ramachandran, et al. Helping or herding? reward model ensembles mitigate but do not eliminate reward hacking. *arXiv preprint arXiv:2312.09244*, 2023.
- [73] Debarun Bhattacharjya, Balaji Ganesan, Junkyu Lee, Radu Marinescu, Katsiaryna Mirylenka, Michael Glass, and Xiao Shou. Simba uq: Similarity-based aggregation for uncertainty quantification in large language models. *arXiv preprint arXiv:2510.13836*, 2025.
- [74] Alexander Nikitin, Jannik Kossen, Yarin Gal, and Pekka Marttinen. Kernel language entropy: Fine-grained uncertainty quantification for llms from semantic similarities, 2024. URL <https://arxiv.org/abs/2405.20003>.
- [75] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation, 2023. URL <https://arxiv.org/abs/2302.09664>.
- [76] Jannik Kossen, Jiatong Han, Muhammed Razzak, Lisa Schut, Shreshth Malik, and Yarin Gal. Semantic entropy probes: Robust and cheap hallucination detection in llms, 2024. URL <https://arxiv.org/abs/2406.15927>.
- [77] Zhen Lin, Shubhendu Trivedi, and Jimeng Sun. Generating with confidence: Uncertainty quantification for black-box large language models. *arXiv preprint arXiv:2305.19187*, 2023.
- [78] Dang Nguyen, Ali Payani, and Baharan Mirzasoleiman. Beyond semantic entropy: Boosting llm uncertainty quantification with pairwise semantic similarity, 2025. URL <https://arxiv.org/abs/2506.00245>.
- [79] Edward Phillips, Sean Wu, Soheila Molaei, Danielle Belgrave, Anshul Thakur, and David Clifton. Geometric uncertainty for detecting and correcting hallucinations in llms, 2025. URL <https://arxiv.org/abs/2509.13813>.
- [80] Litian Liu, Reza Pourreza, Yubing Jian, Yao Qin, and Roland Memisevic. From out-of-distribution detection to hallucination detection: A geometric view, 2026. URL <https://arxiv.org/abs/2602.07253>.
- [81] Qiwei Zhao, Dong Li, Yanchi Liu, Wei Cheng, Yiyu Sun, Mika Oishi, Takao Osaki, Katsushi Matsuda, Huaxiu Yao, Chen Zhao, Haifeng Chen, and Xujiang Zhao. Uncertainty propagation on LLM agent. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6064–6073, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.302. URL <https://aclanthology.org/2025.acl-long.302/>.
- [82] Dengjia Zhang, Xiaou Liu, Lu Cheng, Yaqing Wang, Kenton Murray, and Hua Wei. Selaurl: Self evolving llm agent via uncertainty-aware rewards. *ArXiv*, abs/2602.21158, 2026. URL <https://api.semanticscholar.org/CorpusID:286000783>.
- [83] Sina Tayebati, Divake Kumar, Nastaran Darabi, Davide Etti, Ranganath Krishnan, and Amit Rangan Trivedi. Tracer: Trajectory risk aggregation for critical episodes in agentic reasoning. *ArXiv*, abs/2602.11409, 2026. URL <https://api.semanticscholar.org/CorpusID:285540323>.

- [84] Jiaxin Zhang, Prafulla Kumar Choubey, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. Agentic uncertainty quantification. *ArXiv*, abs/2601.15703, 2026. URL <https://api.semanticscholar.org/CorpusID:284935450>.
- [85] Jiuzhou Han, Wray Buntine, and Ehsan Shareghi. Towards uncertainty-aware language agent. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 6662–6685, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.398. URL <https://aclanthology.org/2024.findings-acl.398/>.
- [86] Ziwei Deng, Mian Deng, Chenjing Liang, Zeming Gao, Chennan Ma, Chenxing Lin, Haipeng Zhang, Songzhu Mei, Siqi Shen, and Cheng Wang. Planu: Large language model reasoning through planning under uncertainty, 2025. URL <https://arxiv.org/abs/2510.18442>.
- [87] Aman Mehta. When agents disagree with themselves: Behavioral consistency as an uncertainty signal for llm agents. 2026. URL <https://api.semanticscholar.org/CorpusID:290179434>.
- [88] Aman Mehta. When agents commit too soon: Diagnosing premature commitment in llm agents. 2026. URL <https://api.semanticscholar.org/CorpusID:289625632>.
- [89] Yuxuan Zhao, Sijia Chen, and Ningxin Su. When does multi-agent collaboration help? an entropy perspective. 2026. URL <https://api.semanticscholar.org/CorpusID:285285732>.
- [90] Kangkang Sun, Jun Wu, Jianhua Li, Minyi Guo, Xiuzhen Che, and Jianwei Huang. Coe: Collaborative entropy for uncertainty quantification in agentic multi-llm systems. *ArXiv*, abs/2603.28360, 2026. URL <https://api.semanticscholar.org/CorpusID:286975144>.
- [91] Yu Feng, Phu Mon Htut, Zheng Qi, Wei Xiao, Manuel Mager, Nikolaos Pappas, Kishaloy Halder, Yang Li, Yassine Benajiba, and Dan Roth. Rethinking llm uncertainty: A multi-agent approach to estimating black-box model uncertainty. In *Conference on Empirical Methods in Natural Language Processing*, 2024. URL <https://api.semanticscholar.org/CorpusID:274655719>.
- [92] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher D. Manning. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. *ArXiv*, abs/2305.14975, 2023. URL <https://arxiv.org/abs/2305.14975>.
- [93] Gabrielle Kaili-May Liu, Gal Yona, Avi Caciularu, Idan Szpektor, Tim G. J. Rudner, and Arman Cohan. Metafaith: Faithful natural language uncertainty expression in llms. *ArXiv*, abs/2505.24858, 2025. URL <https://arxiv.org/abs/2505.24858>.
- [94] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can llms express their uncertainty? an empirical evaluation of confidence elicitation in llms. *ArXiv*, abs/2306.13063, 2023. URL <https://api.semanticscholar.org/CorpusID:259224389>.
- [95] Gal Yona, Roei Aharoni, and Mor Geva. Can large language models faithfully express their intrinsic uncertainty in words? *ArXiv*, abs/2405.16908, 2024. URL <https://arxiv.org/abs/2405.16908>.
- [96] Jean Kaddour, Sr Patel, Gbètondji J.-S. Dovonon, Leo Richter, Pasquale Minervini, and Matt J. Kusner. Agentic uncertainty reveals agentic overconfidence. *ArXiv*, abs/2602.06948, 2026. URL <https://api.semanticscholar.org/CorpusID:285401837>.
- [97] Caiqi Zhang, Ruihan Yang, Zhisong Zhang, Xinting Huang, Sen Yang, Dong Yu, and Nigel Collier. Atomic calibration of llms in long-form generations. In *IJCNLP-AAACL*, 2024. URL <https://api.semanticscholar.org/CorpusID:273403321>.
- [98] Maohao Shen, Subhro Das, Kristjan Greenewald, Prasanna Sattigeri, Gregory Wornell, and Soumya Ghosh. Thermometer: Towards universal calibration for large language models. *ArXiv*, abs/2403.08819, 2024. URL <https://arxiv.org/abs/2403.08819>.
- [99] Dennis Ulmer, Martin Gubri, Hwaran Lee, Sangdoo Yun, and Seong Joon Oh. Calibrating large language models using their generations only. *ArXiv*, abs/2403.05973, 2024. URL <https://arxiv.org/abs/2403.05973>.

- [100] Beier Luo, Shuoyuan Wang, Yixuan Li, and Hongxin Wei. Your pre-trained llm is secretly an unsupervised confidence calibrator. *ArXiv*, abs/2505.16690, 2025. URL <https://arxiv.org/abs/2505.16690>.
- [101] Hexiang Tan, Wanli Yang, Junwei Zhang, Xin Chen, Rui Tang, Du Su, Jingang Wang, Yuanzhuo Wang, Fei Sun, and Xueqi Cheng. Basecal: Unsupervised confidence calibration via base model signals. *ArXiv*, abs/2601.03042, 2026. URL <https://arxiv.org/abs/2601.03042>.
- [102] Caiqi Zhang, Ruihan Yang, Xiaochen Zhu, Chengzu Li, Tiancheng Hu, Yijiang River Dong, Deqing Yang, and Nigel Collier. Confidence estimation for llms in multi-turn interactions. *ArXiv*, abs/2601.02179, 2026. URL <https://arxiv.org/abs/2601.02179>.
- [103] Zhaohan Zhang, Chengzhengxu Li, Xiaoming Liu, Chao Shen, Ziquan Liu, and Ioannis Patras. Confidence should be calibrated more than one turn deep. *ArXiv*, abs/2604.05397, 2026. URL <https://arxiv.org/abs/2604.05397>.
- [104] Joss Armstrong. Margin: Runtime confidence calibration for multi-agent foundation model coordination. 2026. URL <https://api.semanticscholar.org/CorpusID:288650943>.
- [105] Elias Stengel-Eskin, Peter Hase, and Mohit Bansal. Lacie: Listener-aware finetuning for confidence calibration in large language models. *ArXiv*, abs/2405.21028, 2024. URL <https://arxiv.org/abs/2405.21028>.
- [106] Yibo Li, Miao Xiong, Jiaying Wu, and Bryan Hooi. Conftuner: Training large language models to express their confidence verbally. *ArXiv*, abs/2508.18847, 2025. URL <https://arxiv.org/abs/2508.18847>.
- [107] Hanning Zhang, Shizhe Diao, Yong Lin, Yi R. Fung, Qing Lian, Xingyao Wang, Yangyi Chen, Heng Ji, and Tong Zhang. R-tuning: Instructing large language models to say ‘i don’t know’. *ArXiv*, abs/2311.09677, 2023. URL <https://arxiv.org/abs/2311.09677>.
- [108] Sanyam Kapoor, Nate Gruver, Manley Roberts, Katherine Collins, Arka Pal, Umang Bhatt, Adrian Weller, Samuel Dooley, Micah Goldblum, and Andrew Gordon Wilson. Large language models must be taught to know what they don’t know. *ArXiv*, abs/2406.08391, 2024. URL <https://arxiv.org/abs/2406.08391>.
- [109] Neil Band, Xuechen Li, Tengyu Ma, and Tatsunori Hashimoto. Linguistic calibration of long-form generations. *ArXiv*, abs/2404.00474, 2024. URL <https://arxiv.org/abs/2404.00474>.
- [110] Tianyang Xu, Shujin Wu, Shizhe Diao, Xiaoze Liu, Xingyao Wang, Yangyi Chen, and Jing Gao. Sayself: Teaching llms to express confidence with self-reflective rationales. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 5985–5998, 2024.
- [111] Ruihan Yang, Caiqi Zhang, Zhisong Zhang, Xinting Huang, Sen Yang, Nigel Collier, Dong Yu, and Deqing Yang. Logu: Long-form generation with uncertainty expressions. *ArXiv*, abs/2410.14309, 2024. URL <https://api.semanticscholar.org/CorpusID:273482351>.
- [112] Caiqi Zhang, Xiaochen Zhu, Chengzu Li, Nigel Collier, and Andreas Vlachos. Reinforcement learning for better verbalized confidence in long-form generation. *ArXiv*, abs/2505.23912, 2025. URL <https://arxiv.org/abs/2505.23912>.
- [113] Xiaochen Zhu, Caiqi Zhang, Yizhou Chi, Tom Stafford, Nigel Collier, and Andreas Vlachos. Demystifying multi-agent debate: The role of confidence and diversity. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 33909–33930, 2026.
- [114] Paul Stangel, David Bani-Harouni, Chantal Pellegrini, Ege Özsoy, Kamilia Zaripova, Matthias Keicher, and Nassir Navab. Rewarding doubt: A reinforcement learning approach to calibrated confidence expression of large language models. *ArXiv*, abs/2503.02623, 2025. URL <https://arxiv.org/abs/2503.02623>.
- [115] Shuchang Tao, Liuyi Yao, Hanxing Ding, Yuexiang Xie, Qi Cao, Fei Sun, Jinyang Gao, Huawei Shen, and Bolin Ding. When to trust llms: Aligning confidence with response quality. *ArXiv*, abs/2404.17287, 2024. URL <https://arxiv.org/abs/2404.17287>.

- [116] Chen Li, Xiaolin Hu, Songzhu Zheng, Jiawei Zhou, and Chaoran Chen. Orce: Order-aware alignment of verbalized confidence in large language models. 2026. URL <https://api.semanticscholar.org/CorpusID:288259355>.
- [117] Liaoyaqi Wang, Chunsheng Zuo, William Jurayj, Benjamin Van Durme, and Anqi Liu. Process supervision of confidence margin for calibrated llm reasoning. *ArXiv*, abs/2604.23333, 2026. URL <https://arxiv.org/abs/2604.23333>.
- [118] Mengyi Deng, Zhiwei Li, Xin Li, Tingyu Zhu, Ying Zhao, Zhijiang Guo, and Wei Wang. Uncertainty-aware clarification in llm agents with information gain. 2026. URL <https://api.semanticscholar.org/CorpusID:288863573>.
- [119] Yijin Zhou, Linqian Zeng, Xiaoya Lu, Wenyuan Xie, Dongrui Liu, Junchi Yan, and Jing Shao. Exploring agentic tool-calling decisions via uncertainty-aligned reinforcement learning. 2026. URL <https://api.semanticscholar.org/CorpusID:288984888>.
- [120] Jixuan Leng, Chengsong Huang, Banghua Zhu, and Jiaxin Huang. Taming overconfidence in llms: Reward calibration in rlhf. *ArXiv*, abs/2410.09724, 2024. URL <https://arxiv.org/abs/2410.09724>.
- [121] Victor Quach, Adam Fisch, Tal Schuster, Adam Yala, Jae Ho Sohn, Tommi S. Jaakkola, and Regina Barzilay. Conformal language modeling. *ArXiv*, abs/2306.10193, 2023. URL <https://arxiv.org/abs/2306.10193>.
- [122] Christopher Mohri and Tatsunori Hashimoto. Language models with conformal factuality guarantees. *ArXiv*, abs/2402.10978, 2024. URL <https://arxiv.org/abs/2402.10978>.
- [123] Zhengping Jiang, Anqi Liu, and Benjamin Van Durme. Conformal linguistic calibration: Trading-off between factuality and specificity. *ArXiv*, abs/2502.19110, 2025. URL <https://arxiv.org/abs/2502.19110>.
- [124] Wenxuan Wang, Juluan Shi, Chaozheng Wang, Cheryl Lee, Youliang Yuan, Jen-Tse Huang, and Michael R. Lyu. Learning to ask: When llm agents meet unclear instruction. In *Conference on Empirical Methods in Natural Language Processing*, 2024. URL <https://api.semanticscholar.org/CorpusID:272368496>.
- [125] Nicholas Edwards and Sebastian Schuster. Ask or assume? uncertainty-aware clarification-seeking in coding agents. *ArXiv*, abs/2603.26233, 2026. URL <https://api.semanticscholar.org/CorpusID:286942481>.
- [126] Gregory Matsnev. Uncertainty decomposition for clarification seeking in llm agents. 2026. URL <https://api.semanticscholar.org/CorpusID:289627068>.
- [127] Chao Hao, Shuai Wang, and Kaiwen Zhou. Uncertainty-aware gui agent: Adaptive perception through component recommendation and human-in-the-loop refinement. *ArXiv*, abs/2508.04025, 2025. URL <https://api.semanticscholar.org/CorpusID:280536944>.
- [128] Hung-Ting Su, Ching yuan Chen, Po-Chen Ko, Jia-Fong Yeh, Min Sun, and Winston H. Hsu. Context-aware replanning with pre-explored semantic map for object navigation. In *Conference on Robot Learning*, 2024. URL <https://api.semanticscholar.org/CorpusID:272525114>.
- [129] Weihang Su, Yichen Tang, Qingyao Ai, Zhijing Wu, and Yiqun Liu. Dragin: Dynamic retrieval augmented generation based on the real-time information needs of large language models. In *Annual Meeting of the Association for Computational Linguistics*, 2024. URL <https://api.semanticscholar.org/CorpusID:268509926>.
- [130] Zefang Zong, Dingwei Chen, Yang Li, Qi Yi, Bo Zhou, Chengming Li, Bo Qian, Peng Chen, and Jie Jiang. At2po: Agentic turn-based policy optimization via tree search. *ArXiv*, abs/2601.04767, 2026. URL <https://api.semanticscholar.org/CorpusID:284544537>.
- [131] Jiuzhou Han, Wray L. Buntine, and Ehsan Shareghi. Towards uncertainty-aware language agent. *ArXiv*, abs/2401.14016, 2024. URL <https://api.semanticscholar.org/CorpusID:267211955>.
- [132] Seung hun Seo, Soobin Lim, SeongRae Noh, Haneul Kim, and HyeongYeop Kang. From assumptions to actions: Turning llm reasoning into uncertainty-aware planning for embodied agents. *ArXiv*, abs/2602.04326, 2026. URL <https://api.semanticscholar.org/CorpusID:285285787>.

- [133] Nicholas Lee, Lutfi Eren Erdogan, Chris Joseph John, Surya Krishnapillai, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. Agentic test-time scaling for webagents. *ArXiv*, abs/2602.12276, 2026. URL <https://api.semanticscholar.org/CorpusID:285540495>.
- [134] Chenhui Mao, Yu Lei, Zhixiang Wei, Ming Liang, Zhixiang Wang, Jing Xu, Dajun Chen, Wei Jiang, and Yong Li. Egss: Entropy-guided stepwise scaling for reliable software engineering. *ArXiv*, abs/2602.05242, 2026. URL <https://api.semanticscholar.org/CorpusID:285304036>.
- [135] Xiaochen Zhu, Caiqi Zhang, Yizhou Chi, Tom Stafford, Nigel Collier, and Andreas Vlachos. Demystifying multi-agent debate: The role of confidence and diversity. *ArXiv*, abs/2601.19921, 2026. URL <https://api.semanticscholar.org/CorpusID:285102089>.
- [136] Luke Yoffe, Alfonso Amayuelas, and William Yang Wang. Debunc: Mitigating hallucinations in large language model agent communication with uncertainty estimations. *ArXiv*, abs/2407.06426, 2024. URL <https://api.semanticscholar.org/CorpusID:280677460>.
- [137] Guanting Dong, Hangyu Mao, Kai Ma, Licheng Bao, Yifei Chen, Zhongyuan Wang, Zhongxia Chen, Jiazhen Du, Huiyang Wang, Fuzheng Zhang, Guorui Zhou, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Agentic reinforced policy optimization. *ArXiv*, abs/2507.19849, 2025. URL <https://api.semanticscholar.org/CorpusID:280323841>.
- [138] Guanting Dong, Licheng Bao, Zhongyuan Wang, Kangzhi Zhao, Xiaoxi Li, Jiajie Jin, Jinghan Yang, Hangyu Mao, Fuzheng Zhang, Kun Gai, Guorui Zhou, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Agentic entropy-balanced policy optimization. *ArXiv*, abs/2510.14545, 2025. URL <https://api.semanticscholar.org/CorpusID:282138579>.
- [139] Minki Kang, Shizhe Diao, Ryo Hachiuma, Sung Ju Hwang, Pavlo Molchanov, Yu-Chiang Frank Wang, and Byung-Kwan Lee. Agent explorative policy optimization for multimodal agentic reasoning. 2026. URL <https://api.semanticscholar.org/CorpusID:288702015>.
- [140] Peilin Wu, Mian Zhang, Xinlu Zhang, Xinya Du, and Zhiyu Zoey Chen. Search wisely: Mitigating sub-optimal agentic searches by reducing uncertainty. *ArXiv*, abs/2505.17281, 2025. URL <https://api.semanticscholar.org/CorpusID:278886677>.
- [141] Panagiotis Lymperopoulos and Vasanth Sarathy. Tools in the loop: Quantifying uncertainty of llm question answering systems that use tools. In *Adaptive Agents and Multi-Agent Systems*, 2025. URL <https://api.semanticscholar.org/CorpusID:278788702>.
- [142] Yingfang Yuan, Xuhong Wang, and Shanzhe Lei. Towards trustworthy report generation: A deep research agent with progressive confidence estimation and calibration. *ArXiv*, abs/2604.05952, 2026. URL <https://api.semanticscholar.org/CorpusID:287209448>.
- [143] Ruixin Yang, Dheeraj Rajagopal, Shirley Anugrah Hayati, Bin Hu, and Dongyeop Kang. Confidence calibration and rationalization for llms via multi-agent deliberation. *ArXiv*, abs/2404.09127, 2024. URL <https://api.semanticscholar.org/CorpusID:269149545>.
- [144] Vamshi Krishna Bonagiri, Ponnurangam Kumaraguru, Khanh Xuan Nguyen, and Benjamin Plaut. Check yourself before you wreck yourself: Selectively quitting improves llm agent safety. *ArXiv*, abs/2510.16492, 2025. URL <https://api.semanticscholar.org/CorpusID:282210328>.
- [145] Jiawei Wang, Jiakai Liu, Yu Fu, Yingru Li, Xintao Wang, Yuan Lin, Yu Yue, Lin Zhang, Yang Wang, and Ke Wang. Harnessing uncertainty: Entropy-modulated policy gradients for long-horizon llm agents. *ArXiv*, abs/2509.09265, 2025. URL <https://api.semanticscholar.org/CorpusID:281252591>.
- [146] Zeping Li, Hongru Wang, Yiwen Zhao, Guanhua Chen, Yixia Li, Keyang Chen, Yixin Cao, Guangnan Ye, Hongfeng Chai, and Zhenfei Yin. Rethinking the role of entropy in optimizing tool-use behaviors for large language model agents. *ArXiv*, abs/2602.02050, 2026. URL <https://api.semanticscholar.org/CorpusID:285271471>.
- [147] Zihan Lin, Xiaohan Wang, Jie Cao, Jiajun Chai, Guojun Yin, Wei Lin, and Ran He. Rest: Reshaping token-level policy gradients for tool-use large language models. *ArXiv*, abs/2509.21826, 2025. URL <https://api.semanticscholar.org/CorpusID:281658680>.

- [148] Jiahao Yu, Zelei Cheng, Xian Wu, and Xinyu Xing. Building coding agents via entropy-enhanced multi-turn preference optimization. *ArXiv*, abs/2509.12434, 2025. URL <https://api.semanticscholar.org/CorpusID:281325091>.
- [149] Wujiang Xu, Wentian Zhao, Zhenting Wang, Yu-Jhe Li, Can Jin, Mingyu Jin, Kai Mei, Kun Wang, and Dimitris N. Metaxas. Epo: Entropy-regularized policy optimization for llm agents reinforcement learning. *ArXiv*, abs/2509.22576, 2025. URL <https://api.semanticscholar.org/CorpusID:281658637>.
- [150] Yulei Qin, Xiaoyu Tan, Zhengbao He, Gang Li, Haojia Lin, Zongyi Li, Zihan Xu, Yuchen Shi, Siqu Cai, Renting Rui, Shaofei Cai, Yuzheng Cai, Xuan Zhang, Shengyong Ye, Ke Li, and Xing Sun. Learn the ropes, then trust the wins: Self-imitation with progressive exploration for agentic reinforcement learning. *ArXiv*, abs/2509.22601, 2025. URL <https://api.semanticscholar.org/CorpusID:281659069>.
- [151] Siwei Zhang, Yun Xiong, Xi Chen, Zian Jia, Renhong Huang, Jiarong Xu, and Jiawei Zhang. Rapo: Expanding exploration for llm agents via retrieval-augmented policy optimization. *ArXiv*, abs/2603.03078, 2026. URL <https://api.semanticscholar.org/CorpusID:286229081>.
- [152] Minghan Chen et al. Seed-grpo: Semantic entropy enhanced grpo for uncertainty-aware policy optimization. *arXiv preprint arXiv:2505.12346*, 2025.
- [153] Senkang Hu, Yong Dai, Yuzhi Zhao, Yihang Tao, Yu Guo, Zhengru Fang, Sam Tak Wu Kwong, and Yuguang Fang. Optimizing agentic reasoning with retrieval via synthetic semantic information gain reward. *ArXiv*, abs/2602.00845, 2026. URL <https://api.semanticscholar.org/CorpusID:285270202>.
- [154] Hao-Dong Zhao, Songlin Zhou, Yuxin Zhang, Stephen S.-T. Yau, Wenyu Zhang, Lun Tian, Tianshun Zhu, Yifeng Huang, Yuchen Zeng, Jing Gu, Daxiang Dong, and Jianmin Wu. Aem: Adaptive entropy modulation for multi-turn agentic reinforcement learning. 2026. URL <https://api.semanticscholar.org/CorpusID:287929623>.
- [155] Ayush Pandey, Jai Bardhan, Ishita Jain, Ramya Hebbalaguppe, Rohan Raju Dhanakshirur, and Lovekesh Vig. Refine and align: Confidence calibration through multi-agent interaction in vqa. *ArXiv*, abs/2511.11169, 2025. URL <https://api.semanticscholar.org/CorpusID:283054903>.
- [156] Kongcheng Zhang et al. Consistent paths lead to truth: Self-rewarding reinforcement learning for llm reasoning. *arXiv preprint arXiv:2506.08745*, 2025.
- [157] Joykirat Singh, Zaid Khan, Archiki Prasad, Justin Chih-Yao Chen, Akshay Nambi, Hyunji Lee, Elias Stengel-Eskin, and Mohit Bansal. Agent-brace: Decoupling beliefs from actions in long-horizon tasks via verbalized state uncertainty. *arXiv preprint arXiv:2605.11436*, 2026.
- [158] Qiwei Zhao, Xujiang Zhao, Yanchi Liu, Wei Cheng, Yiyu Sun, Mika Oishi, Takao Osaki, Katsushi Matsuda, Huaxiu Yao, and Haifeng Chen. Saup: Situation awareness uncertainty propagation on llm agent. *ArXiv*, abs/2412.01033, 2024. URL <https://api.semanticscholar.org/CorpusID:274437487>.
- [159] Eyke Hüllermeier and Willem Waegeman. Aleatoric and epistemic uncertainty in machine learning: an introduction to concepts and methods. *Machine Learning*, 110:457 – 506, 2019. URL <https://api.semanticscholar.org/CorpusID:216465307>.
- [160] Saurav Kadavath, Tom Conerly, Amanda Askell, Thomas Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zachary Dodds, Nova Dassarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, John Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom B. Brown, Jack Clark, Nicholas Joseph, Benjamin Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language models (mostly) know what they know. *ArXiv*, abs/2207.05221, 2022. URL <https://api.semanticscholar.org/CorpusID:250451161>.
- [161] Michel Cannarsa. Ethics guidelines for trustworthy ai. *The Cambridge Handbook of Lawyering in the Digital Age*, 2021. URL <https://api.semanticscholar.org/CorpusID:244427297>.

- [162] AI Nist, Secretary Gina M. Raimondo, and Laurie E. Locascio. Artificial intelligence risk management framework (ai rmf 1.0). 2023. URL <https://api.semanticscholar.org/CorpusID:261258266>.
- [163] Haochen Liu, Yiqi Wang, Wenqi Fan, Xiaorui Liu, Yaxin Li, Shaili Jain, Anil K. Jain, and Jiliang Tang. Trustworthy ai: A computational perspective. *ACM Transactions on Intelligent Systems and Technology*, 14:1 – 59, 2021. URL <https://api.semanticscholar.org/CorpusID:235829506>.
- [164] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. *ArXiv*, abs/1706.04599, 2017. URL <https://api.semanticscholar.org/CorpusID:28671436>.
- [165] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. *ArXiv*, abs/1610.02136, 2016. URL <https://api.semanticscholar.org/CorpusID:13046179>.
- [166] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Neural Information Processing Systems*, 2016. URL <https://api.semanticscholar.org/CorpusID:6294674>.
- [167] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. *ArXiv*, abs/1705.08500, 2017. URL <https://api.semanticscholar.org/CorpusID:491127>.
- [168] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. *arXiv preprint arXiv:2302.09664*, 2023.
- [169] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can llms express their uncertainty. *An empirical evaluation of confidence elicitation in LLMs*. *arXiv*, 2306, 2024.
- [170] Thomas Coste, Usman Anwar, Robert Kirk, and David Krueger. Reward model ensembles help mitigate overoptimization. *arXiv preprint arXiv:2310.02743*, 2023.
- [171] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024.
- [172] Alexander Nikitin, Jannik Kossen, Yarin Gal, and Pekka Marttinen. Kernel language entropy: Fine-grained uncertainty quantification for llms from semantic similarities. *Advances in Neural Information Processing Systems*, 37:8901–8929, 2024.
- [173] Daniel Yang, Yao-Hung Hubert Tsai, and Makoto Yamada. On verbalized confidence scores for llms. *arXiv preprint arXiv:2412.14737*, 2024.
- [174] Potsawee Manakul, Adian Liusie, and Mark John Francis Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. *ArXiv*, abs/2303.08896, 2023. URL <https://api.semanticscholar.org/CorpusID:257557820>.
- [175] Nico Wagner, Michael Desmond, Rahul Nair, Zahra Ashktorab, Elizabeth M. Daly, Qian Pan, Martín Santillán Cooper, James M. Johnson, and Werner Geyer. Black-box uncertainty quantification method for llm-as-a-judge. *ArXiv*, abs/2410.11594, 2024. URL <https://api.semanticscholar.org/CorpusID:273350620>.
- [176] Gwenth Portillo Wightman, Alexandra Delucia, and Mark Dredze. Strength in numbers: Estimating confidence of large language models by prompt agreement. In *Proceedings of the 3rd Workshop on Trustworthy Natural Language Processing (TrustNLP 2023)*, pages 326–362, 2023.
- [177] Ante Wang, Linfeng Song, Ye Tian, Baolin Peng, Lifeng Jin, Haitao Mi, Jinsong Su, and Dong Yu. Self-consistency boosts calibration for math reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 6023–6029, 2024.
- [178] Qing Lyu, Kumar Shridhar, Chaitanya Malaviya, Li Zhang, Yanai Elazar, Niket Tandon, Marianna Apidianaki, Mrinmaya Sachan, and Chris Callison-Burch. Calibrating large language models with sample consistency. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 19260–19268, 2025.
- [179] Zhen Lin, Shubhendu Trivedi, and Jimeng Sun. Generating with confidence: Uncertainty quantification for black-box large language models. *Trans. Mach. Learn. Res.*, 2024, 2023. URL <https://api.semanticscholar.org/CorpusID:258967487>.

- [180] Caiqi Zhang, Fangyu Liu, Marco Basaldella, and Nigel Collier. Luq: Long-text uncertainty quantification for llms. *ArXiv*, abs/2403.20279, 2024. URL <https://api.semanticscholar.org/CorpusID:268793903>.
- [181] Mingjian Jiang, Yangjun Ruan, Prasanna Sattigeri, Salim Roukos, and Tatsunori B. Hashimoto. Graph-based uncertainty metrics for long-form language model outputs. *ArXiv*, abs/2410.20783, 2024. URL <https://api.semanticscholar.org/CorpusID:273654396>.
- [182] Yashvir S. Grewal, Edwin V. Bonilla, and Thang Duc Bui. Improving uncertainty quantification in large language models via semantic embeddings. *ArXiv*, abs/2410.22685, 2024. URL <https://api.semanticscholar.org/CorpusID:273695256>.
- [183] Dang Nguyen, Ali Payani, and Baharan Mirzasoleiman. Beyond semantic entropy: Boosting llm uncertainty quantification with pairwise semantic similarity. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 4530–4540, 2025.
- [184] Jannik Kossen, Jiatong Han, Muhammed Razzak, Lisa Schut, Shreshth Malik, and Yarin Gal. Semantic entropy probes: Robust and cheap hallucination detection in llms. *arXiv preprint arXiv:2406.15927*, 2024.
- [185] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher D Manning. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5433–5442, 2023.
- [186] Stephanie Lin, Jacob Hilton, and Owain Evans. Teaching models to express their uncertainty in words. *arXiv preprint arXiv:2205.14334*, 2022.
- [187] Yuzi Yan, Xingzhou Lou, Jialian Li, Yiping Zhang, Jian Xie, Chao Yu, Yu Wang, Dong Yan, and Yuan Shen. Reward-robust rlhf in llms. *arXiv preprint arXiv:2409.15360*, 2024.
- [188] Adam X Yang, Maxime Robeyns, Thomas Coste, Zhengyan Shi, Jun Wang, Haitham Bou-Ammar, and Laurence Aitchison. Bayesian reward models for llm alignment. *arXiv preprint arXiv:2402.13210*, 2024.
- [189] Ahmed M Ahmed, Rafael Rafailov, Stepan Sharkov, Xuechen Li, and Sanmi Koyejo. Scalable ensembling for mitigating reward overoptimisation. *arXiv preprint arXiv:2406.01013*, 2024.
- [190] Ziwei Deng, Mian Deng, Chenjing Liang, Zeming Gao, Chennan Ma, Chenxing Lin, Haipeng Zhang, Songzhu Mei, Siqi Shen, and Cheng Wang. Planu: Large language model reasoning through planning under uncertainty. *Advances in Neural Information Processing Systems*, 38:142636–142673, 2026.
- [191] Juyeon Heo, Miao Xiong, Christina Heinze-Deml, and Jaya Narain. Do llms estimate uncertainty well in instruction-following? In *International Conference on Learning Representations*, volume 2025, pages 95951–95974, 2025.
- [192] Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision? *Advances in neural information processing systems*, 30, 2017.
- [193] Sewon Min, Julian Michael, Hannaneh Hajishirzi, and Luke Zettlemoyer. Ambigqa: Answering ambiguous open-domain questions. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pages 5783–5797, 2020.
- [194] Tong Zhang, Peixin Qin, Yang Deng, Chen Huang, Wenqiang Lei, Junhong Liu, Dingnan Jin, Hongru Liang, and Tat-Seng Chua. Clamber: A benchmark of identifying and clarifying ambiguous information needs in large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10746–10766, 2024.
- [195] Qian Xiong, Yuekai Huang, Ziyu Jiang, Zhiyuan Chang, Yujia Zheng, Tianhao Li, and Mingyang Li. Butterfly effects in toolchains: A comprehensive analysis of failed parameter filling in llm tool-agent systems. In *EMNLP (Findings)*, pages 16712–16729, 2025.
- [196] Han Bao, Zheyuan Zhang, Pengcheng Jing, Zhengqing Yuan, Kaiwen Shi, and Yanfang Ye. Drift-bench: Diagnosing cooperative breakdowns in llm agents under input faults via multi-turn interaction. *arXiv preprint arXiv:2602.02455*, 2026.

- [197] Yizhe Xie, Congcong Zhu, Xinyue Zhang, Tianqing Zhu, Dayong Ye, Minfeng Qi, Huajie Chen, and Wanlei Zhou. From spark to fire: Modeling and mitigating error cascades in llm-based multi-agent collaboration. *arXiv preprint arXiv:2603.04474*, 2026.
- [198] Ruipeng Wang, Yuxin Chen, Yukai Wang, Chang Wu, Junfeng Fang, Xiaodong Cai, Qi Gu, Hui Su, An Zhang, Xiang Wang, et al. Agentnoisebench: Benchmarking robustness of tool-using llm agents under noisy condition. *arXiv preprint arXiv:2602.11348*, 2026.
- [199] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *International conference on machine learning*, pages 1321–1330. PMLR, 2017.
- [200] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher Manning. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5433–5442, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.330. URL <https://aclanthology.org/2023.emnlp-main.330/>.
- [201] Suresh Raghu, Satwik Pandey, and Shashwat Pandey. Proper scoring rules for agentic uncertainty quantification. *arXiv preprint arXiv:2605.24756*, 2026.
- [202] Liancheng Yang, Sining Li, and Aobo Deng. Dynamic consensus communication mechanism for large language model-based multi-agent systems. *Journal of Signal Processing Systems*, 98, 2026. URL <https://api.semanticscholar.org/CorpusID:284713487>.
- [203] Lei Wang, Chengbang Ma, Xueyang Feng, Zeyu Zhang, Hao ran Yang, Jingsen Zhang, Zhi-Yang Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18, 2023. URL <https://api.semanticscholar.org/CorpusID:261064713>.
- [204] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. Understanding the planning of llm agents: A survey. *ArXiv*, abs/2402.02716, 2024. URL <https://api.semanticscholar.org/CorpusID:267411892>.
- [205] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.
- [206] Joon Sung Park, Joseph O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. URL <https://api.semanticscholar.org/CorpusID:258040990>.
- [207] Manh Hung Nguyen, Anh Hong Nguyen, Dung Nguyen, Svetha Venkatesh, and Hung Le. Hear both sides: Efficient multi-agent debate via diversity-aware message retention. *ArXiv*, abs/2603.20640, 2026. URL <https://api.semanticscholar.org/CorpusID:286762641>.
- [208] Blake R Duschatko, Jonathan Vandermause, Nicola Molinari, and Boris Kozinsky. Uncertainty driven active learning of coarse grained free energy models. *NPJ Computational Materials*, 10(1):9, 2024.
- [209] Fuhao Ji, Auralee Edelen, Ryan Roussel, Xiaozhe Shen, Sara Miskovich, Stephen Weathersby, Duan Luo, Mianzhen Mo, Patrick Kramer, Christopher Mayes, et al. Multi-objective bayesian active learning for mev-ultrafast electron diffraction. *Nature Communications*, 15(1):4726, 2024.
- [210] Derek Van Tilborg and Francesca Grisoni. Traversing chemical space with active deep learning for low-data drug discovery. *Nature Computational Science*, 4(10):786–796, 2024.
- [211] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 2016.
- [212] Tom Rainforth, Adam Foster, Desi R. Ivanova, and Freddie Bickford Smith. Modern Bayesian experimental design. *Statistical Science*, 2024.

- [213] B. P. MacLeod, F. G. L. Parlane, T. D. Morrissey, F. Häse, L. M. Roch, K. E. Dettelbach, R. Moreira, L. P. E. Yunker, M. B. Rooney, J. R. Deeth, V. Lai, G. J. Ng, H. Situ, R. H. Zhang, M. S. Elliott, T. H. Haley, D. J. Dvorak, A. Aspuru-Guzik, J. E. Hein, and C. P. Berlinguette. Self-driving laboratory for accelerated discovery of thin-film materials. *Science Advances*, 2020.
- [214] Benjamin Burger, Phillip M. Maffettone, Vladimir V. Gusev, Catherine M. Aitchison, Yang Bai, Xiaoyan Wang, Xiaobo Li, Ben M. Alston, Buyi Li, Rob Clowes, Nicola Rankin, Brandon Harris, Reiner Sebastian Sprick, and Andrew I. Cooper. A mobile robotic chemist. *Nature*, 2020.
- [215] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 2023.
- [216] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 2024.
- [217] Nathan J. Szymanski, Bernardus Rendy, Yuxing Fei, Rishi E. Kumar, Tanjin He, David Milsted, Matthew J. McDermott, Max Gallant, Ekin Dogus Cubuk, Amil Merchant, Haegyeom Kim, Anubhav Jain, Christopher J. Bartel, Kristin Persson, Yan Zeng, and Gerbrand Ceder. An autonomous laboratory for the accelerated synthesis of inorganic materials. *Nature*, 2023.
- [218] Kyle Swanson, Wesley Wu, Nash L. Bulaong, John E. Pak, and James Zou. The virtual lab of AI agents designs new SARS-CoV-2 nanobodies. *Nature*, 2025.
- [219] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Petar Sirkovic, Artiom Myaskovsky, Grzegorz Glowaty, Felix Weissenberger, Alessio Orlandi, Dan Popovici, Anil Palepu, Keran Rong, Ryutaro Tanno, Khaled Saab, Fan Zhang, Jacob Blum, Andrew Carroll, Kavita Kulkarni, Nenad Tomašev, Dina Zverinski, Ivor Rendulic, Elahe Vedadi, Florian Hasler, Luka Rimanic, Marina Boia, Ivan Budiselic, Ben Feinstein, Mathias Bellaiche, Tom Sheffer, Jan Freyberg, Jeremy Ratcliff, Ottavia Bertolli, Katherine Chou, Avinatan Hassidim, Burak Gokturk, Amin Vahdat, Yuan Guan, Vikram Dhillon, Eeshit Dhaval Vaishnav, Byron Lee, Tiago R. D. Costa, José R. Penadés, Gary Peltz, Yossi Matias, James Manyika, Demis Hassabis, Yunhan Xu, Pushmeet Kohli, Annalisa Pawlosky, Alan Karthikesalingam, and Vivek Natarajan. Accelerating scientific discovery with Co-Scientist. *Nature*, 2026.
- [220] Kexin Huang, Serena Zhang, Hanchen Wang, Yuanhao Qu, Yingzhou Lu, Ryan Li, Yusuf Roohani, Lin Qiu, Shiyi Cao, Gavin Li, Junze Zhang, Di Yin, Rick Wierenga, Deniz Kavi, Sherry Liu, Tianwei She, Shruti Marwaha, Jennefer N. Carter, Xin Zhou, Matthew T. Wheeler, Jonathan A. Bernstein, Mengdi Wang, Peng He, Jingtian Zhou, Michael P. Snyder, Le Cong, Aviv Regev, and Jure Leskovec. Autonomous biomedical research with an artificial intelligence agent. *Science*, 2026.
- [221] Nathan J. Szymanski, Bernardus Rendy, Yuxing Fei, Rishi E. Kumar, Tanjin He, David Milsted, Matthew J. McDermott, Max Gallant, Ekin Dogus Cubuk, Amil Merchant, Haegyeom Kim, Anubhav Jain, Christopher J. Bartel, Kristin Persson, Yan Zeng, and Gerbrand Ceder. Author correction: An autonomous laboratory for the accelerated synthesis of inorganic materials. *Nature*, 2026.
- [222] Josh Leeman, Yuhan Liu, Joseph Stiles, Scott B. Lee, Prajna Bhatt, Leslie M. Schoop, and Robert G. Palgrave. Challenges in high-throughput inorganic materials prediction and autonomous synthesis. *PRX Energy*, 2024.
- [223] Shiyang Duan, Yuan Tian, Qi Bing, and Xiaowei Shao. Bayes-entropy collaborative driven agents for research hypotheses generation and optimization. *arXiv preprint arXiv:2508.01746*, 2025.
- [224] Siyuan Qiu, Philip D Suh, Nhat Huy Tran, Xirui Wang, Heonjoon Park, Kutay Akin, Kevin KS Multani, Seungwon Jung, Wenkai Cai, Xinyu Liu, et al. Aims: An uncertainty-aware ai experimentalist for quantum matter. *arXiv preprint arXiv:2607.16544*, 2026.
- [225] Zehong Wang, Yijun Ma, Connor R. Schmidt, Tianyi Ma, Weixiang Sun, Ziming Li, Xiaoguang Guo, Chuxu Zhang, Matthew J. Webber, and Yanfang Ye. MDForge: Agentic molecular dynamics pipeline design under sparse simulator feedback. *arXiv preprint arXiv:2606.12916*, 2026.
- [226] Samir Abdaljalil, Erchin Serpedin, and Hasan Kurban. Knowing when not to answer: Abstention-aware scientific reasoning. *arXiv preprint arXiv:2602.14189*, 2026.

- [227] Josefa Lia Stoisser, Marc Boubnovski Martell, Lawrence Phillips, Gianluca Mazzoni, Lea Mørch Harder, Philip Torr, Jesper Ferkinghoff-Borg, Kaspar Martens, and Julien Fauqueur. Towards agents that know when they don't know: Uncertainty as a control signal for structured reasoning. *arXiv preprint arXiv:2509.02401*, 2025.
- [228] Bojana Ranković, Ryan-Rhys Griffiths, and Philippe Schwaller. Large language models as uncertainty-calibrated optimizers for experimental discovery. *arXiv preprint arXiv:2504.06265*, 2025.
- [229] Steven Young Eulig. Position: Correct answer, wrong mechanism – when AI scientists defend general claims their own data contradicts. *arXiv preprint arXiv:2606.23175*, 2026.
- [230] Tianyu Ding, Aditya Nannapaneni, Bingfan Liu, and Ling Zhang. Autonomous research agents: A survey of AI scientists and the verification gap. *arXiv preprint arXiv:2608.05179*, 2026.
- [231] Zizhang Chen, Peizhao Li, Xiaomeng Dong, and Pengyu Hong. Uncertainty quantification for clinical outcome predictions with (large) language models. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 7527–7538, 2025.
- [232] Xueyang Li, Mingze Jiang, Gelei Xu, Jun Xia, Mengzhao Jia, Danny Chen, and Yiyu Shi. At-cxr: Uncertainty-aware agentic triage for chest x-rays. *arXiv preprint arXiv:2508.19322*, 2025.
- [233] Liuxin Bao, Zhihao Peng, Xiaofei Zhou, Runmin Cong, Jiyong Zhang, and Yixuan Yuan. Expertise-aware multi-llm recruitment and collaboration for medical decision-making. *arXiv preprint arXiv:2508.13754*, 2025.
- [234] Farhad Nooralahzadeh, Omid Rohanian, Yi Zhang, Jonathan Fürst, and Kurt Stockinger. Arbitration failure, not perceptual blindness: How vision-language models resolve visual-linguistic conflicts. *arXiv preprint arXiv:2604.09364*, 2026.
- [235] Alessandro Favero, Luca Zancato, Matthew Trager, Siddharth Choudhary, Pramuditha Perera, Alessandro Achille, Ashwin Swaminathan, and Stefano Soatto. Multi-modal hallucination control by visual information grounding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14303–14312, 2024. *arXiv:2403.14003*.
- [236] Ruiyang Zhang, Hu Zhang, and Zhedong Zheng. VI-uncertainty: Detecting hallucination in large vision-language model via uncertainty estimation. *arXiv preprint arXiv:2411.11919*, 2024.
- [237] Khang Nhat Hoang Vo, Artem Vazhentsev, Artem Shelmanov, Timothy Baldwin, and Yova Kementchedjheva. Does it fail to see or fail to know? attributing errors in vision-language models. *arXiv preprint arXiv:2607.04683*, 2026.
- [238] Neel P Bhatt, Yunhao Yang, Rohan Siva, Daniel Milan, Ufuk Topcu, and Zhangyang Wang. Know where you're uncertain when planning with multimodal foundation models: A formal framework. *MLSys*, 2025.
- [239] Shelly Francis-Meretzki, Mirco Mutti, Yaniv Romano, and Aviv Tamar. Temporal difference calibration in sequential tasks: Application to Vision-Language-Action models. In *ICML*, 2026.
- [240] Hyeonbeom Choi, Daechul Ahn, Youhan Lee, Taewook Kang, Seongwon Cho, and Jonghyun Choi. SCALE: Self-uncertainty conditioned adaptive looking and execution for Vision-Language-Action models. In *ICML*, 2026.
- [241] Ralf Römer, Maximilian Seeliger, Saida Liu, Ben Sturgis, Marco Bagatella, Daniel Marta, Andreas Krause, and Angela P Schoellig. Uncertainty quantification for flow-based vision-language-action models. *arXiv preprint arXiv:2606.18043*, 2026.
- [242] Allen Z. Ren, Anushri Dixit, Alexandra Bodrova, Sumeet Singh, Stephen Tu, Noah Brown, Peng Xu, Leila Takayama, Fei Xia, Jake Varley, Zhenjia Xu, Dorsa Sadigh, Andy Zeng, and Anirudha Majumdar. Robots that ask for help: Uncertainty alignment for large language model planners. In *CoRL*, 2023.
- [243] Ram Ramrakhya, Matthew Chang, Xavier Puig, Ruta Desai, Zsolt Kira, and Roozbeh Mottaghi. Grounding multimodal llms to embodied agents that ask for help with reinforcement learning. *arXiv preprint arXiv:2504.00907*, 2025.
- [244] Temitayo Olamilekan Ogunsusi, Lijun Qian, and Xishuang Dong. Ua-chatdev: Uncertainty-aware multi-agent collaboration for reliable software development. *arXiv preprint arXiv:2607.02186*, 2026.

- [245] Georgii Andriushchenko, Roman Garaev, Lyudmila Rvanova, Artem Shelmanov, and Vladimir V Ivanov. Efficient hallucination detection in automatic code generation. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 43197–43220, 2026.